

Coverage versus Selectivity in Pre-Export Row Scaling for Generated Mixed-Integer Programs

Mathias Rodríguez Castro

¹Facultad de Ingeniería, Universidad de la República, Montevideo, Uruguay

Correspondence

Mathias Rodríguez Castro, Facultad de Ingeniería, Universidad de la República, Montevideo, Uruguay.

Email: mathiasr@fing.edu.uy (ORCID: 0009-0002-7235-7677)

Abstract

Generated mixed-integer programs (MIPs) reach solvers as flat coefficient matrices, although their generators retain row roles, block ownership, and coupling incidence. We study whether this metadata should select which constraints receive positive pre-export row scaling. The proposed local–block–coupling layer applies geometric-mean row normalization while preserving the MIP exactly; returned solutions are verified on the original constraints. We evaluate it on three hydrothermal-dispatch classes, two commercial solvers, and two MIPGap tolerances, against unscaled, Ruiz-kernel, solver-internal, and role-blind full-coverage controls. The structure-aware variants generally reduce own-incumbent fixed-LP diagnostics, but a synthetic mechanism check finds no general exported-range advantage over applying the same per-row kernel to every row. Under fixed-pool intent-to-treat PAR10, at least one structure-aware variant is below Base in every Gurobi class and tolerance, whereas no robust gain appears under CPLEX. Direct exploratory Gurobi controls show that the coverage ordering depends on class, kernel, and endpoint. Capped solver-time effects are small: only the augmented Simple contrast at 0.1% has nominal adjusted values below 0.05 across the reported multiplicity and operational-family analyses, while reliability-adjusted PAR10 favors the selective rule there. The current prototype has lower preprocessing-plus-solver time under full coverage in the two fast classes, but most of that preprocessing difference comes from rebuilding metadata the generator already possesses; a cached-metadata sensitivity on a release build moves the effects toward parity and, under the baseline cross-protocol construction, leaves one of the eight fast-class contrasts with a within-tolerance nominal adjusted value below 0.05. In Full, the selective direction is not supported by the adversarial signed-rank sensitivity over the unrecorded pairs. The observed effect of external preprocessing differs between the tested Gurobi and CPLEX pipelines, though no formal cross-solver interaction is estimated. Overall, metadata-driven selectivity is not uniformly superior, and improved numerical diagnostics need not translate into faster MIP search.

KEYWORDS

Numerical scaling, mixed-integer programming, generated formulations, block-structured optimization, engineering optimization, hydrothermal dispatch

PRACTITIONER POINTS

- Model generators know each row’s role, block ownership, and coupling incidence—metadata a flat export discards but that can drive numerical preprocessing.
- Positive, structure-keyed row factors applied in a local–block–coupling order lowered the reported own-incumbent path-specific fixed-LP diagnostic without changing the MIP; on an already well-scaled model the selective rule can widen the exported range instead.
- In exploratory single-default-seed Gurobi control batches, coverage versus selectivity depends on the endpoint. On preprocessing-plus-solver time the implemented role-blind pipeline has lower observed time in Simple and SG-Ter-Mer at both tolerances (a margin driven mainly by the current prototype’s metadata-processing cost in Simple and by a mixture

of that cost and solver response in SG-Ter-Mer, and worth tens of milliseconds per instance in absolute terms); the Full selectivity direction is conditional on the 27 eligible pairs.

- When the selective rule is charged only for diagnosis and factor application, as in a generator that reuses its structural taxonomy, most of that observed Flat timing advantage disappears: six directional orderings remain, but on a release build—under the baseline cross-protocol construction—only one contrast (Simple, augmented, 0.1%; about 0.9% in median) keeps a within-tolerance nominal adjusted value below 0.05. On capped solver time the medians are at most 1.6% and the augmented Simple contrast at 0.1% is the only comparison whose nominal adjusted values remain below 0.05 across the reported multiplicity and operational-family analyses; all coefficient-only comparisons remain inconclusive.
- Under fixed-pool intent-to-treat PAR10, at least one structure-aware variant is below Base in every Gurobi class and tolerance. SG-Ter-Mer at 0.1% is the only cell with both strict-adjusted costs below Base, but the SA-Aug margin is negligible and the pattern is not replicated at 1%; no robust gain appears under CPLEX. In the direct SA-Flat reliability audit, the Simple strict/mixed aggregate ordering reverses toward SA-Aug, while SG-Ter-Mer retains only a small descriptive Flat-Aug advantage. Cross-solver persistence of the Flat-versus-selective ordering remains untested.

1 | INTRODUCTION

A model generator and a solver see different representations of the same optimization problem. The generator builds the model from components: hydro plants, thermal units, market rules, resource limits, demand balances, and discrete operating decisions. While doing so, it knows which rows are local to one component, which variables that component owns, and which constraints link several components. The solver receives the exported version: rows, columns, coefficients, bounds, and tolerances. In that exported matrix, much of the generator’s structural knowledge has disappeared.

This loss matters because numerical representation affects computation. Two formulations can describe the same feasible set and objective, yet lead a solver through different presolve reductions, relaxation bases, cuts, incumbent searches, branching decisions, and feasibility checks. Generic scaling routines can inspect coefficient magnitudes after export. They usually cannot tell whether a row is a local turbine restriction, a thermal limit, or a system-wide balance. The generator can make that distinction before export. Mainstream modeling environments—Pyomo, JuMP, GAMS, AMPL—are exactly such generators: they assemble the model component by component but, in their standard export paths, hand the solver a flat matrix that no longer marks which rows are local, which variables a component owns, or which rows couple components. Block-structured extensions of these ecosystems do transport such structure—graph-based and block-decomposition modeling layers, and solver-side decomposition files—but they feed it to decomposition algorithms rather than to numerical preprocessing, which is the gap addressed here.

This is not made obsolete by solver-internal scaling. Commercial solvers already contain sophisticated numerical preprocessing, and the experiments below leave it enabled; the question here is complementary—whether an external layer can supply structural numerical information before the solver sees the matrix as an anonymous list of rows and columns. A positive answer would not replace solver scaling, but change the representation on which the solver’s pipeline starts.

The resulting question is simple: can generator-level block information guide positive row scalings that improve the solver-facing representation without changing the MIP? The question is not whether positive row scaling is valid. Multiplying a complete constraint by a strictly positive scalar preserves the feasible set. The question is whether the scaling factors should depend on information that the generator still has: local rows, component blocks, and coupling rows. In this sense, the paper treats pre-export metadata as numerical information rather than only as modeling bookkeeping.

We answer this question with a structure-aware row-scaling framework. The method scales only whole constraints, including their right-hand sides. Hence it preserves the original mixed-integer optimization problem. Its novelty lies in the order and source of the scales, not in claiming that row scaling is new. The generator first scales local component rows. It then summarizes the scale of each locally scaled block. Finally, it scales global coupling rows with the scales of the blocks they connect. Throughout the paper, this local–block–coupling architecture is treated as a *candidate* structure-aware policy whose stages, coverage, and solver dependence are evaluated—and partly refuted—empirically.

The contribution is not a new equilibration kernel—the per-row factor is a known geometric-mean normalization, and the results show that this kernel carries most of the effect. The central contribution is instead a controlled study of *coverage versus selectivity*: whether restricting a known per-row scaling kernel to the rows a generator’s role metadata selects helps or hurts, relative to applying it to every row. Concretely, the paper contributes: (i) a controlled, single-default-seed empirical study of that coverage-versus-selectivity trade-off—via augmented and coefficient-only role-blind controls run on all three classes

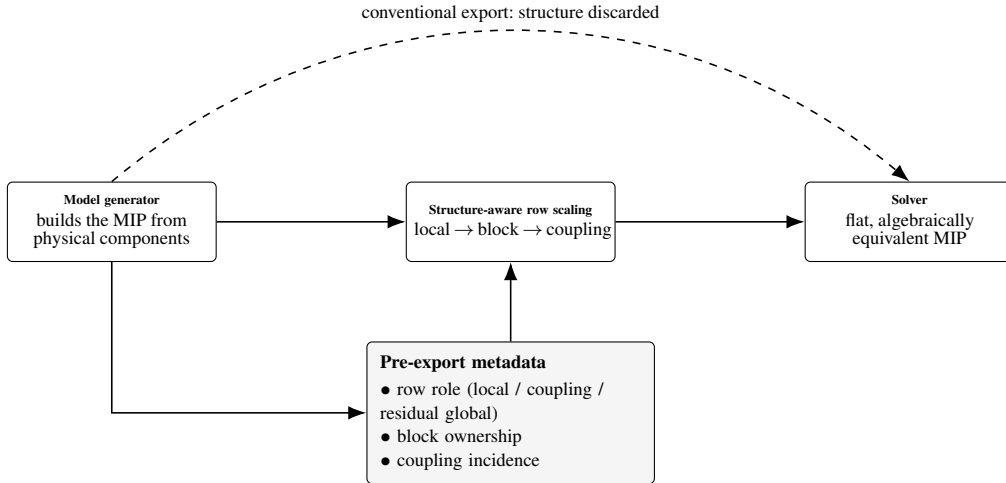


FIGURE 1 The proposed layer in one picture. A generator retains each row’s role—local to a component, coupling between components, or residual global (global but not coupling)—block ownership, and coupling incidence before export. A conventional export (dashed) hands the solver a flat matrix in which this structure is lost. The structure-aware layer instead uses this metadata to assign positive row factors in a local–block–coupling order and exports an *algebraically equivalent* flat MIP; residual-global rows are left unchanged by the selective policy. The solver still receives a single flat matrix; only its numbers change. (The role-blind flat control and the coverage cost of this selectivity are analyzed in [section 6.8](#).)

under Gurobi—showing that an incomplete structural taxonomy can miss numerically dominant rows and that the evidence for a selection policy depends on class and kernel; (ii) an algebraically exact, auditable row-scaling layer for generated block-structured MIPs, with the returned solutions verified on the original, unscaled constraints; (iii) the use of generator metadata as a *selection policy* that decides which rows to normalize, with idealized conditioning arguments motivating the local–block–coupling order; and (iv) an explicit separation of numerical improvement from computational acceleration, demonstrating that the two need not coincide. [Figure 1](#) summarizes the proposed layer at a glance.

The numerical rationale is also local–block–coupling. In an ideal block-diagonal matrix, scalar block scaling can reduce scale separation between blocks but cannot repair a block’s intrinsic conditioning; once coupling rows are added, their relevant magnitude is their size relative to the scaled local blocks, not their raw coefficients. Idealized conditioning results make this precise and justify the rule’s architecture, without claiming to predict mixed-integer runtime.

We test the method on short-term hydrothermal dispatch, used not as a new formulation but as a demanding testbed: it creates local hydro, thermal, market, shortage, and demand modules linked by global balances, with coefficients mixing water volumes, turbine flows, power outputs, costs, and penalties—block structure with strong scale heterogeneity. The intended generality, however, is the generated block-structured MIP, not the hydrothermal equations. The layer requires only metadata that many modular generators already have—which rows are local, which block owns a variable or constraint, which global rows couple blocks—while reservoir dynamics, turbine curves, and market costs determine the instances but never enter the scaling rule. The hydrothermal model thus supplies a realistic stress test for a layer that targets any generated MIP with local components and sparse global coupling.

The hydrothermal formulation used in the experiments descends from models for the Río Negro hydroelectric complex and extensions to hydrothermal dispatch with Salto Grande, developed by Risso and coauthors, together with a modular C++ implementation used as the experimental platform^{1,2,3}. We take the formulation as given and insert the proposed preprocessing layer after model generation and before solver invocation.

The paper makes three claims of different strength: the algebraic claim is exact (positive row scaling preserves the MIP); the numerical claim is empirical (structure-aware scaling lowers own-incumbent path-specific fixed-LP diagnostics in the tested instances); and the performance claim is conditional. Under fixed-pool intent-to-treat PAR10, at least one structure-aware variant is below Base in each Gurobi class and tolerance, while under CPLEX 22.1.2.0 there is no robust gain. SG-Ter-Mer at 0.1% is the only cell in which both variants remain below Base after strict reliability adjustment; this is a descriptive penalized-cost ordering, the SA-Aug margin is negligible, and it is not replicated at 1%. The controlled attribution analyses sharpen the

result: most of the Gurobi runtime effect flows through per-row normalization, while the direct Flat comparisons make the role-based selection policy class-, kernel-, and endpoint-dependent in the reported Gurobi campaigns. The path-specific diagnostic moves in the intended numerical direction, but it is neither a pure measure of representation nor a proven cause of acceleration. The rest of the paper follows this chain of evidence. [Section 2](#) identifies the research niche. [Section 3](#) defines the generated model structure and testbed. [Section 4](#) gives the scaling method. [Section 5](#) explains how the experiments test the claims. [Section 6](#) reports numerical and runtime evidence. [Section 7](#) interprets that evidence and answers likely objections. [Section 8](#) states the main limitations and reproducibility implications, and [section 9](#) summarizes the contribution.

2 | RELATED WORK

The literature sets the stage but leaves a specific niche. Hydrothermal-dispatch models provide realistic generated block-structured MIPs with heterogeneous units and global balances; numerical-scaling work explains why coefficient magnitudes matter, but its standard tools operate on the exported matrix; and formulation-quality practice shows that algebraic representation can matter as much as the underlying set. The missing argument is narrower than “scaling is useful”: when a generator still knows each row’s role and each variable’s block, that metadata can be used as numerical information before export. The resulting transformation must then be evaluated separately as an algebraic equivalence, a representation change, and a solver-dependent performance intervention.

2.1 | Hydrothermal dispatch and MIP formulations

Hydrothermal dispatch has been studied with dynamic programming, stochastic dual dynamic programming, nonlinear programming, and mathematical programming. Classical references emphasize the trade-off between thermal generation costs, hydro opportunity costs, demand satisfaction, and intertemporal reservoir management^{4,5}. In short-term settings, MIP formulations are attractive because they incorporate discrete decisions, piecewise-linear approximations, local operating constraints, and system-wide balance equations in a single model⁶.

This literature motivates the experimental domain but not the novelty claim. The dispatch model is used because it is a representative generated block-structured MIP: local physical modules are created separately, their variables have heterogeneous units, and a smaller set of global rows links the modules through system-wide balances. These are precisely the structural features exploited by the preprocessing layer.

2.2 | Dispatch models used in this study

The computational experiments use a reference formulation from a research line on MIP-based dispatch models for the Uruguayan power system. Risso et al. introduced a mixed combinatorial optimization model for the Río Negro hydroelectric complex¹ and extended the modeling line to hydrothermal dispatch for the Uruguay River hydroelectric plant, with emphasis on Salto Grande and coupled hydrothermal operation², and an enriched formulation of the Río Negro model, the closest published reference for the dispatch features used by the Full class⁷. The implementation used here builds on a modular C++ version of the Río Negro dispatch model³.

This provenance fixes the scope of the experiments. The dispatch formulation, data pipeline, and physical components are taken as given. The proposed method acts after the model has been generated and before the solver is called. Consequently, any observed difference between variants is attributed to an algebraically equivalent representation of the same generated MIP, not to a modified dispatch model.

2.3 | Numerical scaling and solver-facing representation

Scaling is a standard preprocessing operation in linear and mixed-integer optimization. Multiplying a constraint by a positive factor preserves the feasible region but changes the numerical magnitudes passed to the solver. These magnitudes can affect feasibility tolerances, relaxation stability, presolve transformations, cut generation, basis quality, and the numerical behavior

of LP relaxations. Practical guidelines for difficult LP and MIP models warn that large coefficient ranges and poor scaling can harm solver performance even when the mathematical formulation is correct^{8,9}.

This paper uses that observation in a restricted way: it does not claim coefficient compression is itself an objective, nor that a smaller coefficient range must speed up branch-and-bound. A MIP solver is a pipeline—presolve, internal scaling, root relaxation, cuts, heuristics, incumbents, branching, node processing, termination—whose stages can respond differently to the same external scaling. The experiments therefore distinguish exported coefficient-range ratios from solver-level conditioning diagnostics and from runtime, a distinction central to interpreting the Gurobi–CPLEX contrast.

2.4 | Generic equilibration and its limitation for generated models

Generic equilibration procedures balance rows and columns according to matrix norms. This is a mature subfield of numerical linear algebra. Geometric-mean and arithmetic-mean (ℓ_∞) scaling normalize each row or column by an average of its coefficient magnitudes¹⁰; Curtis–Reid scaling chooses logarithmic diagonal factors that minimize a least-squares measure of coefficient spread¹¹; Sinkhorn–Knopp iteration drives a nonnegative matrix toward a doubly stochastic form by alternating row and column normalization¹²; and Ruiz-type scaling iteratively equilibrates row and column norms through diagonal transformations¹³. These methods are the standard tools and are already embedded in solver presolve and internal scaling¹⁴. Their computational payoff, however, is known to be irregular: the largest empirical study of LP scaling to date found that no scaling method consistently improves—and can even degrade—simplex performance across solvers¹⁵, an early warning that coefficient compression alone is not the lever. Such methods are useful because they are model-agnostic and can be applied after export. That strength is also their limitation in the setting considered here. A flat-matrix method can see coefficient magnitudes and sparsity, but it is not told whether a row is a local turbine restriction, a thermal bound, a market-cost relation, a shortage constraint, or a system-wide balance.

Against this background, the proposed rule is not a new equilibration kernel: its local row factor is, in isolation, a geometric-mean normalization of the row’s nonzero coefficient magnitudes, optionally augmented with the right-hand side (the augmented SA-Aug kernel includes $|b_r|$ among the extremes; the matrix-only SA-Mat kernel does not). What is new is *where the scales come from and in what order*—the factors are keyed to generator metadata (row role, block ownership, coupling incidence) rather than the flat matrix, and applied in a local–block–coupling order so coupling rows are normalized relative to the already-scaled blocks they connect, not their raw coefficients. A flat method (Ruiz, Curtis–Reid, geometric-mean, Sinkhorn–Knopp) cannot make this local-versus-coupling distinction, because role information is exactly what export discards. Concretely, take two rows with identical coefficients: a flat method cannot tell them apart and scales them identically, whereas the structure-aware rule need not. If one is a local block row and the other a coupling row, the local row is scaled by its own coefficients while the coupling row is scaled relative to the *already-scaled* blocks it links—so a coupling row with small raw coefficients can still receive a large factor when it joins a poorly scaled block, and conversely. Identical numbers, different roles, different factors: that is the distinction a flat matrix cannot expose—though, as [section 6.8](#) shows, this representational capability is not itself the runtime lever. The model stays monolithic, as with generic scaling: an external, auditable layer that uses generator metadata before the exported matrix loses it.

2.5 | Formulation quality, generated structure, and decomposition

The MIP literature also stresses that formulation quality matters: two algebraically related formulations can describe the same modeling intention but lead to different relaxations, numerics, and search behavior⁶. The present work is adjacent to that tradition but changes a different object. It does not strengthen the formulation, add valid inequalities, alter variables, or change the feasible set. It changes only the positive row multipliers applied to complete constraints.

The method is also adjacent to structural decomposition. Like Dantzig–Wolfe, Benders, or Lagrangian approaches, it recognizes local blocks and global linking rows. Unlike those methods, it does not decompose the optimization problem, introduce a master problem, or exchange prices or cuts between subproblems. The solver still receives one monolithic MIP. The use of block structure is numerical rather than algorithmic decomposition: block metadata guide the representation presented to the solver.

2.6 | Remaining niche

The resulting contribution is deliberately narrow: it tests whether local rows, block scales, and coupling rows should be treated as different numerical objects before a generated block-structured MIP is exported. The performance comparison rests on distributional evidence over heterogeneous instances rather than a single runtime average, using performance profiles¹⁶, paired tests, deterministic-effort metrics, and solver-level conditioning diagnostics.

3 | MATERIALS AND METHODS: GENERATED MODULAR FORMULATION AND TESTBED

We now define the objects that the scaling rule changes and the objects it must leave unchanged. The generator supplies the block structure; the preprocessing layer changes the row magnitudes; the solver then receives an algebraically equivalent MIP. The dispatch model itself is not new. What matters here is the structure that the generator exposes and the original scale in which all feasibility and objective checks are evaluated after the solve. We consider generated modular MIP instances formulated as

$$\begin{aligned} \min_x \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b, \\ & x_j \in \mathbb{Z}, \quad j \in \mathcal{I}, \\ & x_j \in \mathbb{R}, \quad j \notin \mathcal{I}. \end{aligned} \tag{1}$$

In the general generated setting, the vector x collects all variables created by the modules, while A and b contain both local component constraints and global linking constraints. In the hydrothermal testbed used here, x includes generation, reservoir, flow, spillage, shortage, and operating variables. Equalities are represented either explicitly or by pairs of inequalities, depending on the solver interface.

The notation is intentionally application-agnostic: the layer uses the generator’s structural map—row role, block ownership, coupling incidence—not hydrological, thermal, or economic semantics. The hydrothermal variables above explain the physical origin of the coefficients; they are not assumptions of the scaling method.

3.1 | Hydrothermal dispatch testbed and scope

The computational testbed uses the reference model generated by the modular dispatch platform developed around Río Negro and Salto Grande hydrothermal dispatch models^{1,2,3}. The application is useful because it exhibits the target structure: local modules with heterogeneous physical units and global rows that couple those modules through system-wide balances. The hydraulic, thermal, and market-cost constraints are those of the reference model and its implementation.

A caveat about those heterogeneous units is in order, because it is tempting to read the imbalance as artificial. Unit heterogeneity (reservoir volumes in hm^3 , flows in m^3/s , power in MW) lives on the *columns*, and rescaling units is, mathematically, column scaling: an exact reformulation for continuous variables, whereas arbitrary continuous column scaling does not in general preserve the standard integer lattice unless the variable domains are transformed consistently, so a straightforward unit normalization of a MIP is in practice partial. Our column-normalizing baseline (Ruiz+Cols) is the weakest variant in runtime, and the base is no ill-scaled straw man: it already runs under the solver’s own internal equilibration (section 6.7). In the tested configurations, the row-based variants were therefore more effective than the particular column-inclusive baseline considered here; whether a better-designed column or unit normalization could compete is untested, and the row-only scoping decision is revisited as a limitation in section 8.

The method acts on the generated algebraic model, not the physical dispatch formulation: every transformation must preserve the feasible set and let objective values, constraint violations, and operational variables be read in the original scale after solution. It never uses a hydro-specific rule (reservoir priority, turbine efficiency, unit commitment) to choose factors—those details generate the rows, and only their structural role in the block-coupled MIP is used afterward.

3.2 | Block structure

The method targets models generated by components or agents. Each component $i \in \mathcal{B}$ creates local variables x_i , contributes local constraints $A_i x_i \leq b_i$, and appears in a smaller number of global constraints. Let $\mathcal{R}_i$ denote the local row index set of component i . In the hydrothermal testbed, these components include hydro, thermal, market, and demand modules. If u_{loc} stacks all local variables and g denotes an aggregate coupling variable, the row structure can be written schematically as

$$\begin{bmatrix} D & 0 \\ 0 & I_T \\ L & -I_T \end{bmatrix} \begin{bmatrix} u_{\text{loc}} \\ g \end{bmatrix} \sim \begin{bmatrix} b \\ d \\ 0 \end{bmatrix}, \quad D = \text{diag}(A_1, \dots, A_B). \quad (2)$$

The first block contains local component restrictions. The second block represents an aggregate system restriction. The third block contains coupling equations that link local component decisions with the aggregate system variable. In the dispatch application, these rows correspond to demand and balance relationships. More generally, absorbing the aggregate variable and its defining rows into the coupling block, the coefficient matrix takes the block form

$$A = \begin{bmatrix} A_1 & 0 & \cdots & 0 \\ 0 & A_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & A_B \\ L_1 & L_2 & \cdots & L_B \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_B \\ h \end{bmatrix}. \quad (3)$$

The matrices A_i encode local restrictions. The matrices L_i encode each block's contribution to the global coupling rows, such as demand balance in the hydrothermal testbed.

Row roles and block incidence are separate metadata fields, and the row taxonomy splits the complete row set $\mathcal{R}_{\text{all}}$ into three pairwise disjoint classes: the local sets $\mathcal{R}_i$, the coupling set $\mathcal{R}_{\text{cpl}}$, and a residual set $\mathcal{R}_{\text{other}}$ of rows that are global but not coupling,

$$\mathcal{R}_{\text{all}} = \left(\bigcup_{i \in \mathcal{B}} \mathcal{R}_i \right) \dot{\cup} \mathcal{R}_{\text{cpl}} \dot{\cup} \mathcal{R}_{\text{other}}, \quad \mathcal{R}_i \cap \mathcal{R}_j = \emptyset \ (i \neq j). \quad (4)$$

A coupling row is incident to several blocks through the variables it involves, but it is not classified as a local row of any of them and does not enter the local-row normalization stage; the implementation enforces this by removing role-classified global and coupling rows from every block row set before scaling. The residual class $\mathcal{R}_{\text{other}}$ (for example, the market shortage segments of the testbed) is a genuine design decision, not an afterthought: a policy must be chosen for it—leave it unscaled, normalize it row by row with the same kernel, or diagnose it and normalize only on a trigger. The structure-aware rules studied here take the most selective policy (leave $\mathcal{R}_{\text{other}}$ unscaled), whereas the role-blind Flat controls in [section 6.8](#) realize the opposite extreme (normalize every row); the operational comparison between them measures the cost of this choice for each kernel and class under Gurobi ([table 18](#)).

This structure suggests three scaling decisions: scale local rows inside each block; compute representative block scales after local row scaling; and scale global coupling rows with those block scales. The goal is to reduce harmful scale heterogeneity without erasing the modular meaning of the formulation.

3.3 | Algebraic equivalence and spectral diagnostics

We restrict the preprocessing layer to positive row scalings. This restriction makes the transformation auditable: variables keep their original units, and the generated MIP remains the same optimization problem. Let $D_r = \text{diag}(d_1, \dots, d_m)$, with $d_r > 0$ for every row. We replace (A, b) by

$$\tilde{A} = D_r A, \quad \tilde{b} = D_r b. \quad (5)$$

For every vector x ,

$$Ax \leq b \iff D_r Ax \leq D_r b, \quad (6)$$

because each component of the inequality is multiplied by a strictly positive scalar. Thus positive row scaling preserves the feasible set, the integer restrictions, and the objective function. Any difference observed after preprocessing must therefore come from the numerical representation seen by the solver, not from a different mathematical MIP.

The theoretical arguments below use the spectral condition number as a diagnostic of linear-algebraic sensitivity. For a real matrix M , define

$$\kappa_2(M) = \frac{\sigma_{\max}(M)}{\sigma_{\min}^+(M)}, \quad (7)$$

where $\sigma_{\max}(M)$ is the largest singular value and $\sigma_{\min}^+(M)$ is the smallest positive singular value. For rectangular matrices, this ratio is used only as a diagnostic over the nonzero singular spectrum; in the square benchmarks below the minimum is taken over the full spectrum, so that singularity is reported as an infinite condition number. This diagnostic does not characterize the full difficulty of a MIP. A MIP solver also depends on presolve, cuts, branching, primal heuristics, incumbent discovery, feasibility tolerances, and integrality. The role of κ_2 is narrower: it provides an idealized matrix argument explaining why local rows and coupling rows should not be scaled in exactly the same way.

3.4 | Ideal block scaling

The block structure is not only a modeling convenience. It identifies what scalar scaling can and cannot repair. In a purely block-diagonal matrix $D = \text{diag}(A_1, \dots, A_B)$, a single scalar per block can remove differences of scale between blocks, but it cannot improve the intrinsic conditioning of an individual block. The following benchmark makes this statement precise in the ideal square nonsingular case.

Proposition 1 (Ideal scalar scaling of block-diagonal matrices). *Let $D = \text{diag}(A_1, \dots, A_B)$, where every A_i is square and nonsingular. Let $M_i = \sigma_{\max}(A_i)$ and $m_i = \sigma_{\min}(A_i) > 0$. Then*

$$\min_{\alpha_i > 0} \kappa_2(\text{diag}(\alpha_1 I, \dots, \alpha_B I) D) = \max_i \kappa_2(A_i). \quad (8)$$

One minimizer is

$$\alpha_i = \frac{1}{\sqrt{M_i m_i}} = \frac{1}{\sqrt{\sigma_{\max}(A_i) \sigma_{\min}(A_i)}}. \quad (9)$$

The proof, which constructs the optimal scalar per block and evaluates the resulting spectrum, is given in the Supporting Information (Section S1).

This proposition is not used as an algorithmic recipe. The generated MIPs considered in this paper contain rectangular local matrices, inequalities, bounds, redundant rows, and integer variables; computing singular-value-based block scalings would not be a practical external preprocessing rule for the exported MIP. The proposition instead justifies the architecture of the method. First, poor scale separation between blocks should be treated differently from poor conditioning inside a block. Second, after local row scaling, each block should contribute a representative scale before global coupling rows are processed. The implemented rules therefore replace the ideal singular-value quantities (M_i, m_i) with coefficient-based proxies that are cheap to compute from the generated model. These propositions are design principles, not performance theorems. They should not be read as predictive guarantees for the implemented MIP preprocessing rule; they motivate the distinction between local rows, block scales, and coupling rows.

3.5 | Effect of global coupling rows

Local block scaling does not exhaust the numerical issue because the generated model is not purely block diagonal. In the hydrothermal dispatch testbed, global rows couple local agent decisions through demand and balance equations. The relevant schematic block is

$$A_{\text{bal}} = \begin{bmatrix} D & 0 \\ L & -I_T \end{bmatrix}, \quad (10)$$

where $D = \text{diag}(A_1, \dots, A_B)$ represents the local blocks and L collects the coefficients by which local decisions enter the global balance. If D is nonsingular, then

$$A_{\text{bal}} = \begin{bmatrix} I & 0 \\ LD^{-1} & -I_T \end{bmatrix} \begin{bmatrix} D & 0 \\ 0 & I_T \end{bmatrix}. \quad (11)$$

This factorization separates local scale from coupling scale. The matrix L by itself is not the relevant object: the coupling felt by the balance row after the local block scales are taken into account is LD^{-1} . Thus a row with moderate entries in L can still be numerically influential if some local block contains weak directions.

The next result makes the dependence on the effective coupling norm explicit.

Proposition 2 (Conditioning of a triangular coupling factor). *Let $R \in \mathbb{R}^{m \times n}$ and define*

$$T_R = \begin{bmatrix} I_n & 0 \\ R & I_m \end{bmatrix}. \quad (12)$$

If $\rho = \|R\|_2$, then

$$\kappa_2(T_R) = \frac{\rho^2 + 2 + \rho\sqrt{\rho^2 + 4}}{2} = \left(\frac{\sqrt{\rho^2 + 4} + \rho}{2} \right)^2. \quad (13)$$

The sign of the identity block is immaterial: since $\begin{bmatrix} I & 0 \\ R & -I \end{bmatrix} = \text{diag}(I, -I) \begin{bmatrix} I & 0 \\ -R & I \end{bmatrix}$ and orthogonal factors preserve singular values, the same formula applies to the factor with $-I_T$ in [equation \(11\)](#).

The proof, which bounds the effect of the coupling factor on the extreme singular values, is given in the Supporting Information (Section S1).

Applied to [\(11\)](#), the proposition shows that the balance row should not be treated as an ordinary local row. Its numerical effect depends on the effective coupling proxy $\|LD^{-1}\|_2$, not only on the raw magnitude of the entries in L . Moreover, if a common positive factor γ is applied to the coupling row, the scaled block satisfies

$$\begin{bmatrix} D & 0 \\ \gamma L & -\gamma I_T \end{bmatrix} = \begin{bmatrix} I & 0 \\ \gamma LD^{-1} & -I_T \end{bmatrix} \begin{bmatrix} D & 0 \\ 0 & \gamma I_T \end{bmatrix}. \quad (14)$$

Thus γ changes both the effective coupling term and the relative scale of the aggregate variable in the balance equation: pushing γ down improves the triangular factor of [equation \(14\)](#) but degrades its diagonal factor, and vice versa. This is the trade-off behind the proposed coupling treatment—global rows are scaled after local rows, using block-level representative scales, rather than by an isolated row norm—and the augmented rule resolves it explicitly by minimizing the product of the two factors ([equation \(23\)](#) in [section 4.2](#)). In the implemented variants, D^{-1} and singular values are replaced by coefficient-based block proxies, yielding a cheap operational approximation to the ideal coupling measure.

3.6 | From ideal diagnostics to implementable proxies

The two propositions establish *which* numerical distinctions a structure-aware rule must preserve, and thereby justify the architecture. They imply three design requirements: local scale imbalance should be reduced before global coupling rows are diagnosed; blocks should be summarized only after their local rows are on a comparable scale; and a coupling row should be scaled relative to the blocks it connects, not only relative to its own raw coefficient norm. The implemented method realizes exactly these distinctions through coefficient-based proxies: they are cheap to compute from the generated model before export, auditable because they depend only on visible row and block metadata, and they avoid expensive or unstable singular-value information from the rectangular, redundant, inequality-based matrices that arise in generated MIPs. The row factor based on the geometric mean of the smallest and largest nonzero magnitudes is a robust coefficient-scale proxy that centers each eligible row without changing variables, bounds, integrality, or the objective.

The coupling rule implements the same logic. In [proposition 2](#), the relevant ideal object is the effective coupling magnitude after local scaling, a term of the form LD^{-1} ; since generated blocks are rectangular, possibly redundant, and not inverted by the preprocessing layer, the block scale s_i used below is a representative coefficient scale for block i computed after local row scaling, and the coupling proxy divides each coupling coefficient by the representative scale of the block owning its column. This is a cheap, auditable surrogate for asking whether the coupling row is large relative to the local numerical scale of the blocks it links.

We state the scope of this argument once and plainly: the propositions describe square, nonsingular blocks under exact SVD scaling, the proxies act on rectangular, inequality-based matrices, and we establish no bound relating the proxy scale to the ideal spectral factor in [proposition 1](#). The propositions motivate the architecture, but the controlled synthetic suite of [section 5.2](#) does not test their spectral predictions: it evaluates only the coefficient-and-right-hand-side magnitude-range proxy that the implementation itself uses. The suite is therefore an implementation-level mechanism check—it validates the shared per-row proxy under row-multiplicative imbalance and exercises the common block-relative coupling factor on the oversized branch of its collective design case ([section 6.6](#))—rather than an empirical validation of the spectral surrogacy.

This design rationale also explains why the proposed method is row-based. Positive row scaling preserves the original units and domains of all variables; integer and binary variables do not require reinterpretation, and objective coefficients are left in their original economic units. Column scaling can be useful as a generic numerical device, so a Ruiz-plus-column baseline is included in the experiments. It is not part of the proposed structure-aware layer because the central claim is that row-role and coupling metadata alone already provide useful pre-export numerical information while keeping the transformation algebraically transparent.

4 | NUMERICAL SCALING METHOD

The scaling method keeps the same structural roles visible throughout: local rows, blocks, and coupling rows. Local rows receive row factors first. Blocks then inherit representative scales from their scaled local rows. Coupling rows receive their factors last, using the blocks they connect. This order is the algorithmic expression of the numerical argument in sections 3.4 and 3.5 and of the proxy interpretation in section 3.6.

This section defines the numerical transformations tested in the results. The base variant is the unscaled model generated by the platform. The two proposed variants share the local–block–coupling architecture but are *two different structural rules*: they differ both in whether the right-hand side participates in the scale diagnosis and in how coupling rows are treated (a single grid-selected factor in SA-Aug, one factor per row in SA-Mat), so their head-to-head comparison is not by itself a clean right-hand-side ablation. The clean ablation of the right-hand side is the local-only pair of section 6.8, whose members differ *only* in whether b enters the per-row extremes. The comparison with the Ruiz-kernel baselines tests the choice of per-row kernel under the same structural pipeline; the fully role-blind comparisons are carried by kernel-matched Flat-Aug and Flat-Mat controls (sections 5.4 and 6.8).

4.1 | Base formulation

The Base variant is passed to the solver without external scaling. Internal solver presolve and internal scaling remain enabled unless stated otherwise. All speedups, objective deviations, and original-scale reliability checks are computed relative to this variant on the same instance, solver, and gap setting (algebraic equivalence itself is proved analytically and needs no per-instance check).

4.2 | Augmented structure-aware scaling

The first proposed variant, denoted SA-Aug, computes row factors from both coefficient magnitudes and right-hand sides. The motivation is that a generated row may have moderate coefficients but a right-hand side in a very different physical or economic scale. Such a row can still lead to a poorly balanced numerical representation.

For each local row r , define

$$\mathcal{S}_r^{A,b} = \{|a_{rj}| : a_{rj} \neq 0\} \cup \{|b_r| : b_r \neq 0\}. \quad (15)$$

If $\mathcal{S}_r^{A,b} \neq \emptyset$, let

$$M_r^{A,b} = \max \mathcal{S}_r^{A,b}, \quad m_r^{A,b} = \min \mathcal{S}_r^{A,b}. \quad (16)$$

The local row factor is

$$\alpha_r^{A,b} = \frac{1}{\sqrt{M_r^{A,b} m_r^{A,b}}}. \quad (17)$$

The scaled row is

$$\alpha_r^{A,b} a_r^\top x \leq \alpha_r^{A,b} b_r. \quad (18)$$

The augmented rule is not only a row-by-row normalization. It is a three-stage structure-aware procedure that first rescales local rows, then summarizes the resulting local block scales, and finally uses those block scales to treat coupling rows separately. After local scaling, let $\tilde{a}_{rj} = d_r a_{rj}$ for local rows, where d_r is the factor *actually applied*— $d_r = \alpha_r^{A,b}$ when the candidate factor

$\alpha_r^{A,b} = 1/\sqrt{M_r^{A,b} m_r^{A,b}}$ passes the safeguards of [section 4.4](#) and $d_r = 1$ when it is rejected—so the extremes below are taken on the matrix that is actually exported, not on candidate factors. For each local block i , define

$$M_i^{A,b} = \max_{r \in \mathcal{R}_i, j: \tilde{a}_{rj} \neq 0} |\tilde{a}_{rj}|, \quad m_i^{A,b} = \min_{r \in \mathcal{R}_i, j: \tilde{a}_{rj} \neq 0} |\tilde{a}_{rj}|. \quad (19)$$

The representative scale of block i is

$$s_i^{A,b} = \sqrt{M_i^{A,b} m_i^{A,b}}. \quad (20)$$

For a coupling row r , let $\beta(j)$ denote the local block that owns column j . Columns owned by no block enter at the arithmetic mean of the valid block scales,

$$s_{\text{free}} = \frac{1}{|\mathcal{B}_+|} \sum_{i \in \mathcal{B}_+} s_i^{A,b}, \quad \mathcal{B}_+ = \{i : 0 < s_i^{A,b} < \infty\}, \quad (21)$$

falling back to $s_{\text{free}} = 1$ if no block has a valid scale, and components that contribute no local rows enter at unit scale. The effective coupling proxy of row r is

$$\hat{\rho}_r^{A,b} = \sqrt{\sum_j \left(\frac{|a_{rj}|}{s_{\beta(j)}^{A,b}} \right)^2}, \quad (22)$$

the magnitude of the row relative to the representative scales of the blocks it connects, and $\rho = \max_r \hat{\rho}_r^{A,b}$ is the worst such magnitude over the coupling rows. SA-Aug then computes a *single candidate* factor $\gamma^{A,b}$ for the coupling block, chosen to minimize the conditioning surrogate that the idealized analysis suggests, and applies it to each coupling row that passes the per-row safeguards of [section 4.4](#):

$$\gamma^{A,b} = \arg \min_{\gamma} \Phi(\gamma), \quad \Phi(\gamma) = \kappa_T(\gamma\rho) \kappa_B(\gamma), \quad \kappa_B(\gamma) = \frac{\max(\bar{M}, \gamma)}{\min(\bar{m}, \gamma)}, \quad (23)$$

where $\kappa_T(\cdot)$ is exactly κ_2 of the triangular coupling factor ([equation \(13\)](#)), κ_B is the diagonal spread once the aggregate variable enters at scale γ , and, writing $\kappa_i = M_i^{A,b}/m_i^{A,b}$ for the post-local-normalization coefficient spread of block i ,

$$\bar{M} = \max_i \sqrt{\kappa_i}, \quad \bar{m} = \min_i \frac{1}{\sqrt{\kappa_i}}. \quad (24)$$

These are the proxy extrema each block *would* present if it were virtually expressed relative to its representative scale $s_i^{A,b}$: dividing block i 's coefficients by $s_i^{A,b} = \sqrt{M_i^{A,b} m_i^{A,b}}$ maps its extremes $M_i^{A,b}, m_i^{A,b}$ to $\sqrt{\kappa_i}, 1/\sqrt{\kappa_i}$. This normalization is virtual—the algorithm does not apply $1/s_i^{A,b}$ to the blocks—and enters only the conditioning surrogate for the diagonal factor $\tilde{D}$; the block scales $s_i^{A,b}$ themselves are used only in the coupling proxy $\hat{\rho}_r$ of [equation \(22\)](#). The minimizer is found on the fixed logarithmic grid $\gamma = 10^{k/10}$, $k = -60, \dots, 60$ (121 log-uniform points spanning $[10^{-6}, 10^6]$). The stage is skipped as a whole when the effective coupling is declared moderate, using the same threshold $\rho \leq 10$ in every experiment (this is a per-instance test—on the operational classes it holds throughout, whereas on the synthetic coupling patterns it fails for the oversized instances, which is precisely where the stage activates)—or when the minimizer falls inside the near-identity band $(1/u, u) = (0.5, 2)$ of the safeguards. After these block-level tests, each coupling row is additionally subject to the per-row coefficient-window and reliability-floor safeguards of [section 4.4](#), which may veto the factor on individual rows; the final transformation is therefore a row-selective diagonal scaling with entries in $\{1, \gamma^{A,b}\}$, not necessarily a uniform scaling of the whole coupling block. Since $\rho = \max_r \hat{\rho}_r$ is taken over all coupling rows, the row that determines $\gamma^{A,b}$ may itself be vetoed by the floor, so $\gamma^{A,b}$ is a *single-pass candidate* rather than the minimizer of the surrogate for the final exported transformation; a fixed-point refinement that re-stabilizes the admissible set is a natural improvement we did not pursue, the stage being inactive on all operational classes. The grid search resolves explicitly the trade-off exposed by [equation \(14\)](#): decreasing γ improves the triangular factor but degrades the diagonal one. In the ideal spectral reading ($\rho = \|LD^{-1}\|_2$ and κ_B from the singular values of $\tilde{D}$), submultiplicativity of κ_2 over that factorization gives κ_2 of the scaled balance block at most $\kappa_T(\gamma\rho) \kappa_B(\gamma) = \Phi(\gamma)$, so minimizing Φ minimizes this ideal-scale bound; on the exported rectangular matrices Φ is evaluated from cheap coefficient proxies as a surrogate, no formal bound to the true κ_2 is claimed ([section 3.6](#)), and when the per-row safeguards veto the factor on some rows the factorization-based surrogate motivates the *candidate* but does not describe the exported, row-selective transformation. The released implementation evaluates these proxies from coefficient summaries of the generated model; the exact evaluation order is documented in the reproducibility repository. This sequence is the main structural feature of the proposed method: local

physical scales are used first, block scales are estimated after local normalization, and global coupling rows are scaled only after the blocks they connect have been put on comparable numerical footing.

4.3 | Matrix-only structure-aware scaling

The second proposed variant, denoted SA-Mat, uses the same row-block-coupling architecture but excludes b from the diagnosis. The right-hand side is still multiplied by the selected row factor to preserve equivalence, but it does not influence factor selection.

For each local row r , define the nonzero coefficient extremes

$$M_r^A = \max_{j: a_{rj} \neq 0} |a_{rj}|, \quad m_r^A = \min_{j: a_{rj} \neq 0} |a_{rj}|. \quad (25)$$

If the row has at least one nonzero coefficient, the row factor is

$$\alpha_r^A = \frac{1}{\sqrt{M_r^A m_r^A}}. \quad (26)$$

The scaled row is

$$\alpha_r^A a_r^\top x \leq \alpha_r^A b_r. \quad (27)$$

This factor centers the smallest and largest nonzero coefficient magnitudes of the row around a common scale. Unlike Euclidean-norm scaling, it does not directly depend on the number of nonzeros in the row.

Let $\tilde{a}_{rj} = d_r a_{rj}$ after local row scaling, where $d_r = \alpha_r^A$ when the candidate factor passes the safeguards of [section 4.4](#) and $d_r = 1$ otherwise, so the block extremes below are taken on the matrix actually exported, as in SA-Aug. For each local block i , define

$$M_i^A = \max_{r \in \mathcal{R}_i, j: \tilde{a}_{rj} \neq 0} |\tilde{a}_{rj}|, \quad m_i^A = \min_{r \in \mathcal{R}_i, j: \tilde{a}_{rj} \neq 0} |\tilde{a}_{rj}|. \quad (28)$$

The representative block scale is

$$s_i^A = \sqrt{M_i^A m_i^A}. \quad (29)$$

No additional multiplier is applied to all rows or columns of block i at this stage. The quantity s_i^A is an auxiliary representative scale: it is used only to normalize the contribution of block i when the coupling-row proxy is computed.

For a coupling row r , with the same ownership map $\beta(\cdot)$ and the same convention for blockless columns as in [section 4.2](#), define

$$\hat{\rho}_r^A = \sqrt{\sum_j \left(\frac{|a_{rj}|}{s_{\beta(j)}^A} \right)^2}. \quad (30)$$

If $\hat{\rho}_r^A > 0$, the coupling-row factor is per row,

$$\gamma_r^A = \frac{1}{\hat{\rho}_r^A}, \quad (31)$$

clipped to $[10^{-8}, 10^8]$ and subject to the safeguards below. When the block extremes of [equation \(28\)](#) are computed, the implementation restricts them to coefficients inside a fixed moderation window $(10^{-6}, 10^6)$, so a single extreme coefficient cannot distort s_i^A . If no post-local-scaling coefficient of block i falls inside that window, block i contributes no representative scale and its columns enter the coupling proxy at the matrix-only mean block scale

$$s_{\text{free}}^A = \frac{1}{|\mathcal{B}_A^+|} \sum_{i \in \mathcal{B}_A^+} s_i^A, \quad \mathcal{B}_A^+ = \{i : 0 < s_i^A < \infty\}, \quad (32)$$

with $s_{\text{free}}^A = 1$ when $\mathcal{B}_A^+ = \emptyset$; this is the matrix-only counterpart of [equation \(21\)](#), computed from s_i^A rather than $s_i^{A,b}$.

SA-Aug and SA-Mat therefore differ in *two* respects: whether b participates in factor selection, and the granularity and criterion of the coupling treatment (one grid-selected γ for the whole coupling block versus one γ_r per row). A difference between them cannot be attributed to the right-hand side alone; the clean right-hand-side ablation is the local-only pair reported with the coupling-stage activation audit ([section 6.8](#)), which shares every other design choice.

4.4 | Safeguards

All applied scaling factors—the local row factors and the coupling-row factors in the augmented or matrix-only variants—are subject to the near-identity and admissible-coefficient-range safeguards, applied independently to each factor. The collective SA-Aug coupling factor is additionally subject to the reliability floor defined at the end of this subsection, and the optional local reliability floor of that same paragraph applies to the augmented local factors. The representative block scales s_i are auxiliary quantities used to compute coupling-row proxies; they are not applied as independent row or column scalings.

4.4.0.1 | Near-identity threshold

A factor is applied only if it departs from one by at least a factor of $u = 2$; factors falling in the open band $(1/u, u) = (0.5, 2)$ are treated as negligible and skipped:

$$\theta \text{ is applied} \iff \left(\theta \leq \frac{1}{u} \right) \text{ or } (\theta \geq u), \quad u = 2. \quad (33)$$

This prevents re-scaling rows or coupling rows that are already at a reasonable scale.

4.4.0.2 | Admissible coefficient range

A factor is applied only if it keeps every affected coefficient within the window $[\varepsilon_{\min}, \varepsilon_{\max}] = [10^{-9}, 10^9]$. Writing $c_{\min}$ and $c_{\max}$ for the smallest and largest magnitudes of the affected coefficients, the factor θ is admitted only if

$$\begin{aligned} \varepsilon_{\min} &\leq c_{\min} \theta, & c_{\max} \theta &\leq \varepsilon_{\max}, \\ [\varepsilon_{\min}, \varepsilon_{\max}] &= [10^{-9}, 10^9]. \end{aligned} \quad (34)$$

Otherwise, the factor is not applied and the corresponding row or coupling row is left at its base scale. The coupling factor is additionally hard-clipped to $\gamma \in [10^{-8}, 10^8]$ prior to this check.

4.4.0.3 | Reliability safeguards

The coefficient window bounds the magnitudes of the scaled coefficients, but it does not by itself bound the effect of a small total row factor d_r on the *original-scale* residual. The exported row is d_r times the original, so *if* the returned point satisfies the exported row to absolute residual at most ε , the corresponding original-scale residual is at most ε/d_r ; a small d_r amplifies it. This is a conditional relation, not a solver-level guarantee: commercial solvers enforce feasibility on internally scaled and presolved representations, so a small ε on the exported row is expected rather than certified, and the floors below control the potential amplification from d_r rather than certify the residual. The *coefficient-only* per-row kernel is structurally safe against this amplification for its coefficients: its factor $\alpha_r = 1/\sqrt{M_r m_r}$ maps the largest coefficient to $\alpha_r M_r = \sqrt{M_r/m_r} \geq 1$, so the largest scaled coefficient is at least one (this centers the extremes geometrically around one; it does *not* preserve the original magnitude—for $M_r/m_r = 10^{12}$ a leading coefficient of 10^{12} becomes 10^3), and consequently, under a fixed residual bound on the exported row, the original residual relative to the row's leading coefficient is not amplified above that bound. The *augmented* kernel does not share this property: because $|b_r|$ enters the extremes, a row whose right-hand side dominates its coefficients receives a tiny d_r , and this is visibly the source of the largest absolute entries in the operational audit, which sit on rows of very large right-hand side (section 6.1). Two reliability safeguards are considered: the collective coupling floor, enabled by default, and an optional local floor evaluated separately.

The *collective coupling floor* guards the single SA-Aug coupling factor, which is chosen to fit the largest coupling rows and has no per-row protection: γ is applied to a coupling row only if the row's largest scaled coefficient magnitude remains at least $\varepsilon_{\text{floor}}^{\text{cpl}} = 10^{-3}$; rows below it keep their base scale. This is enabled by default, and it eliminates the common-factor failure mode on the synthetic suite (section 6.6). The optional *local floor* defines a variant we call SA-Aug-LF, which guards the augmented local factors directly: the local factor is applied only if it also satisfies

$$\alpha_r^{A,b} \geq \varepsilon_{\text{floor}}^{\text{loc}} \quad (\text{in addition to equations (33) and (34)}), \quad (35)$$

so that $d_r \geq \varepsilon_{\text{floor}}^{\text{loc}}$ bounds the conditional original residual by $\varepsilon/\varepsilon_{\text{floor}}^{\text{loc}}$; a factor below the floor leaves the row at base scale. SA-Aug-LF is off by default and is evaluated separately at $\varepsilon_{\text{floor}}^{\text{loc}} = 10^{-2}$. Because the condition on α_r is not invariant to a positive re-emission of a row, it is keyed to the generator's canonical physical-unit export—the reference used by the original-scale

audit; that caveat, invariant alternatives, and the sensitivity sweep are in Table S4. Neither floor is a general guarantee, and the primary SA-Aug results deliberately retain the unresolved local-reliability behavior rather than masking it.

Because each admitted factor is a strictly positive multiplier applied to a complete constraint—both its coefficients and its right-hand side—the feasible set and the objective value are preserved exactly. The safeguards therefore restrict only the numerical aggressiveness of the transformation, never its equivalence to the base model.

4.4.0.4 | Verification protocol

Algebraic equivalence concerns the mathematical program; it does not by itself guarantee that the solution the solver returns for the scaled formulation satisfies the original constraints within a given absolute tolerance. Every experiment therefore follows a fixed protocol: (i) solve the scaled formulation; (ii) evaluate the returned solution on the original, unscaled model (mapping variables back to original units only where column scaling is used); (iii) check the absolute residuals of every row, bound, and integrality restriction against the thresholds of [section 5.5](#)—the formal pass/fail criterion is absolute; right-hand-side-normalized residuals ([equation \(50\)](#)) are reported descriptively when interpreting exceedances; and (iv) treat a failed check as a numerical-reliability event—in an operational deployment such a run would be repeated with more conservative solver settings or fall back to the base formulation. [Section 6.1](#) reports this verification for all campaigns.

4.5 | Ruiz-type baselines

We compare the proposed variants with two *kernel-swap* baselines that run inside the same structural pipeline. The first, Ruiz, replaces the local per-row kernel with iterative max-norm equilibration applied to the block row sets; the second, Ruiz+Cols, adds continuous-column scaling and physical normalization. Both retain the row classification and the shared coupling stage (which never activates on the operational classes, where $\hat{\rho} \leq 10$ throughout, but does act on the synthetic benchmark; [section 6.6](#)). They are therefore *not* fully role-blind flat-matrix methods: they isolate the choice of *kernel* under the same selection policy, while the fully role-blind selection controls—each kernel with no roles at all—are Flat-Aug and Flat-Mat in [section 6.8](#).

Ruiz is deliberately not a weak foil: max-norm equilibration is a standard, strong scheme, and Ruiz+Cols additionally grants it the column-scaling lever the structure-aware rule forgoes. Together with the Flat controls these baselines split the comparison into two clean questions—does the geometric-mean kernel beat a Ruiz kernel under the same selection policy, and does the role-based selection beat full coverage under the same kernel; the separate “could the solver have done this itself” control is the internal-scaling baseline of [section 6.7](#). Every variant hands the solver the same monolithic MIP; only the pre-export row factors differ.

The final implementation uses

$$K = 4 \tag{36}$$

Ruiz iterations. Binary and integer variables are not rescaled unless explicitly stated, so that bounds, objective interpretation, and integrality remain unchanged.

4.6 | Algorithmic summary

[Algorithm 1](#) collects the augmented rule end to end.

The matrix-only algorithm follows the same local and block stages with the coefficient-only set $\{|a_{rj}| : a_{rj} \neq 0\}$ and the representative block scales and coupling proxies of [equations \(29\) and \(30\)](#); its coupling stage differs in granularity, assigning each coupling row its own factor $\gamma_r^A = 1/\hat{\rho}_r^A$ ([equation \(31\)](#)) instead of the single grid-selected $\gamma^{A,b}$. SA-Mat is therefore a second structural rule, not a right-hand-side ablation of SA-Aug: the two differ both in whether b enters the diagnosis and in the coupling criterion, and only their local-only reductions ([section 6.8](#)) isolate the right-hand side cleanly.

Algorithm 1 Augmented structure-aware row scaling

Require: Matrix A , right-hand side b , row roles (local/coupling/residual-global), block ownership $\beta(\cdot)$, coupling incidence; optionally useLocalFloor and $\varepsilon_{\text{floor}}^{\text{loc}}$ (default: off, describing the primary SA-Aug variant)

Ensure: Scaled matrix $\tilde{A}$, scaled right-hand side $\tilde{b}$

- 1: Initialize row factors $d_r \leftarrow 1$ $\triangleright$ residual-global rows in $\mathcal{R}_{\text{other}}$ keep $d_r = 1$ throughout
- 2: **for** each local block $i \in \mathcal{B}$ **do**
- 3: **for** each local row $r \in \mathcal{R}_i$ **do**
- 4: Build $S_r^{A,b}$ from nonzero coefficients and nonzero right-hand side
- 5: **if** $S_r^{A,b} \neq \emptyset$ **then**
- 6: Compute $M_r^{A,b}$ and $m_r^{A,b}$ using [equation \(16\)](#)
- 7: Set $\alpha_r^{A,b} \leftarrow 1/\sqrt{M_r^{A,b} m_r^{A,b}}$
- 8: **if** the safeguards admit $\alpha_r^{A,b}$ (and, if useLocalFloor, $\alpha_r^{A,b} \geq \varepsilon_{\text{floor}}^{\text{loc}}$) **then**
- 9: Update $d_r \leftarrow d_r \alpha_r^{A,b}$
- 10: **end if**
- 11: **end if**
- 12: **end for**
- 13: **end for**
- 14: Compute the locally scaled coefficients $\tilde{a}_{rj} = d_r a_{rj}$
- 15: Compute block scales $s_i^{A,b} = \sqrt{M_i^{A,b} m_i^{A,b}}$ using [equation \(20\)](#)
- 16: Compute $\hat{\rho}_r^{A,b}$ for every coupling row using [equation \(22\)](#); set $\rho \leftarrow \max_r \hat{\rho}_r^{A,b}$
- 17: **if** ρ exceeds the coupling threshold **then**
- 18: Set $\gamma^{A,b} \leftarrow \arg \min_{\gamma} \Phi(\gamma)$ over the logarithmic grid $\gamma \in [10^{-6}, 10^6]$ ([equation \(23\)](#))
- 19: Clip $\gamma^{A,b}$ to $[10^{-8}, 10^8]$
- 20: **if** $\gamma^{A,b}$ lies outside the near-identity band ([33](#)) **then**
- 21: **for** each coupling row r **do**
- 22: **if** $\gamma^{A,b}$ keeps row r inside the coefficient window ([34](#)) **and** above the reliability floor ([section 4.4](#)) **then**
- 23: Update $d_r \leftarrow d_r \gamma^{A,b}$
- 24: **end if**
- 25: **end for**
- 26: **end if**
- 27: **end if**
- 28: Return $\tilde{A} = \text{diag}(d)A$ and $\tilde{b} = \text{diag}(d)b$

5 | EXPERIMENTAL DESIGN

5.1 | Instance classes and scenarios

The experiments are organized around four reader questions: does scaling preserve the original optimization problem, does it improve the numerical representation, does that change solver behavior, and does the effect survive changes in class, solver, and tolerance? The study uses three instance classes generated by the same modular hydrothermal platform, built from operational instances with available real input data. The pools are balanced by construction: every variant is attempted on every instance of a class (204 Simple, 68 SG-Ter-Mer, 34 Full). Valid-run counts nevertheless differ slightly across class–scenario–variant cells because individual runs can time out, abort, or lack a required diagnostic. Descriptive metrics use all valid runs for a row (so a cell may report, say, 195–199 of 204 Simple instances). Speedups and paired tests use scenario-specific matched base–variant pairs. One-sided timeouts or algorithmic aborts are retained at the capped time, as defined in the statistical-analysis subsection; pairs are excluded only when both sides fail to complete or when the instance is uniformly unavailable for the documented infrastructure and artifact-curation reasons reported in Supporting Information.

TABLE 1 Three instance classes test the method on progressively different generated hydrothermal models.

Class	Configuration	Main role	Instances
Simple	Río Negro + fixed demand + fixed dispatch	Basic numerical behavior	204
Full	Río Negro nonlinear + Salto Grande + thermal units + market	Largest integrated model	34
SG-Ter-Mer	Salto Grande + thermal units + market	Coupled hydrothermal subset	68

The counts report the maximum operational instance pools with available real input data. Scenario tables may show different n values because each aggregate row uses the valid runs available for that solver–gap–variant combination. Pairwise speedups and tests use matched base–variant pairs only.

TABLE 2 Four solver–tolerance scenarios separate solver dependence from gap–tolerance effects.

Scenario	Solver	Relative MIPGap
1	Gurobi	10^{-2}
2	Gurobi	10^{-3}
3	CPLEX	10^{-2}
4	CPLEX	10^{-3}

Deterministic time is reported when available:
Work units for Gurobi and ticks for CPLEX.

Each measurement is tied to the part of the argument it can support: algebraic equivalence is proved analytically, and the numerical reliability of each returned solution is checked by the original-scale feasibility and integrality tests (with variables mapped back to original units only for the column-scaling baseline); the numerical claim by exported coefficient-range summaries and solver-level conditioning diagnostics; the performance claim by matched base–variant speedups, deterministic effort, performance profiles, and primal–dual progress; and solver dependence by separate Gurobi and CPLEX scenarios at two tolerances.

Table 1 separates the three pools: Simple gives the largest sample for distributional evidence, Full is the smallest and most integrated model, and SG-Ter-Mer isolates a coupled hydrothermal subset—so the averages and paired tests do not come from a single homogeneous population.

The class names are application-specific, but the contrast is structural: whether the rule behaves consistently as the generated MIP moves from a simpler modular model to larger or more strongly coupled ones. The conclusions are thus stated at two levels—empirical evidence from hydrothermal dispatch, methodological object the use of generator metadata in block-structured engineering MIPs.

We evaluate four solver–gap scenarios. The operational time limit is uniformly $T_{\max} = 3600$ s (one hour) per solve; the separate synthetic mechanism check uses 600 s. Internal solver presolve and internal scaling are left enabled, because the goal is to test whether external preprocessing helps in a realistic solver pipeline. All main experiments were run on ClusterUY, a collaborative high-performance computing platform for scientific computing in Uruguay¹⁷.

The four scenarios in table 2 vary two dimensions separately: the stopping tolerance and the solver pipeline. Scenarios 1–2 test Gurobi at two MIPGap values; scenarios 3–4 repeat the comparison under CPLEX and serve as a solver contrast.

The configuration in table 3 matters because the proposed preprocessing is tested on top of each solver’s default numerical pipeline. The CPLEX conclusions below are therefore statements about this version and ClusterUY environment, not claims about all CPLEX releases or parameter choices.

TABLE 3 The computational environment fixes solver versions, time limits, and default presolve settings for all comparisons.

Item	Configuration
Cluster	ClusterUY, partition <code>ute</code>
Processor	Intel Xeon Gold 6138
Threads	16
Memory limit	12 GB
Time limit	1 hour per solve
Gurobi version	13.0.1 ¹⁸
CPLEX version	22.1.2.0 ¹⁹
Presolve	solver default, enabled

Solver-internal scaling and presolve were left at their default enabled settings unless otherwise stated. CPLEX 22.1.2.0 is the version available in the ClusterUY environment used for the long runs; conclusions involving CPLEX should therefore be interpreted as solver-version and environment specific.

5.2 | Controlled synthetic block-structured benchmark

The hydrothermal testbed evaluates the proposed preprocessing rule on realistic operational generated MIPs. To complement this real-data setting, we define a controlled synthetic benchmark whose purpose is diagnostic rather than physical. The synthetic instances are not intended to model a power system. They isolate the numerical mechanism targeted by the method by allowing local-row scale imbalance and coupling-row scale imbalance to be varied independently.

Each synthetic instance consists of B local blocks. Block i contains continuous activity variables $y_i \in \mathbb{R}_+^{n_y}$ and binary activation variables $z_i \in \{0, 1\}^{n_z}$. A nonnegative slack vector s is added to the global coupling constraints to guarantee feasibility. The generic synthetic MIP is

$$\begin{aligned}
\min_{y, z, s} \quad & \sum_{i=1}^B c_i^\top y_i + \sum_{i=1}^B q_i^\top z_i + P \mathbf{1}^\top s \\
\text{s.t.} \quad & R_i y_i + G_i z_i \leq r_i, \quad i = 1, \dots, B, \\
& 0 \leq y_{ik} \leq U_{ik} z_{ik}, \quad i = 1, \dots, B, \quad k = 1, \dots, n_y, \\
& \sum_{i=1}^B L_i y_i + \sum_{i=1}^B H_i z_i + s \geq d, \\
& s \geq 0, \quad z_i \in \{0, 1\}^{n_z}.
\end{aligned} \tag{37}$$

The first group of constraints defines local block restrictions, the second group links continuous activity variables to binary activation variables, and the third group creates global coupling rows. The penalty P is chosen large enough that the slack variable is used only when the activated block capacity cannot satisfy the synthetic demand.

The unscaled coefficients are generated from bounded random distributions with controlled sparsity. Local matrices R_i and G_i define block-internal resource constraints. Coupling matrices L_i and H_i define the contribution of each block to the global rows. The generator records row metadata

$$\text{row_type} \in \{\text{local}, \text{coupling}\}, \quad \text{block_id} \in \{1, \dots, B\}, \tag{38}$$

and column metadata identifying the block associated with each variable. This metadata is the same information required by the structure-aware scaling rules.

Scale imbalance is introduced after the clean synthetic MIP has been generated. For each eligible row r , a positive row multiplier

$$\theta_r = 10^{\xi_r}, \quad \xi_r \sim \mathcal{U}[-S, S], \tag{39}$$

is drawn and applied to the entire row, including its right-hand side. Therefore, the scaled representation is algebraically equivalent to the clean synthetic MIP. The severity parameter S controls the possible scale range. We use

$$S \in \{0, 3, 6\}, \tag{40}$$

TABLE 4 Controlled synthetic benchmark factors isolate the numerical mechanisms targeted by the scaling rule.

Factor	Levels used	Controlled object	Purpose
Scale pattern	none, local, coupling, coupling-uniform, mixed	Row class affected by artificial scaling	Separate local imbalance from per-row and collective coupling imbalance
Severity S	0, 3, 6	Range of row multipliers 10^{-S} – 10^S	Test mild and severe numerical heterogeneity
Blocks and size	$B = 10, n_y = n_z = 8$	Number and size of generated local components	Keep a fixed block structure while varying row-scale imbalance
Rows and seeds	8 local rows per block, 3 coupling rows, 20 seeds	Random coefficients and sparsity patterns	Estimate distributional behavior rather than one instance

The synthetic instances are not physical power-system models. They are controlled block-structured MIPs designed to test whether the preprocessing rule responds to the type of scale imbalance it is designed to exploit. The reported synthetic grid uses the middle size $B = 10$, the five patterns, the three severity levels, and twenty random seeds per pattern–severity cell.

corresponding to no artificial imbalance, moderate imbalance, and strong imbalance.

Five scale patterns are considered:

1. **None:** no artificial row scaling is applied.
2. **Local:** only local block rows are multiplied by factors (39).
3. **Coupling:** only global coupling rows are multiplied by factors (39), one independent draw per row—heterogeneous imbalance *among* the coupling rows.
4. **Coupling-uniform:** all global coupling rows are multiplied by a *single* shared factor $\theta = 10^\xi$, $\xi \sim \mathcal{U}[-S, S]$, one draw per instance—a collective, units-mismatch-style displacement of the coupling rows relative to the blocks, which is the case the block-relative coupling stage is designed for.
5. **Mixed:** both local block rows and global coupling rows are multiplied, independently per row.

The “none” pattern is a negative control: a useful preprocessing rule should not create large artificial changes when the representation is already reasonably scaled. The “local” and two coupling patterns isolate the numerical objects distinguished by the proposed method, and the two coupling patterns separate the collective displacement a single block-relative factor can repair from the per-row heterogeneity it cannot. The “mixed” pattern approximates the combined heterogeneity observed in generated models. These controlled patterns are mechanism checks. They verify that the implementation reacts correctly when the source of imbalance is known by construction, but they are not an operational local-only versus coupling-only ablation on the real hydrothermal instances. In the generated instances the local and coupling row sets are disjoint, as in the method’s row taxonomy (section 4): a coupling row is incident to several blocks through its variables, but it is not a local row of any of them and does not enter the local-row normalization stage.

The reported synthetic runs follow the factor grid summarized in table 4, using the five scaling variants in section 5.3 plus the role-blind flat control introduced in section 6.8, the same original-scale feasibility and objective checks, Gurobi with relative MIPGap 10^{-3} , and a 600 second time limit. They are intentionally small: the benchmark is a controlled mechanism check for global magnitude-range behavior and original-scale numerical reliability, not a performance experiment. Runtime claims are reserved for the operational hydrothermal instances. The legacy CSV fields `kappa_antes/kappa_despues` store the augmented global magnitude-range proxy

$$\mathcal{M}(A, b) = \{ |a_{rj}| : a_{rj} \neq 0 \} \cup \{ |b_r| : b_r \neq 0 \}, \quad \hat{\kappa}_{\text{range}}(A, b) = \frac{\max \mathcal{M}(A, b)}{\min \mathcal{M}(A, b)}, \quad (41)$$

not the spectral condition number $\kappa_2(A)$. The synthetic tables, figures, and tests below use equation (41); no singular-value condition number is computed for this grid.

5.3 | Scaling variants

The compared variants are: Base, the original formulation; SA-Aug, the augmented structure-aware scaling rule; SA-Mat, the matrix-only structure-aware rule with safeguards; Ruiz, a Ruiz-kernel baseline that swaps the local geometric-mean kernel for

max-norm row equilibration ($K = 4$ iterations) inside the same structural pipeline; and Ruiz+Cols, the same with continuous-column scaling and physical normalization. Both Ruiz variants keep the row taxonomy and the shared coupling stage and are therefore *not* fully role-blind; the fully role-blind operational baselines are Flat-Aug and Flat-Mat in [section 6.8](#) (the synthetic grid uses Flat-Aug). Binary and integer variables are not rescaled.

5.4 | Additional controls: internal scaling, sensitivity, and coupling-stage activation

Three further controls sharpen the attribution of the main comparison. They reuse the same matched-pairs design, the same instance classes, and the same original-scale verification, and they leave the five variants above unchanged.

5.4.0.1 | Solver-internal-scaling baseline

To separate “structure-aware external scaling helps” from “any extra scaling pass helps, and the solver could have produced it,” we run the *unscaled* model under the solver’s own scaling controls and compare it, within each scenario, against the structure-aware variants run under default internal scaling. For Gurobi we sweep the scaling flag over its admissible settings (no scaling, equilibration, geometric, and aggressive modes); for CPLEX we sweep its read-scaling parameter (none, default equilibration, aggressive). If a tuned internal setting matches the structure-aware runtime on the same instances, the distinctive claim collapses to generic coefficient compression; if it does not, the gain cannot be attributed to a scaling pass the solver could have produced itself. Presolve remains enabled throughout. Internal scaling is varied only in this control’s solver-specific sweep (including the setting that disables it), while the structure-aware variants always run under the solver’s default internal-scaling configuration, as in the main experiments.

5.4.0.2 | Hyperparameter sensitivity

The constants of the proposed rules are, for SA-Aug: the near-identity band $u = 2$ ([equation \(33\)](#)), the admissible coefficient window $[\varepsilon_{\min}, \varepsilon_{\max}] = [10^{-9}, 10^9]$ ([equation \(34\)](#)), the coupling activation threshold $\hat{\rho} > 10$, the collective coupling reliability floor $\varepsilon_{\text{floor}}^{\text{cpl}} = 10^{-3}$, the γ grid (121 log-uniform points on $[10^{-6}, 10^6]$), and the hard clip $[10^{-8}, 10^8]$; the optional SA-Aug-LF variant adds the local floor $\varepsilon_{\text{floor}}^{\text{loc}} = 10^{-2}$ (off by default). SA-Mat shares u , the window, and the clip, and adds the moderation window $(10^{-6}, 10^6)$ on the block extremes. The Ruiz iteration count $K = 4$ is a parameter of the baselines, not of the proposed rules. Three sensitivity analyses are reported. On the Simple class under Gurobi we sweep $u \in \{1.5, 2, 4\}$ and the coefficient window over $\{[10^{-6}, 10^6], [10^{-9}, 10^9], [10^{-12}, 10^{12}]\}$: the runtime conclusions are insensitive (sgm within $[2.01, 2.12]$ s across the sweep, against 3.73 s for the base; Table S1, Supporting Information). The collective coupling floor $\varepsilon_{\text{floor}}^{\text{cpl}}$ is swept over three orders of magnitude on independent synthetic seeds ([section 6.6](#); Table S3), and the local floor $\varepsilon_{\text{floor}}^{\text{loc}}$ over $\{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}\}$ on the failing Simple subset ([section 6.1](#); Table S4). The coupling threshold is swept over $\hat{\rho}_{\text{trigger}} \in \{2, 5, 10, 20, 50\}$ on the synthetic coupling patterns, where the stage is active and the threshold could in principle discriminate: the medians are unchanged, because the coupling proxy is bimodal and essentially never lands in that band ([section 6.6](#)). On the operational classes the same proxy sits far from the threshold on both sides ($\hat{\rho} \approx 3 \times 10^{-4}$ in Simple, ≈ 1 in SG-Ter-Mer and Full), so no nearby value changes any operational run; the coupling-stage activation audit confirms zero operational activations. A separate $K \in \{2, 4, 8\}$ sweep for the Ruiz baselines confirms their irregular matched behavior is not an artifact of the iteration count.

5.4.0.3 | Operational coupling-stage activation audit

The synthetic benchmark isolates local and coupling imbalance by construction, but the operational comparison evaluates the full local–block–coupling architecture. We run two single-stage variants on the hydrothermal classes—local rows scaled but coupling rows left at base scale, and coupling rows scaled but local rows left at base scale—against the same base. As [section 6.8](#) shows, the coupling stage does not activate on any operational instance, so this comparison is an *activation audit* rather than a true ablation: it establishes that the coupling stage is inactive operationally (full and local-only export identical matrices) and that the entire operational effect is the local-row normalization; the coupling stage’s marginal contribution is measured only on the synthetic benchmark, where it does activate.

5.5 | Metrics and reporting conventions

The analysis reports three families of metrics. First, numerical feasibility of the returned solution is checked on the original, unscaled constraints. Row-only variants do not transform the variables, so the returned primal vector is evaluated directly;

when column scaling is used (the Ruiz+Cols baseline), the variables are first mapped back to their original units. The primary post-solve check is the maximum violation of the original constraints, together with the maximum integrality violation. A run is labeled feasible in original scale if the maximum row violation is at most 10^{-4} and the maximum integer violation is at most 10^{-5} . Recomputing the objective in original units is retained as an auxiliary consistency diagnostic, but it is not the primary feasibility criterion. Objective agreement with the base is measured by

$$\Delta z_{\text{rel}} = \frac{|z_v - z_{\text{base}}|}{\max\{1, |z_{\text{base}}|\}}. \quad (42)$$

This deviation is descriptive only, and it is not by itself evidence of an error in the scaled variant, because the base can itself return an invalid claimed optimum (section 6.1). The correctness reference is therefore the best *verified* incumbent on each instance,

$$z_i^{\text{ver}} = \min\{z_{iv} : \text{run}(i, v) \text{ passes original-scale verification}\}, \quad \Delta z_{iv}^{\text{ver}} = \frac{|z_{iv} - z_i^{\text{ver}}|}{\max\{1, |z_i^{\text{ver}}|\}}, \quad (43)$$

taken within each scenario (solver $\times$ tolerance) over the variants that complete and pass verification; the failure analysis of section 6.1 uses z_i^{ver} , judging the base against it like any variant, while equation (42) is retained in the audit table as the historical base-referenced spread with both directions flagged. When no run of an instance passes verification within a scenario, the set is empty; such an instance is classified as unresolved in that scenario and excluded from objective-distance summaries, but retained as a failure in the operational reliability metrics. In the reported data the rule is inactive in the four Gurobi scenarios—there every instance has at least one verified incumbent among the available base and scaled runs, so no instance is unresolved—but it is active under CPLEX, where verification failures are both frequent and correlated across variants: 63 of 204 Simple instances at 1%, 55 of 204 at 0.1%, and one of 34 Full instances at 0.1% have no verified run at all and are therefore unresolved for the purpose of equation (43). This is a further consequence of the CPLEX reliability behavior documented in section 6.1, and it is why the verified-incumbent reference is used for the Gurobi comparisons on which the runtime conclusions rest; no reported quantity is computed from equation (43) in the affected CPLEX cells.

Second, numerical behavior is a primary outcome, not only an explanatory variable for speed. We distinguish three quantities. The first is the spectral condition number used in section 3.3 to motivate the rule in an idealized linear-algebraic setting. The second is the exported coefficient-range ratio

$$\rho_{\kappa}(v) = \frac{(\max |a| / \min |a|)_v}{(\max |a| / \min |a|)_{\text{base}}}, \quad (44)$$

where the extrema are computed over nonzero coefficients in the generated matrix. This is a static proxy and should not be interpreted as the condition number of the MIP. The third is the solver-level ratio

$$\rho_{\kappa}^{\text{solver}}(v) = \frac{\kappa_{\text{solver}}(v)}{\kappa_{\text{solver}}(\text{base})}, \quad (45)$$

where κ_{solver} is the basis condition number reported by the solver on the LP obtained by fixing the integer variables to the incumbent *each variant found* and re-solving with dual simplex (Gurobi's `fixedModel`), measured after the main MIP solve so it does not enter the timer. A consequence must be stated plainly: because each variant fixes to its own incumbent, the ratio (45) can reflect not only the change of representation but also differences in the integer decisions the variants reached; it is a diagnostic of the solver-facing numerics along each variant's own solution path, not of a single common LP. This is one more reason the diagnostic is read as a partial, not a definitive, mediator of runtime, and why the primary runtime evidence is the matched timing rather than the conditioning ratio. For CPLEX we also record tree-level KappaStats (maximum sampled basis condition number, unstable and ill-conditioned fractions, numerical attention score), which sample bases across the actual tree. Parsed solver warnings are kept as a descriptive alert, not a ranking criterion. Values below one indicate improvement relative to the base. The paper thus separates the numerical effect from whether it is converted into faster MIP search.

Third, efficiency is measured by wall-clock time, solver time, total time including preprocessing, deterministic time, node counts, iterations, final gap, and callback-based gap progress. For any time measure T_q , the paired speedup is

$$\text{speedup}_q(v) = \frac{T_{q,\text{base}}}{T_{q,v}}, \quad q \in \{\text{solver, total, det}\}. \quad (46)$$

Values above one indicate that variant v is faster than the base.

Two total-time notions appear in this paper and they must not be conflated. In the main campaign, T_{total} is reconstructed by the driver as the logged preprocessing time plus the logged solver time—the same formula as the $T_{\text{pre+solve}}$ of the dedicated SA–Flat batches (section 5.6), and likewise excluding model construction, solver loading, and verification. The two differ in *measurement resolution*, not in definition. In the archived main-campaign records the preprocessing component is stored at whole-second resolution: it is exactly 0 for the base and for both structure-aware variants, and 2–63 s for the Ruiz baselines. Consequently the main-campaign T_{total} of a structure-aware variant coincides numerically with its solver time and understates its true preprocessing cost, which the separately instrumented dedicated batches measure at 0.02–0.13 s per call. The Ruiz overhead visible in the total-time figures is therefore real and large, whereas the structure-aware overhead is invisible at that resolution; only the dedicated batches support preprocessing-plus-solver statements about the structure-aware and Flat policies, and levels of the two quantities are never compared across campaigns.

Because solution times are highly skewed, we report shifted geometric means

$$\text{sgm}_s(\{x_i\}) = \exp\left(\frac{1}{n} \sum_i \log(x_i + s)\right) - s, \quad (47)$$

using $s = 1$ s for times and $s = 10$ for node counts. For callback reports (t_k, γ_k) , where $\gamma_k \in [0, 1]$ is the relative optimality gap and is set to one before the first incumbent, we report the time-averaged relative gap

$$\bar{\gamma}_{\text{cb}} = \frac{1}{t_K} \sum_{k=2}^K \frac{\gamma_{k-1} + \gamma_k}{2} (t_k - t_{k-1}). \quad (48)$$

This is the implemented trapezoidal average over the observed callback horizon, normalized by the final callback timestamp t_K ; no extrapolation is made before the first report or after the last. It is dimensionless and removes the direct duration penalty, so it complements solver time and fixed-pool PAR10 rather than replacing either. Because callback cadence and gap reporting differ between solvers, the absolute level of $\bar{\gamma}_{\text{cb}}$ is comparable within a scenario table but not across solvers. Timeouts and non-timeout solver or pipeline aborts are summarized by PAR10 on a common instance pool. For every operational scenario the penalty is $10T_{\text{max}} = 36,000$ s; the three per-run cost definitions are given with the original-scale reliability criteria below.

Two reporting conventions apply throughout. The descriptive sgm in the per-scenario tables is computed over each variant’s own completed runs, so the count n can differ by a few instances between variants (timeouts, aborts, or missing diagnostics); a raw sgm comparison across variants can therefore mix speed with which instances each variant solved. The sgm tables are accordingly read as descriptive. Inferential comparisons use each Base–variant eligible matched pool (with one-sided non-completions capped), while the fixed-pool PAR10 of table 5 uses one scenario pool for every displayed variant.

The Dolan–Moré profiles use a separate but equally fixed convention. For scenario s , let $\mathcal{I}_s^{\text{prof}}$ be the common pool after removing uniformly unavailable instances and instances for which no variant has an admissible normally terminated solver time. Define

$$c_{iv}^{\text{prof}} = \begin{cases} t_{iv}, & \text{if variant } v \text{ terminates normally,} \\ 10T_{\text{max}}, & \text{for a timeout, abort, missing run, or invalid individual time,} \end{cases} \quad r_{iv} = \frac{c_{iv}^{\text{prof}}}{\min_{v'} c_{iv'}^{\text{prof}}}, \quad (49)$$

and $P_v(\tau) = |\mathcal{I}_s^{\text{prof}}|^{-1} \sum_{i \in \mathcal{I}_s^{\text{prof}}} \mathbf{1}\{r_{iv} \leq \tau\}$. Every curve in a panel therefore has the same denominator. The logarithmic horizontal range runs from one to the largest finite penalized ratio in that panel. The common-pool sizes for Simple/Full/ SG–Ter–Mer are 199/32/67 (Gurobi 1%), 200/29/64 (Gurobi 0.1%), 204/32/68 (CPLEX 1%), and 204/30/68 (CPLEX 0.1%).

5.6 | Statistical analysis

The statistical analysis is paired by construction: every variant is compared with Base on the same instance, solver, and MIPGap tolerance. Since solve times are skewed, we apply the Wilcoxon signed-rank test to the scenario-specific eligible matched pool. For a one-sided timeout, setting the failed side to T_{max} preserves the known direction while censoring the rank magnitude. For a one-sided algorithmic or pipeline abort, assigning T_{max} is an operational-failure convention, not a censoring argument; a sensitivity assigns such aborts $10T_{\text{max}}$. A pair is excluded only when both sides do not complete, when a required time is absent, or when the instance is uniformly unavailable under the documented data criterion. Thus a variant is not rewarded for aborting.

Table S6 reports every excluded and capped case; the abort sensitivity leaves all signs and inferential decisions unchanged (the largest structure-aware change is Gurobi Full/SA-Aug at 0.1%: $q_{\text{BH}} = 0.110$ to 0.116).

We use a two-sided Wilcoxon signed-rank test on $d_i = t_{i,\text{variant}} - t_{i,\text{Base}}$. Exact zeros are discarded (“wilcox” convention), tied absolute differences receive midranks, and no continuity correction is applied. The exact null distribution is used when there are at most 50 nonzero differences and neither zeros nor tied absolute ranks; otherwise we use the tie-corrected normal approximation. In the displayed structure-aware rows, Simple and SG-Ter-Mer use the approximation and Full uses the exact distribution; no zero or tied absolute differences occur. As the effect size we report the matched-pairs rank-biserial correlation $r_{\text{rb}} = (T^+ - T^-)/(T^+ + T^-)$, computed from the same signed ranks as W , together with the median of the per-instance paired speedup ratios; both are genuinely paired quantities, unlike distribution-overlap measures such as Cliff’s δ , which we do not use. We also report the raw p -value and the Benjamini–Hochberg adjusted value q_{BH} . The Benjamini–Hochberg correction is applied within each scenario (solver $\times$ tolerance), over the family of all four scaled variants across the three instance classes (twelve paired tests); the tables display the two structure-aware variants, while q_{BH} reflects that full family. In the synthetic benchmark, the adjustment is applied jointly over the 60 paired magnitude-range-proxy comparisons—four variants against the base, five imbalance patterns, three severity levels—within that proxy metric; Ruiz+Cols is excluded from that family because it coincides exactly with Ruiz on the synthetic instances (no unit metadata), so including it would duplicate a full set of hypotheses. With the convention used here, negative values of r_{rb} indicate that the variant tends to be faster than the base, whereas positive values indicate deterioration.

The direct SA–Flat analysis has its own endpoints and multiplicity structure, and one disclosure comes first: *the endpoint hierarchy was selected after inspection of the direct-control data and is therefore exploratory*. We report *prototype preprocessing-plus-solver* time $T_{\text{pre+solve}}^{\text{prototype}} = T_{\text{preproc}} + T_{\text{solver}}$ as the observed implementation endpoint, and capped solver time as its decomposition. The prototype endpoint charges each rule the cost of the implementation used in the dedicated batches, including metadata reconstruction; a release-build cached-metadata sensitivity (table 19) approximates a generator-integrated implementation in which the structural taxonomy is reused. Neither is treated as a preregistered confirmatory outcome, and we report both. The name is deliberately narrow: $T_{\text{pre+solve}}$ excludes model construction, solver model loading, and post-solve verification, which are common to both policies, so it is not literally the whole pipeline. Here T_{preproc} is the wall time of the preprocessing call on the constructed model—row-role classification, scale diagnosis (including the coupling-stage grid search where applicable), and in-place application of the factors. In the dedicated batches each call was timed once with the C++ high-resolution wall clock, in the batch’s fixed execution order, without repetition; per-policy medians and IQRs are in Table S11 (medians 0.02–0.13 s, IQRs ≤ 0.003 s). Because a single unrandomized measurement cannot separate the policy effect from systematic order effects—cache warming, allocator state, machine state—we re-timed the preprocessing call alone in a dedicated replication that never solves the MIP: for every instance of all three classes and both kernels, $R = 20$ repetitions with the SA/Flat order drawn from a seeded RNG within each repetition and each call in a fresh process (`scripts/preproc_timing_replication.py`); the per-instance statistic is the median over repetitions. Each timed call starts after model construction and excludes process startup and input parsing; the two policies of a repetition run sequentially in the randomized order, timed with a monotonic steady clock, over 12 concurrent workers on a 16-core workstation without CPU pinning, so they share machine state but compete with unrelated instances (full protocol and per-cell statistics in Table S11; dispersion is summarized by the interquartile range relative to the median, the coefficient of variation being unstable for short, skewed timings). Section 6.8 reports the outcome: the direction replicates on every instance, while the absolute gap is load-dependent. The deterministic Gurobi Work comparison and the solver-time decomposition are exploratory; their adjusted values are reported descriptively and define no additional confirmatory conclusions. For the time endpoints the paired variable is $d_i = \log(T_{\text{Flat},i}/T_{\text{SA},i})$: the Wilcoxon signed-rank test, the rank-biserial correlation, the operational-family sign-flip sensitivity, and the family bootstrap are computed on this same relative scale and share its direction convention (negative favors full coverage), but they do not target an identical estimand: the Wilcoxon test targets a signed-rank location under symmetry, r_{rb} is a rank-weighted effect size, and the median ratio summarizes the sample median. As a location estimate aligned with the test we also report the Hodges–Lehmann pseudomedian $\hat{\theta}_{\text{HL}} = \text{median}_{i \leq j} (d_i + d_j)/2$, expressed as the typical SA/Flat ratio $\exp(-\hat{\theta}_{\text{HL}})$ (Table S10). (Work, a deterministic count that can be zero, keeps the paired difference.) The analysis convention defines each MIPGap tolerance as a distinct six-test family (three classes by two kernels), because changing the stopping tolerance changes the operating regime; the joint BH adjustment over all twelve contrasts is also reported, and each endpoint forms its own BH families. Both readings initially treat instances as independent. As a sensitivity under family-level joint sign symmetry—it does not correct for arbitrary dependence—one common sign is flipped for every prespecified operational family while the within-family absolute ranks are retained; the exact null distribution is evaluated by dynamic programming and BH-adjusted over the

same six- and twelve-test families. The Simple pool contains 34 operational families of six scenarios each (`casoXX[a-f]`, six perturbations of one operating scenario). The SG-Ter-Mer pool contains 34 paired families and no singletons: the released identifiers span `instance001–instance096` and pair exactly under the $XXX \leftrightarrow XXX+48$ correspondence encoded by the generator’s scenario numbering, which links the two variants derived from the same operating scenario. The Full pool contains 34 singleton families. Family counts and pair-specific attrition after exclusions are reported per contrast in Table S11. For the two central augmented contrasts, a 2000-resample percentile bootstrap of whole families (seed 12345) provides a nominal 95% interval for the median log time ratio of both endpoints. Table S10 reports these sensitivities and the parallel Gurobi Work-unit comparison.

A word on what the instances are replicates of. The 204, 68, and 34 instances of a class are distinct operational scenarios—different inflow, demand, and cost trajectories and initial conditions—of the *same* physical system and generator, so they are not independent draws from a population of models: they share the component structure, the row taxonomy, and, where scenarios correspond to related operating periods, possibly correlated data. The paired design conditions on the instance, so each test compares variants within a scenario; but to the extent that scenarios are correlated, the effective number of independent replicates is smaller than the nominal n and the reported instance-level p -values may understate uncertainty. On the solver-time decomposition the family-sign sensitivity leaves only the augmented Simple contrast below 0.05 after adjustment; on the *prototype* preprocessing-plus-solver endpoint it leaves every Simple and SG-Ter-Mer contrast below the nominal 0.05 threshold, while on the release-build cached-metadata sensitivity, under its baseline cross-protocol construction, only the augmented Simple contrast at 0.1% remains below the reported joint and operational-family thresholds and the augmented Full contrast stays conditional on its eligible pairs (section 6.8). We therefore call the SG-Ter-Mer solver-time result nominal, read the tests as evidence about scenarios of this platform, and lean on effect sizes and cross-class consistency rather than on any single p -value (section 8).

To connect numerical and algorithmic effects, we compare solver speedup with the exported coefficient-range ratio ρ_κ and the path-specific ratio $\rho_\kappa^{\text{solver}}$. Because each instance contributes several formulations, the primary confidence intervals use 2000 cluster-bootstrap resamples of whole instances. The pooled solver-level association is negative and more pronounced than the essentially null exported-range association (table 11). This remains a weak between-formulation result: it is carried mainly by the Ruiz-kernel baselines, is essentially absent within SA-Aug and SA-Mat, and still pools heterogeneous classes. It therefore describes solver paths rather than predicting a proposed rule’s instance-level speedup. Full bootstrap accounting, within-class coefficients, and deliberately nonconfirmatory unclustered checks are reported with Table S6.

6 | COMPUTATIONAL RESULTS

The results follow the hierarchy of claims. We first report path-specific numerical diagnostics, without treating them as pure measures of representation. We then ask whether Gurobi shows lower matched runtime or deterministic effort, and use CPLEX as a contrast for the external-preprocessing effect.

The order matters because numerical improvement and algorithmic acceleration are related but not identical outcomes—a conditioning table alone would imply too much, a timing table alone too little—so the numerical and algorithmic evidence must be read together. The main pattern is clear but bounded: under Gurobi, the structure-aware variants lower the reported path-specific conditioning diagnostic and often reduce completed-run time or deterministic effort; under CPLEX 22.1.2.0, the same external scalings do not produce a robust runtime gain.

6.1 | Original-scale numerical verification across the operational campaigns

Before the per-scenario results, we report the original-scale verification promised in section 5.5 for the four operational campaigns; this is not a secondary audit. It complements the algebraic equivalence result—which is proved exactly by the positivity of the factors and needs no empirical support—by assessing a distinct question: whether the finite-precision solution the solver returns satisfies the original formulation within the stated tolerances. After every completed run, the returned solution is evaluated on the original, unscaled constraints (the row-only variants transform no variables; only Ruiz+Cols requires mapping the variables back to their original units); table 6 is a pooled completed-run diagnostic audit: it combines the four scaling variants per scenario and instance class and reports the share of returned solutions meeting the primary criterion (maximum row violation at most 10^{-4} , maximum integrality violation at most 10^{-5}), together with the median and worst row violation and

the objective deviation Δz_{rel} of equation (42). Three facts hold uniformly over the 4,806 variant runs. The integrality criterion is met in every run (worst violation 9.9×10^{-6}). The objective recomputed on the original model from the returned solution matches the solver incumbent to at most 1.0×10^{-6} , so the transformation bookkeeping itself is exact to solver precision. Every exceedance discussed below is therefore a property of the solution returned by the solver, not an artifact of rescaling or of the bookkeeping that undoes it.

The pooled rates are descriptive and are not evidence about any individual variant; the per-variant attempted, normal-termination, timeout, abort, strict-pass, and mixed-pass counts are in Table S2. At the pooled level, feasibility pass rates are high wherever the solver returns clean solutions: 89% of runs pooled over the Gurobi campaigns and 94% of the CPLEX runs outside the Simple class meet the strict criterion. The visible exception is Simple under CPLEX (32% pass). The unscaled base is the control that locates the cause: it fails the same checker at essentially the same rate (25–26% pass), on essentially the same instances (151 of the 154 base failures at 1% MIPGap recur under SA-Aug), on the same rows (balance equalities with zero right-hand side), and with the same magnitude (median residual near 3×10^{-4}), while Gurobi passes the identical instances at 99.5%. The exceedance is attributable to the solutions CPLEX returns on this class, not to the external scaling, and is consistent with feasibility tolerances being enforced on the solver’s internally scaled and presolved model rather than on the original rows. A fixed absolute threshold also explains the largest worst-case entries: a residual at solver tolerance on a scaled row maps back to ε/α_r on the original row, and the extremes sit on rows of large magnitude. The worst absolute violation in the table (7.9×10^2 , Simple under CPLEX at 0.1%) carries a relative residual of at most 1.6×10^{-6} on a constraint whose right-hand side exceeds 5×10^8 in original units, and the SG-Ter-Mer maximum of 1.1×10^{-1} is below 4.5×10^{-10} relative to its right-hand side. The remaining Gurobi exceedances are a bounded tail: the scaled variants exceed the absolute threshold in 13–15% of Simple runs (22–28% for the Ruiz row variant, 6–15% for the structure-aware variants), and in the worst cell, Full under SA-Aug at 1% (79% pass), the seven failing runs violate original rows by 2.8×10^{-4} to 2.6×10^{-3} (all but one on equality rows) with zero integrality violation and objective agreement with the base within the gap band.

Objective differences between independently gap-terminated solves are descriptive, not correctness tests: MIPGap bounds each incumbent against its own dual bound, not against another run’s incumbent. The correctness reference is therefore the best original-scale-verified incumbent on each instance, with Base judged like every variant. The 72 beyond-two-gap cases and the individual Base/Ruiz anomalies are documented with Table S2 in Supporting Information. For runs that complete but exceed the strict original-scale threshold we report two readings rather than choose one. The *intent-to-treat* analyses of the following subsections retain them as returned, since filtering on an outcome correlated with the treatment would bias the runtime comparison. A *reliability-adjusted* complement penalizes every such run as a computational failure at PAR10 cost, exactly like a timeout (the base column is essentially unchanged by this adjustment: it passes at 97.1–100% in the Gurobi scenarios). Because the strict criterion is absolute and therefore not invariant to the physical units of a row, we also report a third, *mixed* absolute–relative reading, defined precisely as follows. For each row the verifier computes the residual $v_r = (a_r^\top x - b_r)_+$ for $\leq$ rows, $v_r = (b_r - a_r^\top x)_+$ for $\geq$ rows, and $v_r = |a_r^\top x - b_r|$ for equalities, and the *right-hand-side-normalized* residual

$$v_r^{\text{rel}} = \frac{v_r}{1 + |b_r|}. \quad (50)$$

The name is deliberate: the denominator uses $|b_r|$ as a row-magnitude proxy, not the full activity $\sum_j |a_{rj}| |x_j|$ or $|a_r^\top x|$, so it under-normalizes rows whose activity is large while the right-hand side is small—in particular zero-right-hand-side balance equalities, where it reduces to the absolute residual and offers no relief. A fuller normalization by $\max\{1, |b_r|, \sum_j |a_{rj}| |x_j|\}$ is the natural refinement; we keep the right-hand-side form because it is the quantity the verifier already records and because it is the conservative choice (it never declares a zero-right-hand-side violation benign). Bound and integrality checks keep their absolute thresholds. A run passes the mixed reading if every row satisfies $v_r \leq 10^{-4}$ or $v_r^{\text{rel}} \leq \tau_{\text{rel}}$ with $\tau_{\text{rel}} = 10^{-6}$ (the order of the solvers’ feasibility tolerance), so that a tolerance-level residual on a row of order 10^8 is not treated like an operationally meaningful violation. Formally, for instance i and variant v , the intent-to-treat cost is

$$c_{iv}^{\text{ITT}} = \begin{cases} t_{iv}, & \text{if the solver terminates normally,} \\ 10T_{\text{max}}, & \text{if it reaches the time limit or aborts,} \end{cases} \quad (51)$$

and the two reliability-adjusted costs are

$$c_{iv}^k = \begin{cases} c_{iv}^{\text{ITT}}, & \text{if it does not complete, or completes and passes reading } k, \\ 10T_{\text{max}}, & \text{if it completes but fails reading } k, \end{cases} \quad k \in \{\text{strict}, \text{mixed}\}. \quad (52)$$

Thus an abort is penalized under *all three* readings, not only under the reliability-adjusted ones. On the scenario-specific common operational pool $\mathcal{I}_s$,

$$\text{PAR10}_v^{(k)} = \frac{1}{|\mathcal{I}_s|} \sum_{i \in \mathcal{I}_s} c_{iv}^{(k)}, \quad k \in \{\text{ITT, strict, mixed}\}. \quad (53)$$

For every operational cell $T_{\max} = 3600$ s, including Simple; hence every penalized run costs 36,000 s. The unpenalized cost t_{iv} is *capped solver time*: every PAR10 in this paper, including tables 5 and 21 and Tables S2 and S9, uses the solver-time cost $c_{iv}^{(k, \text{solver})}$. The preprocessing-plus-solver counterpart $c_{iv}^{(k, \text{pre+solve})}$ adds the measured preprocessing wall time to both branches; in the direct-control batches, which carry the preprocessing timer, recomputing all three readings with it shifts every reported cell by less than 0.2 s and changes no ordering (`scripts/flat_control_tests.py`). The archived main campaign predates that timer, so its PAR10 is defined on solver time only; the measured preprocessing medians (≤ 0.13 s per call) are orders of magnitude below every margin in table 5. The class separation below does not depend on the mixed-reading threshold convention: sweeping $\tau_{\text{rel}} \in \{10^{-7}, 10^{-6}, 10^{-5}\}$ leaves SG-Ter-Mer at zero failures even at the strictest setting and moves the Simple and Full failure counts by at most a few runs per cell. The full per-variant breakdown—failure counts, median and maximum violations, and PAR10 under all three readings for every scenario, class, and scaled variant—is Table S2 of the Supporting Information; the fixed-pool PAR10 under all three readings for Base/SA-Aug/SA-Mat is table 5. At 1%, the strict absolute penalty removes the advantage in every class. At 0.1%, SG-Ter-Mer is the one exception: despite one strict failure per structure-aware variant, both remain below Base because they also incur fewer non-completions (2237/1130 against 2246 s). The rounded SA-Aug margin is only 9 s (about 0.4%), whereas the SA-Mat difference is substantial. These are descriptive orderings of penalized aggregate costs, not an inferential family-level advantage, and the cell is not replicated at 1%. Thus no class retains a strict-adjusted ordering below Base *at both tolerances*. The SG-Ter-Mer exceedances fail only the absolute component of the declared strict criterion. Their right-hand-side-relative residuals are small, so under the mixed reading they are no longer classified as failures and the class returns to its ITT level below Base at both tolerances (1083/1079 against 1094 s at 1%; 1683/576 against 2246 s at 0.1%). Simple and Full remain above Base under both reliability-adjusted readings. A reader who requires replication across tolerances under the strict criterion should therefore treat the evidence as unproven; under the mixed reading the descriptive cross-tolerance ordering rests on SG-Ter-Mer.

The SA-Aug exceedances in Simple and Full arise from augmented local factors, not from the inactive coupling stage. An exploratory local-floor variant removes the observed mechanism on the twelve selected Simple failures, but this post-treatment subset cannot estimate its full-class reliability or runtime effect. We therefore retain SA-Aug without that floor as the primary variant; Table S4 contains the local-floor definition, sweep, magnitude-range-proxy cost, and limits of the probe. Counts for mechanisms not explained by the augmented right-hand-side factor are reported per variant in Table S2.

Table 5 makes the central reliability conclusion checkable in the body. It uses the scenario-specific common operational pool after excluding instances that meet the documented all-variant nonzero-return-code criterion; timeouts, aborts, and verification failures are penalized rather than dropped. The excluded counts are 5/0/0 at 1% and 4/3/2 at 0.1% for Simple/SG-Ter-Mer/Full. Table S5 gives identifiers, log evidence, and the causal assessment rather than inferring infrastructure from uniform failure alone. A conservative sensitivity that restores those instances and assigns 36,000 s to every excluded cell preserves every within-reading ordering (Table S5). Reliability adjustment is nevertheless consequential: Simple and Full move above Base under both adjusted readings. SG-Ter-Mer moves above Base under the strict reading at 1%, while both variants remain descriptively below at 0.1% (with the negligible SA-Aug margin noted above); under the mixed reading both are below Base at both tolerances. The per-variant, all-scenario breakdown, including the Ruiz baselines, is Table S2.

6.2 | Scenario 1: Gurobi with 1% MIPGap

At 1% MIPGap, completed-run timing, deterministic effort, and capped-time intent-to-treat comparisons favor the structure-aware variants in several cells. The conclusion does not survive strict original-scale reliability adjustment, under which every structure-aware class-level PAR10 is above Base. Within the descriptive completed-run metrics, SA-Mat gives the lowest shifted geometric solver time in Simple and SG-Ter-Mer and improves deterministic effort in all three classes, while SA-Aug is strongest in Full. The Ruiz-type baselines do not show the same class-consistent pattern.

Table 7 shows that SA-Mat leads the completed-run sgm T in Simple and SG-Ter-Mer while SA-Aug leads Full, with no comparable class-consistent pattern for the Ruiz baselines. The common-pool timing tails and non-completions are read from table 5,

TABLE 5 PAR10 (s) under the three readings—intent-to-treat (ITT), strict absolute, and mixed absolute–relative—for Gurobi on the scenario-specific common operational pool after uniformly excluding instances that meet the documented criterion in Table S5 (pool size n). Every timeout or abort costs $10T_{\max} = 36,000$ s under all readings; a normally terminated run that fails a reliability criterion costs the same under that criterion. Counts distinguish timeouts (n_{TO}), aborts (n_{ab}), and normally terminated runs failing strict or mixed verification ($n_{\text{fail,s}}$, $n_{\text{fail,m}}$). Bold entries are descriptive numerical comparisons with Base within the same reading; they do not denote inferential significance.

Scenario	Variant	n	PAR10 reading			n_{TO}	n_{ab}	$n_{\text{fail,s}}$	$n_{\text{fail,m}}$
			ITT	strict	mixed				
Simple, 1%	Base	199	373	554	554	0	2	1	1
	SA-Aug	199	364	2535	2535	0	2	12	12
	SA-Mat	199	183	3620	3439	0	1	19	18
SG-Ter-Mer, 1%	Base	68	1094	1094	1094	1	1	0	0
	SA-Aug	68	1083	1613	1083	1	1	1	0
	SA-Mat	68	1079	1608	1079	0	2	1	0
Full, 1%	Base	34	3385	4436	4436	3	0	1	1
	SA-Aug	34	3308	9640	9640	3	0	6	6
	SA-Mat	34	3473	5586	5586	2	1	2	2
Simple, 0.1%	Base	200	928	1108	1108	0	5	1	1
	SA-Aug	200	928	4705	3987	0	5	21	17
	SA-Mat	200	381	5776	5057	0	2	30	26
SG-Ter-Mer, 0.1%	Base	65	2246	2246	2246	1	3	0	0
	SA-Aug	65	1683	2237	1683	1	2	1	0
	SA-Mat	65	576	1130	576	0	1	1	0
Full, 0.1%	Base	32	5992	5992	5992	4	1	0	0
	SA-Aug	32	5897	8140	8140	3	2	2	2
	SA-Mat	32	6026	9293	7148	4	1	3	1

not from a second quantity with the same PAR10 label. Figure 2 shows the per-instance speedup distributions, figure 3 contrasts the two conditioning diagnostics against speedup, and figure 4 gives the corresponding fixed-pool Dolan–Moré profiles defined in equation (49).

6.3 | Scenario 2: Gurobi with 0.1% MIPGap

The stricter Gurobi tolerance tests the descriptive completed-run timing pattern under a harder stopping criterion, and that pattern largely persists: SA-Mat again gives the best shifted geometric solver time in Simple and SG-Ter-Mer and SA-Aug is stronger in Full, while the Ruiz-type variants remain unstable. This statement is distinct from the strict- and mixed-adjusted PAR10 readings in table 5. Since the matrix-only rule uses no right-hand-side information, much of the useful structural scale information in this testbed is already present in the coefficient matrix.

Table 8 confirms that the descriptive completed-run timing pattern does not hinge on the looser stopping criterion. This statement is distinct from the strict- and mixed-adjusted PAR10 readings in table 5; the distributional and diagnostic views are in Supporting Information, Figures S1–S3.

Table 9 separates two numerical notions that are easy to conflate. Both structure-aware variants drive the solver-level ratio well below one (SA-Mat brings $\rho_{\kappa}^{\text{solver}}$ to 0.07–0.08 in Full and SG-Ter-Mer while compressing ρ_{κ} much like SA-Aug), whereas Ruiz+Cols may compress the exported matrix while worsening the solver-level diagnostic in Full—so the geometric-mean kernel is not interchangeable with the Ruiz kernel even under the same structural pipeline.

The paired tests in table 10 support the descriptive Gurobi pattern. The signs are favorable for both structure-aware variants, and after Benjamini–Hochberg adjustment both are significant in Simple and SG-Ter-Mer in both Gurobi scenarios. In Full, neither SA-Aug result is significant after adjustment ($q_{\text{BH}} = 0.15$ at 1% and 0.11 at 0.1%); the latter changes from the earlier complete-case result once one-sided aborts are retained at the cap. Given the small sample, censoring, and run-to-run variability, Full remains directional rather than inferential evidence. The paired effect sizes range from small or moderate to large (r_{tb} from

TABLE 6 Pooled completed-run diagnostic audit for original-scale reliability in the four operational campaigns. Each returned solution is evaluated on the original, unscaled constraints (variables are mapped back to original units only for the column-scaling baseline); a run passes if the maximum row violation is at most 10^{-4} and the maximum integrality violation at most 10^{-5} (section 5.5). The “pass” column is a *conditional pass rate among completed runs*: it is not a campaign-wide reliability rate, because solver aborts are excluded from its denominator here (they are instead penalized as failures in the PAR10 readings of table 5). Row violations are absolute, in original units; Δz_{rel} is equation (42) against the base incumbent on the same instance (descriptive; the correctness reference is the verified incumbent z_i^{ver} of equation (43)). Pooling can conceal variant heterogeneity and is not used to infer the reliability of a particular method; the per-variant counts are in Table S2.

Scenario	Class	n^a	Cond. pass (%)	Base (%) ^b	viol. med.	viol. max.	Δz_{rel} med.	Δz_{rel} max.
Gurobi, 1%	Simple	792	86.7	99.5	2.9×10^{-6}	4.9×10^{-3}	3.8×10^{-5}	8.7×10^{-1}
	Full	134	89.6	97.1	4.8×10^{-6}	2.6×10^{-3}	9.2×10^{-4}	1.8×10^2
	SG-Ter-Mer	270	98.1	100.0	6.8×10^{-7}	1.1×10^{-1}	7.9×10^{-5}	2.1×10^{-2}
Gurobi, 0.1%	Simple	787	85.4	99.5	3.5×10^{-6}	1.2	2.7×10^{-5}	8.7×10^{-1}
	Full	122	90.2	100.0	4.8×10^{-6}	4.5×10^{-3}	6.3×10^{-5}	1.8×10^2
	SG-Ter-Mer	255	98.4	100.0	6.6×10^{-7}	1.1×10^{-1}	1.5×10^{-14}	2.1×10^{-2}
CPLEX, 1%	Simple	816	32.0	24.5	2.3×10^{-4}	4.4×10^{-1}	1.4×10^{-15}	7.3×10^{-2}
	Full	136	80.9	75.8	1.4×10^{-5}	1.3×10^{-3}	1.0×10^{-3}	5.3×10^1
	SG-Ter-Mer	272	100.0	100.0	6.8×10^{-7}	1.4×10^{-5}	4.3×10^{-14}	9.7×10^{-3}
CPLEX, 0.1%	Simple	816	32.6	26.0	2.4×10^{-4}	7.9×10^2	4.3×10^{-15}	1.1×10^{-1}
	Full	134	83.6	84.8	1.3×10^{-5}	6.2×10^{-4}	7.0×10^{-6}	5.3×10^1
	SG-Ter-Mer	272	100.0	100.0	6.9×10^{-7}	1.4×10^{-5}	9.1×10^{-15}	2.0×10^{-3}

^a Runs with returned solver output, pooled over SA-Aug, SA-Mat, Ruiz, and Ruiz+Cols; time-limited runs with output are included, whereas solver or pipeline aborts are not in the denominator here (they are penalized in table 5). Three runs lacking verification output are counted as non-passing.

^b Pass rate of the unscaled base under the same checker (n per scenario: 195–204 Simple, 31–34 Full, 62–68 SG-Ter-Mer).

TABLE 7 At 1% MIPGap, structure-aware variants improve several descriptive completed-run Gurobi efficiency metrics in Simple and SG-Ter-Mer; fixed-pool intent-to-treat and reliability-adjusted PAR10 are reported in table 5.

Class	Variant	n	sgm T	med. sp. ^{det}	$\bar{\gamma}_{\text{cb}}$
Simple	Base	197	4.05	1.00	0.245
	SA-Aug	197	2.20	1.07	0.242
	SA-Mat	198	2.03	1.06	0.241
	Ruiz	198	8.27	0.88	0.300
	Ruiz+Cols	199	7.52	1.02	0.233
Full	Base	34	82.93	1.00	0.120
	SA-Aug	34	70.56	1.37	0.075
	SA-Mat	34	77.82	1.15	0.055
	Ruiz	33	126.94	1.13	0.263
	Ruiz+Cols	33	75.89	1.32	0.239
SG-Ter-Mer	Base	67	14.14	1.00	0.664
	SA-Aug	67	8.39	1.32	0.566
	SA-Mat	67	6.75	1.40	0.531
	Ruiz	68	14.40	1.00	0.639
	Ruiz+Cols	68	18.35	0.78	0.662

Notes. sgm T : shifted geometric mean of solver time in seconds. med. sp.^{det}: median deterministic speedup with respect to the base model. $\bar{\gamma}_{\text{cb}}$: time-averaged relative gap over the callback horizon. Lower is better for sgm T and $\bar{\gamma}_{\text{cb}}$; higher is better for med. sp.^{det}. Bold values mark the best descriptive value within each class and metric. Fixed-pool ITT and reliability-adjusted PAR10 are reported only in table 5 and Table S2, so no variant-specific available-run penalty is mixed into this table.

757 –0.23 to –0.90), with the strongest paired shifts appearing for SA-Mat in SG-Ter-Mer; the evidence is a distributional shift, not
758 a guarantee that every instance accelerates.

759 Table 11 adds the aggregate correlation check. Across the pooled family of formulations, the own-incumbent path-specific fixed-
760 LP diagnostic is more strongly associated with speedup than the exported-range proxy (which is essentially uninformative); the
761 instance-clustered bootstrap confirms that this contrast survives the within-instance dependence (section 5.5). The association is

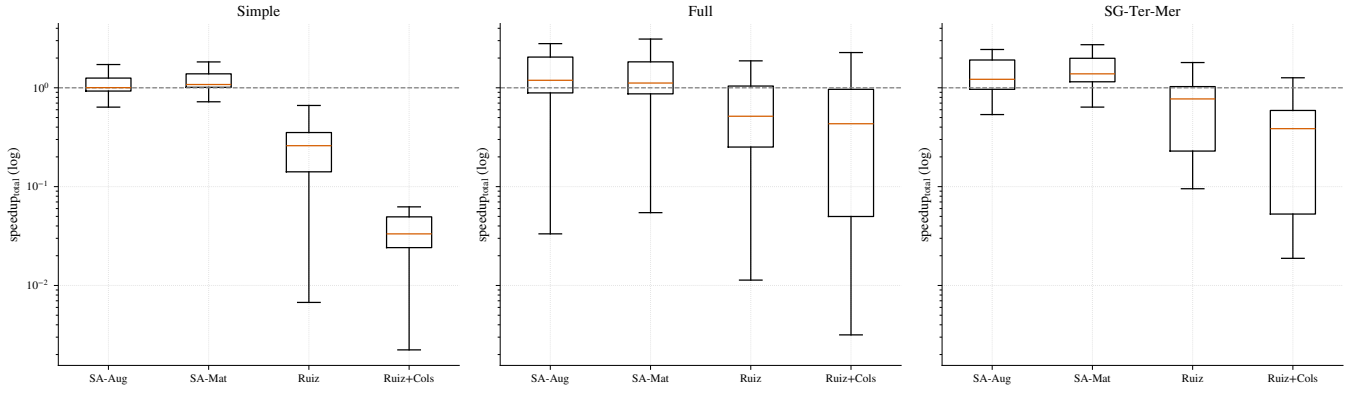


FIGURE 2 At 1% MIPGap, the Gurobi total-time speedup distributions place the structure-aware variants mostly above parity, while Ruiz-type baselines are more dispersed. The axis is the logged *total*-time speedup of the main campaign (equation (46), $q = \text{total}$). Its preprocessing component has whole-second resolution: the Ruiz overhead is visible and pulls those medians well below the solver-time and deterministic-effort parity of table 7 and the $\rho_{\kappa}^{\text{solver}}$ scatter, whereas the subsecond structure-aware overhead is rounded to zero (section 5.5). The figure should therefore not be read as a precise preprocessing-plus-solver comparison for the structure-aware variants; those come from the instrumented control batches of section 6.8. Values above the parity line indicate acceleration relative to the unscaled model. In all box plots, boxes span the interquartile range (IQR) and whiskers extend to the most extreme observation within $1.5 \times \text{IQR}$; outliers are omitted, since tail behavior is captured by PAR10 and the performance profiles.

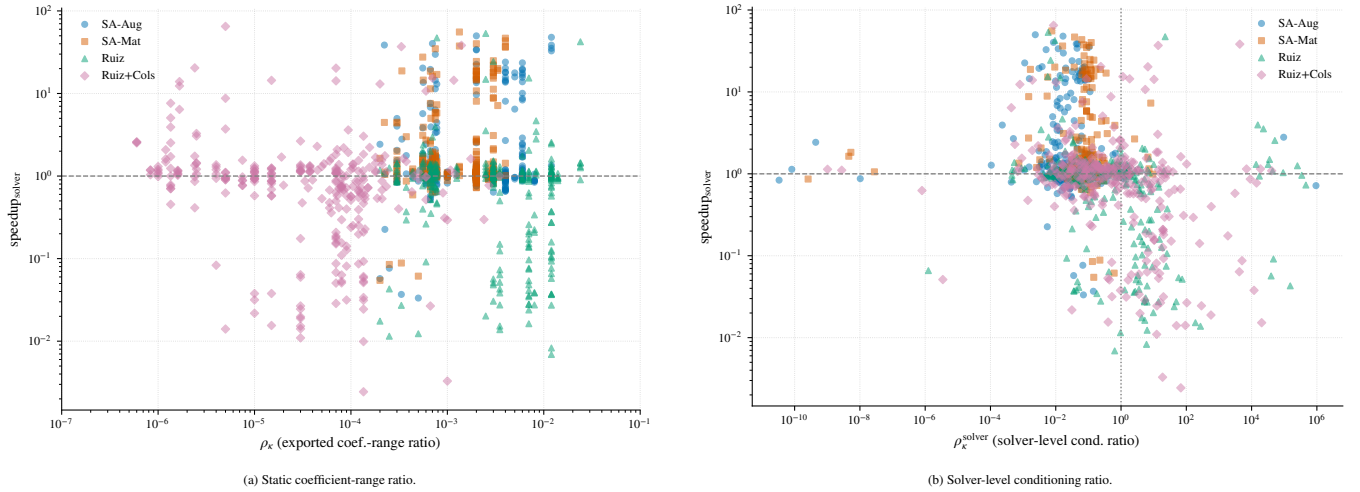


FIGURE 3 At 1% MIPGap, static coefficient-range compression and solver-level conditioning tell different stories. The contrast between panels shows why exported matrix compression alone should not be treated as evidence of easier branch-and-bound search. In panel (a) every static ratio lies below one, so the $\rho_{\kappa} = 1$ reference sits outside the plotted range.

weak, driven mainly by the Ruiz-kernel variants, and provides no useful per-instance runtime prediction for SA-Aug or SA-Mat (essentially null within either; see the table notes). The result supports using the own-incumbent fixed-LP quantity as a pooled description of solver paths rather than treating the exported-range proxy as an explanation of runtime. It does not establish that the former is a purer measure of representation, nor does it provide an instance-level predictor for SA-Aug or SA-Mat.

Together with tables 9 and 11, these results support a two-step interpretation: structure-aware scaling changes the numerical representation seen by Gurobi, and—across the pooled family of formulations, and primarily because of the Ruiz-kernel variants—the own-incumbent path-specific fixed-LP diagnostic is more associated with speedup than the exported-range proxy. This association is not present strongly enough within SA-Aug or SA-Mat to serve as a per-instance runtime predictor for the proposed rules.

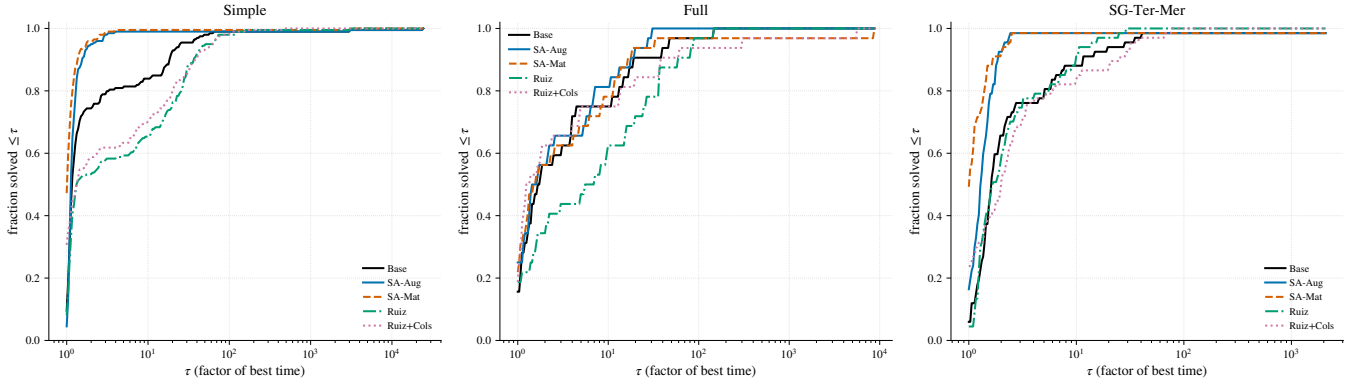


FIGURE 4 At 1% MIPGap, fixed-pool Dolan–Moré profiles use the common denominators and 36,000-s individual failure cost of equation (49). Curves farther toward the upper-left indicate variants that more often lie close to the best admissible time.

TABLE 8 At 0.1% MIPGap, completed-run Gurobi efficiency metrics retain the same descriptive pattern; fixed-pool PAR10 is reported in table 5.

Class	Variant	n	sgm T	med. sp. ^{det}	$\overline{\gamma}_{cb}$
Simple	Base	195	8.28	1.00	0.176
	SA-Aug	195	5.65	1.11	0.180
	SA-Mat	198	5.44	1.10	0.182
	Ruiz	199	14.12	0.88	0.261
	Ruiz+Cols	195	12.55	1.02	0.199
Full	Base	31	305.30	1.00	0.027
	SA-Aug	30	226.90	1.26	0.028
	SA-Mat	31	306.39	1.12	0.025
	Ruiz	30	341.56	0.87	0.027
	Ruiz+Cols	31	267.10	1.62	0.022
SG-Ter-Mer	Base	62	14.49	1.00	0.642
	SA-Aug	63	8.99	1.32	0.473
	SA-Mat	65	7.97	1.35	0.439
	Ruiz	65	14.27	1.05	0.633
	Ruiz+Cols	62	19.16	0.78	0.625

Notes. Bold values mark the best descriptive value within each class and metric; lower is better for sgm T and $\overline{\gamma}_{cb}$, whereas higher is better for med. sp.^{det}. $\overline{\gamma}_{cb}$ is complementary to time and has no direct duration penalty.

TABLE 9 Structure-aware variants are associated with lower own-incumbent path-specific fixed-LP conditioning diagnostics under Gurobi at 0.1% MIPGap, whereas static coefficient compression alone is not.

Class	Variant	n	median ρ_κ (IQR)	median $\rho_\kappa^{\text{solver}}$ (IQR)	warnings
Simple	SA-Aug	195	4.0×10^{-3} (4.0×10^{-3})	0.04 (0.08)	1.0
	SA-Mat	195	2.0×10^{-3} (1.0×10^{-3})	0.08 (0.08)	1.0
	Ruiz	192	0.01 (0.01)	0.75 (3.94)	2.0
	Ruiz+Cols	190	1.5×10^{-5} (6.7×10^{-5})	1.19 (12.70)	1.0
Full	SA-Aug	30	5.6×10^{-4} (3.7×10^{-4})	0.02 (0.04)	1.0
	SA-Mat	31	5.6×10^{-4} (3.7×10^{-4})	0.07 (0.08)	1.0
	Ruiz	29	5.6×10^{-4} (3.7×10^{-4})	0.47 (2.55)	2.0
	Ruiz+Cols	30	6.0×10^{-4} (8.0×10^{-4})	1.60 (24.82)	0.0
SG-Ter-Mer	SA-Aug	62	6.7×10^{-4} (1.8×10^{-4})	0.01 (0.01)	1.0
	SA-Mat	61	6.7×10^{-4} (1.8×10^{-4})	0.08 (0.02)	1.0
	Ruiz	61	6.7×10^{-4} (1.8×10^{-4})	0.06 (0.06)	1.0
	Ruiz+Cols	59	1.0×10^{-4} (8.9×10^{-5})	0.07 (0.11)	0.0

Notes. Values below one indicate improvement relative to the base on the same scenario. ρ_κ is the exported coefficient-range proxy, whereas $\rho_\kappa^{\text{solver}}$ is computed from the solver-level conditioning diagnostic on the fixed-integer LP. Because each variant fixes its own incumbent, these ratios describe the numerics along each variant's solution path and do not compare one common LP (section 5.5). All ratios are strictly positive; the smallest are shown in scientific notation. The warnings column summarizes solver numerical warning flags parsed from logs and is used only as a descriptive diagnostic.

TABLE 10 Paired Gurobi tests show a distributional runtime shift in favor of the structure-aware variants, especially in Simple and SG-Ter-Mer.

Scenario	Class	Variant	n	W	p	q_{BH}	r_{rb}
1% MIPGap	Simple	SA-Aug	197	7535.0	0.0057	0.0097	-0.227
1% MIPGap	Simple	SA-Mat	198	3011.0	$< 10^{-4}$	$< 10^{-4}$	-0.694
1% MIPGap	Full	SA-Aug	32	175.0	0.0984	0.1476	-0.337
1% MIPGap	Full	SA-Mat	32	204.0	0.2699	0.3599	-0.227
1% MIPGap	SG-Ter-Mer	SA-Aug	66	501.0	0.0001	0.0005	-0.547
1% MIPGap	SG-Ter-Mer	SA-Mat	66	115.0	$< 10^{-4}$	$< 10^{-4}$	-0.896
0.1% MIPGap	Simple	SA-Aug	195	6963.0	0.0010	0.0031	-0.271
0.1% MIPGap	Simple	SA-Mat	198	7069.0	0.0006	0.0023	-0.282
0.1% MIPGap	Full	SA-Aug	28	124.0	0.0735	0.1103	-0.389
0.1% MIPGap	Full	SA-Mat	27	173.0	0.7140	0.7140	-0.085
0.1% MIPGap	SG-Ter-Mer	SA-Aug	62	456.0	0.0003	0.0016	-0.533
0.1% MIPGap	SG-Ter-Mer	SA-Mat	64	341.0	$< 10^{-4}$	$< 10^{-4}$	-0.672

Notes. Tests are paired against the base formulation on the scenario-specific eligible matched pool. r_{rb} is the matched-pairs rank-biserial correlation computed from the same signed ranks as W ; negative values mean the variant tends to be faster. Double timeouts and other double non-completions are excluded; a one-sided timeout is censored at T_{max} , whereas assigning T_{max} to a one-sided abort is the operational convention tested by the 10 T_{max} sensitivity. Tests are two-sided, without continuity correction; Simple and SG-Ter-Mer use the normal approximation and Full the exact null distribution, as specified in section 5.5. The displayed n is therefore the scenario-specific eligible matched pool, not the full class pool. Table S6 decomposes attrition into uniformly unavailable instances, double timeouts, double aborts or mixed double non-completions, one-sided capped observations, missing times, and Wilcoxon zeros; no zero or tied-absolute-difference pair occurs in these twelve rows. These tests use capped ITT solver times; they are not inferential tests of the strict- or mixed-adjusted PAR10 orderings in table 5.

TABLE 11 Across the pooled scaling family, the own-incumbent path-specific fixed-LP diagnostic is more associated with Gurobi speedup than the exported-range proxy; within the structure-aware variants the association is essentially null.

Scenario	$\rho_S(\rho_\kappa, \text{sp})$	$\rho_S(\rho_\kappa^{\text{solver}}, \text{sp})$	solver 95% CI	$(\rho^{\text{sol}} - \rho^{\text{prox}})$ CI
1% MIPGap	+0.002	-0.256	[-0.32, -0.19]	[0.16, 0.30]
0.1% MIPGap	+0.010	-0.267	[-0.34, -0.20]	[0.16, 0.32]

Notes. sp denotes solver speedup; ρ_S is the pooled Spearman correlation over all four scaled variants. The confidence intervals are 95% instance-clustered bootstrap intervals (2000 resamples of whole instances, 298 clusters at 1% and 288 at 0.1%; full accounting is reported with Table S6)—the primary inference, since each instance contributes four correlated rows: the solver-level association is negative and its magnitude exceeds the proxy's (the difference CI excludes zero). Decomposed by variant, the negative solver-level association is carried by the Ruiz baselines ($\rho_S \approx -0.33$ for Ruiz and -0.36 for Ruiz+Cols at 1%), whereas for the structure-aware variants it is essentially null (SA-Aug -0.06 ; SA-Mat $+0.11$), with the same pattern at 0.1%, so it provides no per-instance runtime prediction for the proposed rules.

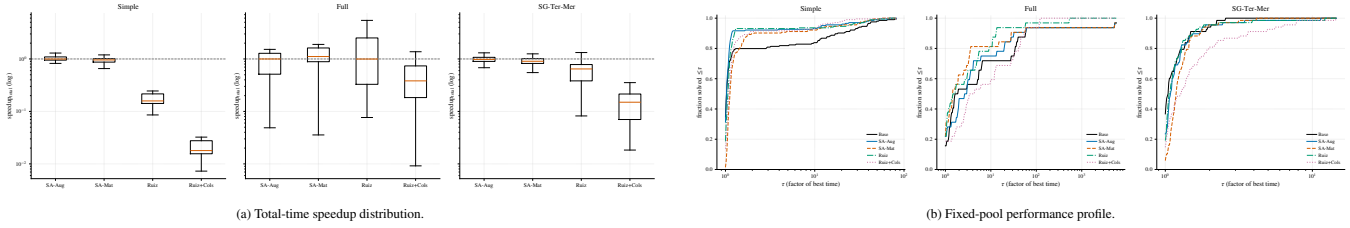


FIGURE 5 At 1% MIPGap, the CPLEX distributional view and the common-pool, failure-penalized profile of equation (49) break the Gurobi pattern. The plots therefore serve as a solver contrast.

6.4 | Scenario 3: CPLEX with 1% MIPGap

The CPLEX scenarios provide a negative solver contrast to the Gurobi results. The structured variants remain close to parity and do not reproduce the systematic deterministic gains observed under Gurobi. Some descriptive wall-time improvements appear, especially in Full for SA-Mat, but the paired tests do not support robust structure-aware acceleration. In fact, some CPLEX signs are unfavorable for the structured variants. The correct interpretation is therefore not that CPLEX improves less than Gurobi, but that the present external scaling layer does not reliably improve CPLEX runtime in this environment. Ruiz is sometimes competitive in Full, but its performance is not stable across classes.

For CPLEX, table 12 has to be interpreted differently from the Gurobi tables: the class-consistent structure-aware pattern is absent. The Simple class shows the descriptive-paired dissociation most clearly: SA-Mat lowers the completed-run class *sgm* ($2.23 \rightarrow 1.68$ s), yet its paired test is significantly *unfavorable* ($r_{tb} = +0.29$, with 61 paired wins against 143 losses, table 14). A few hard instances in the tail move the aggregate while the typical instance slows slightly. Fixed-pool CPLEX PAR10 values are reported in Table S2. Figure 5 shows the corresponding distributional and fixed-pool profile views.

6.5 | Scenario 4: CPLEX with 0.1% MIPGap

The stricter CPLEX setting confirms the same solver dependence and makes the negative runtime conclusion clearer. The structured variants remain at or near parity in deterministic effort, and the clearest descriptive improvement in shifted geometric solver time comes from Ruiz rather than from the proposed structure-aware rules; the matrix-only rule does lower the shifted geometric time on Full (from 421.6 to 370.5 s), but this is descriptive only, as the paired test on that class is not significant ($p = 0.21$). This is a useful negative result: external scaling may improve some numerical diagnostics without becoming a robust runtime accelerator for CPLEX in this environment.

Table 13 reports the cells; the corresponding distributional and profile views (Figure S4, Supporting Information) remain mixed under the stricter tolerance, supporting the solver-dependent interpretation.

The paired CPLEX tests in table 14 keep the descriptive tables in perspective. Unlike the Gurobi evidence, the signs are small, mixed, and sometimes unfavorable; after multiplicity adjustment they do not support a robust positive runtime effect for the structure-aware variants.

The CPLEX runs nevertheless carry their own conditioning evidence, recorded as tree-level KappaStats (section 5.5), and it moves in the same direction as the Gurobi diagnostics. The median sampled-basis condition ratio of the structure-aware variants relative to the base is below one in Simple (0.84–0.87 at 1%, 0.80–0.81 at 0.1%) and drops by four to five orders of magnitude in Full ($\approx 5 \times 10^{-5}$ at 1%, $\approx 2\text{--}3 \times 10^{-6}$ at 0.1%), while CPLEX’s numerical attention score on Full falls from 0.0038 for the base to zero and the sampled unstable fraction vanishes; SG-Ter-Mer sits near parity (0.90–0.99). This descriptive path-diagnostic pattern appears under both solvers, but CPLEX does not convert it into a robust runtime effect.

Summarizing the four scenarios by their best descriptive variant makes the solver contrast visible at a glance: the structure-aware rules hold the lowest shifted geometric times in every Gurobi cell (SA-Mat in Simple and SG-Ter-Mer, SA-Aug in Full, at both tolerances), whereas the CPLEX winners are heterogeneous—varying from SA-Aug to Base to Ruiz by class and tolerance—and must be read together with the paired Wilcoxon tests, which do not support a robust positive structure-aware effect there.

TABLE 12 At 1% MIPGap, CPLEX does not reproduce the Gurobi structure-aware acceleration pattern.

Class	Variant	n	sgm T	med. sp. ^{det}	$\bar{\gamma}_{cb}$
Simple	Base	204	2.23	1.00	0
	SA-Aug	204	1.46	1.04	0
	SA-Mat	204	1.68	1.03	0
	Ruiz	204	1.49	0.99	0
	Ruiz+Cols	204	1.51	0.96	0
Full	Base	33	158.38	1.00	3.1×10^{-8}
	SA-Aug	34	165.43	0.98	3.2×10^{-8}
	SA-Mat	34	115.60	1.00	2.3×10^{-8}
	Ruiz	34	115.96	1.14	2.1×10^{-8}
	Ruiz+Cols	34	225.94	0.70	5.5×10^{-8}
SG-Ter-Mer	Base	68	2.21	1.00	1.0×10^{-10}
	SA-Aug	68	2.37	1.02	1.0×10^{-10}
	SA-Mat	68	2.41	1.00	1.1×10^{-10}
	Ruiz	68	2.33	1.01	9.7×10^{-11}
	Ruiz+Cols	68	3.18	0.88	1.0×10^{-10}

Notes. Bold values mark the best descriptive value within each class and metric. Since the CPLEX statistical tests do not support a robust positive external-scaling effect, these bold entries are descriptive only. $\bar{\gamma}_{cb}$ is accumulated from CPLEX's own callback stream, has no direct duration penalty, and is comparable within this table but not against the Gurobi tables (section 5.5).

TABLE 13 At 0.1% MIPGap, CPLEX remains close to parity for the structure-aware variants, confirming solver dependence.

Class	Variant	n	sgm T	med. sp. ^{det}	$\bar{\gamma}_{cb}$
Simple	Base	204	5.14	1.00	1.2×10^{-12}
	SA-Aug	204	4.20	1.02	6.5×10^{-13}
	SA-Mat	204	4.48	1.01	5.6×10^{-13}
	Ruiz	204	4.13	1.00	1.0×10^{-15}
	Ruiz+Cols	204	5.28	0.94	7.7×10^{-13}
Full	Base	33	421.59	1.00	2.7×10^{-8}
	SA-Aug	34	422.62	1.07	2.7×10^{-8}
	SA-Mat	33	370.45	1.05	2.3×10^{-8}
	Ruiz	34	285.10	1.28	1.9×10^{-8}
	Ruiz+Cols	33	377.41	0.96	5.2×10^{-8}
SG-Ter-Mer	Base	68	2.82	1.00	1.1×10^{-10}
	SA-Aug	68	2.84	1.01	1.0×10^{-10}
	SA-Mat	68	2.84	1.00	1.1×10^{-10}
	Ruiz	68	2.61	1.02	1.1×10^{-10}
	Ruiz+Cols	68	4.02	0.85	1.3×10^{-10}

Notes. Bold values mark the best descriptive value within each class and metric. Under CPLEX, these descriptive minima do not imply a robust positive external-scaling effect. $\bar{\gamma}_{cb}$ is the dimensionless callback-horizon average in equation (48) and does not itself penalize duration.

6.6 | Controlled synthetic mechanism check

The operational experiments above test the full solver–model interaction. The synthetic benchmark has a different role (figure 6): it checks the numerical mechanism under controlled row-scale imbalance. The grid uses $B = 10$ blocks, each with 8 continuous and 8 binary variables, 8 local constraints plus binary-activation rows, and 3 global coupling rows with feasibility slacks. Row imbalance is injected through the pattern $\{none, local, coupling, coupling-uniform, mixed\}$ and $S \in \{0, 3, 6\}$: in *coupling* each selected row r draws its own multiplier $\theta_r = 10^{\xi_r}$, $\xi_r \sim \mathcal{U}[-S, S]$, whereas in *coupling-uniform* all coupling rows share a single draw per instance (section 5.2). The grid contains 20 seeds per pattern–severity cell, for 300 instances and 1800 variant runs (the five variants of section 5.3 plus the role-blind flat control of section 6.8).

The design behaves as intended. For the negative-control pattern *none*, the median pre-scaling magnitude-range proxy is essentially invariant in S , around 1.3×10^3 . When imbalance is injected, $\hat{\kappa}_{range}^{pre}$ grows with severity only in the affected row

TABLE 14 Paired CPLEX tests do not support a robust positive runtime effect for the structure-aware variants.

Scenario	Class	Variant	n	W	p	q_{BH}	r_{rb}
1% MIPGap	Simple	SA-Aug	204	9377.0	0.2016	0.3456	-0.103
1% MIPGap	Simple	SA-Mat	204	7460.0	0.0004	0.0023	+0.286
1% MIPGap	Full	SA-Aug	29	172.0	0.3357	0.4284	+0.209
1% MIPGap	Full	SA-Mat	28	131.0	0.1042	0.2083	-0.355
1% MIPGap	SG-Ter-Mer	SA-Aug	68	1068.0	0.5211	0.5211	+0.090
1% MIPGap	SG-Ter-Mer	SA-Mat	68	740.0	0.0082	0.0326	+0.369
0.1% MIPGap	Simple	SA-Aug	204	8765.0	0.0453	0.1213	-0.162
0.1% MIPGap	Simple	SA-Mat	203	10056.0	0.7230	0.7230	-0.029
0.1% MIPGap	Full	SA-Aug	27	173.0	0.7140	0.7230	-0.085
0.1% MIPGap	Full	SA-Mat	26	125.0	0.2079	0.3563	-0.288
0.1% MIPGap	SG-Ter-Mer	SA-Aug	68	1085.0	0.5908	0.7089	+0.075
0.1% MIPGap	SG-Ter-Mer	SA-Mat	68	985.0	0.2507	0.3723	-0.160

Notes. Tests are paired against the base formulation on matched instances. r_{rb} is the matched-pairs rank-biserial correlation from the same signed ranks as W (negative = variant faster). Under CPLEX, the effect signs are small, inconsistent, or unfavorable. They do not support a robust positive structure-aware runtime effect. The displayed n is the scenario-specific eligible matched pool: one-sided non-completions enter at $T_{\max}$ and double non-completions are excluded. Table S6 gives the attrition audit; no zero-difference pair occurs in the displayed rows.

TABLE 15 Controlled synthetic benchmark: median global coefficient-and-right-hand-side magnitude-range proxy before scaling and after each variant, with worst original-scale violations across the selected cells.

Pattern	S	$\hat{\kappa}_{\text{range}}^{\text{pre}}(\text{Base})$	$\hat{\kappa}_{\text{range}}^{\text{post}}(\text{SA-Aug})$	$\hat{\kappa}_{\text{range}}^{\text{post}}(\text{SA-Mat})$	$\hat{\kappa}_{\text{range}}^{\text{post}}(\text{Ruiz})$	$\hat{\kappa}_{\text{range}}^{\text{post}}(\text{Flat})$	v_{row}	v_{int}
none	0	1.34×10^3	1.26×10^4	2.49×10^3	1.19×10^4	1.09×10^3	2.27×10^{-13}	0
local	3	1.84×10^8	1.44×10^4	4.68×10^3	3.51×10^4	1.07×10^3	2.91×10^{-11}	0
local	6	1.49×10^{14}	1.38×10^4	4.02×10^3	3.77×10^4	1.07×10^3	1.49×10^{-8}	0
coupling	3	6.86×10^5	3.47×10^4	2.53×10^3	6.06×10^4	1.22×10^3	1.16×10^{-10}	0
coupling	6	4.87×10^8	3.33×10^5	2.49×10^3	3.58×10^5	1.22×10^3	1.19×10^{-7}	2.03×10^{-6}
c.-unif., $\theta > 1$ ($n=7$)	3	3.16×10^5	3.99×10^3	2.57×10^3	4.52×10^3	1.05×10^3	1.16×10^{-10}	0
c.-unif., $\theta \leq 1$ ($n=13$)	3	4.11×10^4	4.82×10^3	2.51×10^3	4.46×10^3	1.30×10^3		
c.-unif., $\theta > 1$ ($n=7$)	6	6.94×10^7	3.78×10^3	2.57×10^3	4.37×10^3	1.05×10^3	2.38×10^{-7}	3.27×10^{-7}
c.-unif., $\theta \leq 1$ ($n=13$)	6	3.19×10^6	2.36×10^5	2.50×10^3	2.68×10^5	1.29×10^3		
mixed	3	2.06×10^8	5.00×10^4	4.68×10^3	4.86×10^4	1.18×10^3	5.82×10^{-11}	0
mixed	6	1.52×10^{14}	6.01×10^6	4.02×10^3	6.01×10^6	1.22×10^3	1.31×10^{-6}	1.28×10^{-7}

Notes. Values are medians by pattern, severity, and variant, except for the two violation columns, which report worst original-scale violations in the corresponding pattern-severity cell (the coupling-uniform cells are reported whole). The grid contains 20 seeds per cell; the coupling-uniform rows are split by the direction of the shared factor θ , with subgroup sizes in the row labels. Dispersion for the key contrast at $S = 6$: SA-Aug spans $[3.4, 5.2] \times 10^3$ (interquartile) on the oversized branch it repairs, and $[3.8 \times 10^3, 5.2 \times 10^6]$ on the undersized branch its one-sided trigger leaves untreated, whose pre-scaling proxy is itself $[3.8 \times 10^4, 6.9 \times 10^7]$. The synthetic benchmark is a magnitude-range and reliability check, not a runtime benchmark. Ruiz+Cols is omitted as a column: the synthetic instances carry no unit metadata, so its physical-normalization step is inert and it coincides with Ruiz everywhere. All 1800 runs pass the original-scale verification with the reliability floor of section 4.4 enabled (its ablation is analyzed in the text).

classes: at $S = 6$, it reaches about 1.5×10^{14} for *local* and *mixed*, about 4.9×10^8 for *coupling*, and about 3.2×10^7 for *coupling-uniform*. This confirms that the synthetic generator separates local-row, per-row coupling, and collective coupling imbalance by construction (figure 6a).

The shared per-row kernel is validated cleanly. The two variants that normalize every row they reach individually recover the injected magnitude range in every pattern: the full-coverage flat control brings the median $\hat{\kappa}_{\text{range}}^{\text{post}}$ to $1.1\text{--}1.3 \times 10^3$ everywhere (about eleven orders of recovery for *local* and *mixed* at $S = 6$), and SA-Mat, whose coupling treatment is also per row, stays within $2.5\text{--}4.7 \times 10^3$ (table 15 and figure 6b). A paired Wilcoxon signed-rank analysis with Benjamini–Hochberg correction gives $p_{BH} \leq 1.8 \times 10^{-2}$ for every magnitude-range-proxy comparison and $p_{BH} \leq 4.5 \times 10^{-6}$ for the strong recoveries. Because each injected disturbance is a positive complete-row multiplier, full-coverage geometric-mean normalization cancels it whenever no safeguard or near-identity band binds. Flat is therefore an oracle-like recovery baseline for the injected mechanism. For $\hat{\kappa}_{\text{range}}$, exact per-row centering attains the lower bound given by the largest within-row augmented spread when every factor is

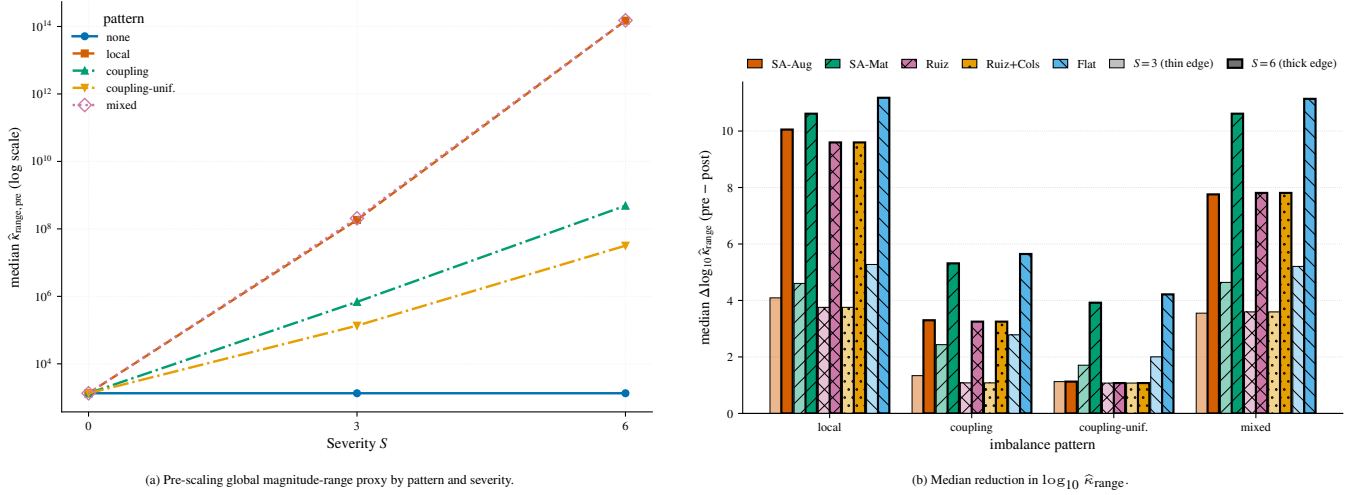


FIGURE 6 Controlled synthetic mechanism check. Panel (a) verifies that the injected scale imbalance is controlled by the pattern and severity parameters. Panel (b) shows recovery of the global coefficient-and-right-hand-side magnitude-range proxy by the four scaling variants and the role-blind flat control on the four imbalance patterns at $S \in \{3, 6\}$. The Base variant is omitted because its reduction is zero by construction; the *none* pattern is omitted from this panel for readability and reported in table 15, where, unlike Base, some rules *worsen* the proxy on that negative control. The coupling-uniform group pools both directions of the shared factor; table 15 splits them. Bars are hatched by variant and outlined with a thin/thick edge (and a lighter/darker fill) for $S = 3/S = 6$, so the grouping reads without color.

applied: every row scaling preserves that within-row spread, while centering all row intervals around one attains their maximum. This statement implies no global optimality for the spectral condition number $\kappa_2(A)$, which is not measured in this synthetic table. What the benchmark can separate is how the *structure-aware* designs behave on rows assigned to the coupling stage.

There the two structure-aware coupling designs separate exactly where the analysis says they should. On *coupling-uniform*, the collective displacement the block-relative stage is designed for, the stage activates and repairs its *oversized* branch: the single factor γ drives the coupling proxy from $\hat{\rho} \approx 10^2$ – 10^5 to $\hat{\rho} \approx 1$ and the median $\hat{\kappa}_{\text{range}}$ from 6.9×10^7 to 3.8×10^3 at $S = 6$ on the $\theta > 1$ subgroup. The activation rule is one-sided by design—the stage fires only when the coupling rows are *large* relative to the blocks ($\hat{\rho} > 10$)—so the undersized subgroup is left at base scale ($3.2 \times 10^6 \rightarrow 2.4 \times 10^5$), which the per-row treatments repair (SA-Mat 2.5×10^3 , flat 1.3×10^3). The stage is therefore validated for oversized collective coupling imbalance only; a symmetric or trigger-free variant (letting the minimization of Φ decide the direction) is untested design space, and not trivially safe to adopt: operationally the Simple class sits deep in the undersized regime ($\hat{\rho} \approx 3 \times 10^{-4}$), where the one-sided rule keeps the stage silent and the activation audit confirms that full and local-only exports coincide there (section 6.8). The complete trigger sweep is reported after Table S3. No displayed cell median changes because the proxy is effectively bimodal, so the one-sided design—not the specific cutoff 10—is what leaves the undersized branch untreated. On the heterogeneous *coupling* pattern the single collective factor cannot compress the spread *among* the coupling rows: with the reliability floor of section 4.4 protecting the individually small rows, SA-Aug recovers partially ($4.9 \times 10^8 \rightarrow 3.3 \times 10^5$ at $S = 6$) while SA-Mat and the flat kernel recover fully (2.5×10^3 and 1.2×10^3): the per-row versus collective design choice, not the metadata itself, is what the coupling patterns discriminate. Two further observations keep the attribution honest. First, the Ruiz variants run inside the same pipeline (their kernel replaces the local stage; the coupling stage is shared), so they track SA-Aug on the coupling patterns and differ only on block rows. Second, the negative control exposes a coverage cost of selectivity: rescaling the local block rows while the coupling rows remain at their natural scale widens the exported range by about an order of magnitude on *none* (SA-Aug $1.3 \times 10^3 \rightarrow 1.3 \times 10^4$), a small but statistically significant worsening that full coverage avoids.

The severe heterogeneous cells expose the common-factor reliability failure that motivated the floor of section 4.4. With it disabled, the common-factor variants fail original-scale verification in 29/1800 severe coupling/mixed runs because one small factor fitted to extreme rows also shrinks rows it does not fit. With the floor enabled all 1800 runs pass. The independent-seed replication and full floor sweep are in Table S3; they validate this safeguard for the mechanism check, not a universal threshold.

This is the implementation-level mechanism check anticipated in section 3.6. It verifies that the proxies react to the row-multiplicative disturbances they are designed to remove and that the one-sided coupling stage activates on its intended oversized

TABLE 16 Descriptive postselected Gurobi internal-scaling comparison at 1% MIPGap. Within each class, the internal mode with the lowest class-level solver-time sgm is selected on the same data; SA-Aug and SA-Mat use default internal scaling. Ratios are medians of $T_{\text{internal}}/T_{\text{SA}}$ under the main paired convention: double non-completions are excluded and a one-sided non-completion is assigned $T_{\text{max}} = 3600$ s. No abort occurs in these six comparisons. This same-sample postselection is descriptive, not inferential.

Class	Selected mode	sgm T_{int}	sgm T_{Aug}	sgm T_{Mat}	n_{Aug}	n_{Mat}	Med. ratio Aug	Med. ratio Mat
Simple	aggressive	3.79	2.13	2.17	204	204	0.97	0.96
SG-Ter-Mer	equilibration	14.66	8.28	8.64	67	67	1.18	1.28
Full	equilibration	128.45	95.24	102.03	32	32	1.21	1.25

branch, driving the block-relative proxy to order one and reducing the exported magnitude-range proxy there. It does not establish that the proxies approximate or improve the spectral condition number. The scope of the check is therefore bounded and explicit: the synthetic suite validates the shared per-row normalization kernel under row-multiplicative imbalance, and it exercises the common block-relative coupling factor only on the *oversized* branch of its collective design case; the undersized branch, severe inter-row heterogeneity, and phenomena beyond row-multiplicative imbalance—scale separation between blocks with ill-conditioned interiors, column imbalance, or rows with identical coefficient extremes but different roles—are either untreated by design or not exercised, and are natural extensions of the generator.

Taken together, the four operational scenarios and the controlled synthetic check support a limited but useful interpretation. Under Gurobi, the structured rules lower path-specific numerical diagnostics and often reduce completed-run solver time or deterministic effort (section 6.1 gives the fixed-pool reliability-adjusted readings). Under CPLEX, the same external transformations do not produce a robust runtime benefit and can be indistinguishable from, or worse than, Base in paired runtime evidence even when the reported path diagnostics move favorably. The empirical message is not that one scaling rule always wins, nor that an own-incumbent fixed-LP diagnostic is a pure measure of representation: performance must be measured through the full solver–model interaction rather than inferred from metadata or a single numerical proxy (section 6.8).

6.7 | Solver-internal-scaling baseline

The control of section 5.4 that most directly tests the contribution asks whether the solver’s *own* scaling could reproduce the structure-aware effect. Table 16 summarizes the Gurobi comparison; the complete Gurobi and CPLEX flag sweep is Table S7.

The displayed paired counts and ratios are reproduced by `scripts/analizar_revision.py rlpairs` on each class folder. Disabling Gurobi scaling (`ScaleFlag=0`) is markedly harmful on Simple and SG-Ter-Mer (sgm = 42 and 57 s), confirming that default internal scaling is already a strong baseline. Within each class, however, we postselect the lowest-sgm internal mode on the same observations used for the comparison. This is a descriptive same-sample summary: the sgm-optimal mode need not be the strongest paired competitor, and no inferential claim follows from the displayed ratios.

The answer is class-dependent. On Simple, the selected internal mode matches the external rules on the typical paired instance despite their aggregate-tail difference (median ratios 0.97/0.96). On SG-Ter-Mer, the ratios 1.18/1.28 show a sizable descriptive paired shift in favor of the external rules. Full is directionally favorable at 1% (ratios 1.21/1.25), whereas at 0.1% the tuned internal modes (337–353 s) overtake SA-Aug (388 s). Thus retuning internal scaling does not substitute for external preprocessing in SG-Ter-Mer; it does substitute for it on the typical Simple instance, while the Full result depends on tolerance. Jointly tuning external and internal scaling would require a factorial control and remains future work.

The anomalously fast CPLEX Full cell with internal scaling disabled is not a reliable alternative: only 26/34 runs pass original-scale verification, and several verified solutions remain materially worse than the best verified incumbent. Table S7 contains the complete verifier audit, objective differences, flag sweep, and the partial or absent CPLEX Full cells at 0.1%; those incomplete cells are excluded from the main comparison.

6.8 | Operational coupling-stage activation audit and local-row attribution

The synthetic benchmark isolates local and coupling imbalance by construction. On the real hydrothermal classes, the operational audit runs local-only and coupling-only variants beside the full architecture under Gurobi. Because the coupling stage

TABLE 17 Operational coupling-stage activation audit under Gurobi (1% MIPGap). Counts are over eligible diagnostic/export pairs. No coupling factor is applied, and every full/local-only export pair is byte-identical; the operational transformation therefore reduces to local-row normalization.

Class	Variant	Eligible pairs	Stage activated	Coupling rows rescaled	Identical full/local
Simple	SA-Aug	197	0	0	197/197
	SA-Mat	197	0	0	197/197
SG-Ter-Mer	SA-Aug	67	0	0	67/67
	SA-Mat	67	0	0	67/67
Full	SA-Aug	34	0	0	34/34
	SA-Mat	34	0	0	34/34

never activates, comparing independent runtimes of full and local-only runs cannot estimate a stage effect: those runs solve byte-identical exported matrices. Table 17 therefore reports the mechanism counts; the corresponding runtime replicates are retained only as an audit trail in Table S8.

For SA-Aug, the coupling proxy is $\hat{\rho} \leq 10$ on every operational instance ($\hat{\rho} \approx 2 \times 10^{-4}$ in Simple and exactly 1 in SG-Ter-Mer and Full), so the coupling stage is skipped. For SA-Mat, every per-row coupling factor lies in the near-identity band on the balance rows and is likewise skipped. Coupling-only therefore coincides with Base, while full coincides with local-only. The operational effect comes entirely from local-row normalization; the coupling mechanism is evaluated only in the synthetic benchmark (section 6.6), where it activates.

Because each local kernel is computable without row-role information, the direct control applies it to *every* row, with no roles, blocks, or coupling stage. We implement both missing controls: Flat-Aug uses the coefficient-and-right-hand-side extremes of SA-Aug, whereas Flat-Mat uses the coefficient-only extremes of SA-Mat. Table 18 compares each Flat rule directly with its selective counterpart on the same eligible capped-time pairs, rather than comparing two separate speedups against Base. Prototype preprocessing-plus-solver time is the implementation endpoint observed in the dedicated solve batches; the cached-metadata sensitivity approximates a generator-integrated implementation (section 5.6; the hierarchy was selected after inspecting these data and is exploratory). On that endpoint the observed ordering is the same at both tolerances in the two fast classes: the *implemented Flat preprocessing pipeline* has lower observed preprocessing-plus-solver time in Simple and SG-Ter-Mer under both kernels, with observed median effects of 2.1–5.3% under the original timing protocol; the nominal adjusted values are below 0.05 in all eight contrasts (worst $q_{12}^{\text{blk}} = 0.031$), while the separate randomized replication below confirms the direction—but not the magnitude—of the preprocessing-cost difference. This difference combines the numerical coverage policy with the cost of the current prototype implementation, and should not be interpreted as a pure causal effect of scaling more rows. The reason is concrete: our implementation *rebuilds* the structural metadata on every call—it re-derives the block map from variable-name prefixes and re-classifies every row—rather than reading labels the generator already holds. Instrumenting the call into metadata reconstruction, scale diagnosis and factor application shows that reconstruction is the dominant term for the selective rule, 72–89% of its preprocessing time by class and kernel, while it is negligible for the role-blind rule, which needs no roles at all (≤ 0.6 ms). Charging only the remaining instrumented diagnosis-and-application work—an estimated cached-metadata counterpart $T_{\text{preproc}}^{\text{cached}} = T_{\text{diagnosis}} + T_{\text{application}}$ that removes the prototype’s metadata-reconstruction stage while retaining diagnosis and factor application—collapses the gap substantially. In the non-optimized concurrent stress test the median paired preprocessing ratio falls from roughly 6–9 $\times$ to 1.08–1.40 $\times$; the cleaner pinned diagnostic protocol produces smaller absolute times, and the release-build absolute preprocessing values used in table 19 are reported in Panel E4 of Table S12. It does not, however, reverse the ordering. The cell-level median cached preprocessing time is lower for the role-blind rule in all six class–kernel cells under each protocol, although the record-level direction is not uniform: under the pinned release build—the sensitivity of record—it has the lower per-instance median on 585 of the 612 instance–kernel records (607/612 pinned diagnostic, 408/612 in the noisier concurrent protocol; computed by scripts/preproc_timing_replication.py from the cached_sa_s and cached_flat_s columns of, respectively, results-revision/preproc-timing/replicated_pinned_release.csv, results-revision/preproc-timing/replicated_pinned.csv and results-revision/preproc-timing/replicated.csv). These record-level counts condition on the $R = 20$ per-instance median estimates: they measure the direction of the estimated medians and do not propagate within-instance timing uncertainty. To screen that noise we classify each record by comparing its paired difference against the dispersion measured inside the same instance, calling it a *within-dispersion tie* when $|\tilde{T}_{\text{SA},i} - \tilde{T}_{\text{Flat},i}| \leq B_i$, with B_i the larger of the two policies’

interquartile ranges over the $R = 20$ repetitions (Table S12). This is a descriptive noise screen, not a confidence interval, an equivalence test, or a domain-calibrated practical-significance margin. The classification gives 461 role-blind-favoured records, 1 selective-favoured record and 150 ties under the pinned release build; 554, 0 and 58 under the pinned diagnostic build; and 37, 0 and 575 under the concurrent protocol, whose count is close to a coin flip precisely because almost every record there is measurement-indeterminate. That residual is markedly less uniform than the full preprocessing gap, which held on every instance and every family: aggregating to operational families, the role-blind rule has the lower family median in 20–32 of the 34 families per cell (median family ratio 1.04–1.32), against 34/34 before the correction, so the direction is more consistent in some cells than in others but is not uniform. Most of the measured preprocessing penalty is therefore an artifact of the prototype rebuilding metadata it was given, and what remains is a small *absolute* descriptive residual—typically tens to hundreds of microseconds—although its relative size within the preprocessing call is larger in some cells, where the cached ratio still reaches 1.15–1.19. These are descriptive counts, not a family-level significance statement.

Because that reconstruction is precisely what the proposed architecture assumes away, we recompute the whole paired comparison on the cached-metadata endpoint $T_{\text{pre+solve}}^{\text{cached}} = T_{\text{diagnosis}} + T_{\text{application}} + T_{\text{solver}}$ (table 19). This is a *cross-protocol counterfactual*: it combines preprocessing medians measured on the pinned workstation with solver times from the ClusterUY batches, so it does not represent an observed single-machine pipeline, and its absolute ratios depend on the relative speed of the two environments. Of the eight fast-class contrasts whose prototype preprocessing-plus-solver ordering had adjusted values below 0.05, six keep a direction favoring the role-blind rule but only one retains a within-tolerance nominal adjusted value below 0.05: the augmented Simple contrast at 0.1% ($q_6 = 0.010$), which also survives the joint and operational-family corrections ($q_{12} = 0.021$, $q_{12}^{\text{blk}} = 0.031$) and is small in median magnitude, about 0.9%. It indicates a systematic directional difference in this campaign, not a large operational speedup. The augmented SG-Ter-Mer contrasts fall to $q_6 = 0.115$ and 0.051, and every coefficient-only fast-class contrast to $q_6 \geq 0.115$. The directional reversals occur in SG-Ter-Mer coefficient-only at 1% and Simple coefficient-only at 0.1%; the other six continue to point toward the role-blind rule.

The build matters here, and measurably. Our campaigns were compiled without optimization—the executable used in the original solver campaigns was compiled with the same `-std=c++17 -g` flags as the instrumented diagnostic build—so we repeated the pinned microbenchmark with a release build (`-O2 -DNDEBUG`) and take it as the implementation-cost sensitivity of record. Optimization shrinks every preprocessing time about sixfold and leaves the full-call ratio essentially unchanged ($5.4\text{--}6.1\times$ against $5.1\text{--}6.0\times$), but it moves the cached ratios towards parity, and enough to change one conclusion: under the diagnostic build the coefficient-only Simple contrast at 1% had $q_6 = 0.0002$, whereas under the release build it falls to $q_6 = 0.115$. Splicing release preprocessing onto diagnostic-build solver times is only legitimate if optimization changes the *cost* of preprocessing without changing the transformation it produces, so we verified that directly over every available record: exporting the preprocessed model with both builds for all 306 instances, both kernels and both coverage policies gives 1204 byte-identical exports out of 1224 (`scripts/build_invariance_audit.py`). Every Simple and SG-Ter-Mer record is byte-identical across builds. These are the two fast classes underlying the eight cached contrasts summarized in the abstract and the only retained adjusted result; for them the build change affects preprocessing time but not the exported model, so the comparison is a clean cost substitution. The 20 exceptions are five Full instances (029, 035, 045, 047, 048), each differing under all four variants. The differing entries are the nonlinear-hydro linearization coefficients, and they differ by one unit in the last place (4.7×10^{-16} relative, in 720 of 47,173 coefficients on a representative instance); the audit therefore localizes the discrepancy to optimization-dependent floating-point evaluation during model construction rather than to a change in the selected scaling factors or safeguard decisions. Two of the five (029, 035) fall inside the analysed Full pools, so the Full cached rows recombine a release preprocessing time with a solver time recorded on a model that differs at the last bit. This build comparison provides one observed policy-specific *toolchain* perturbation rather than a direct estimate of the workstation-to-ClusterUY transfer ratio: the implied ratio $\rho = \lambda_{\text{SA}}/\lambda_{\text{Flat}}$ remains in $[0.94, 1.01]$ across the six class–kernel cells, suggesting limited asymmetry under this particular compiler change, while the cross-environment transfer asymmetry itself remains unmeasured.

Because the cached effects sit close to one, and because they are a cross-protocol construction, we repeat the common- and policy-specific transfer sensitivities using the release-build preprocessing measurements that define table 19; the non-optimized diagnostic build is retained separately as a build sensitivity in Table S11. Writing $T^{\text{cached}}(\lambda) = T_{\text{solver}} + \lambda T_{\text{preproc}}^{\text{cached}}$, a common factor varies the overall weight of preprocessing but does not model policy-specific hardware or compiler effects: the two rules execute different code, so a change of machine or toolchain need not rescale them by the same factor. On the evaluated common-factor grid $\lambda \in \{0.25, 0.5, 1, 2, 4, 8\}$, one fast-class contrast has $q_6 < 0.05$ for $\lambda \leq 2$, and two do at the evaluated points $\lambda = 4$ and 8; the augmented Simple contrast at 0.1% has $q_6 < 0.05$ at every evaluated grid point. On the policy-specific grid $\rho = \lambda_{\text{SA}}/\lambda_{\text{Flat}} \in \{0.25, 0.5, 1, 2, 4\}$ with $\lambda_{\text{Flat}} = 1$, the corresponding counts are one for $\rho \leq 1$, three at $\rho = 2$ and five at $\rho = 4$.

Additional q_6 crossings therefore first appear at the evaluated point $\rho = 2$, but the transition threshold between 1 and 2 is not identified by this grid. These counts concern within-tolerance q_6 only and do not imply retention under the joint or operational-family adjustments. The λ and ρ grids are exploratory log-spaced stress ranges, chosen to vary the relative preprocessing contribution by factors of two over a deliberately broad interval; they are not confidence regions or calibrated estimates of the workstation-to-ClusterUY transfer. The contrasts crossing at each grid point are identified in Table S12. Complete contrast-by-grid values are provided in `results-revision/preproc-timing/cached_transfer_sensitivity.csv` in the reproducibility package.

The absolute magnitudes matter for how much this is worth (table 20). In Simple the median paired preprocessing-plus-solver margin is 0.051 s at 1% and 0.067 s at 0.1%; in SG-Ter-Mer, 0.041 and 0.070 s. In other words, the 2–5% effects in the fast classes are *tens of milliseconds per instance*, and they are decisive only because these runs are themselves short. Full is the opposite regime: its median preprocessing difference is the same 0.08 s, but its median solver difference is –13.4 s, three orders of magnitude larger, which is why its sign reverses.

Attribution across the two components needs care, because these are marginal summaries and the median of a sum is not the sum of medians. Read as marginal summaries, most of the typical Simple difference is associated with preprocessing (0.051 against a 0.067 s total at 0.1%) and rather less of the SG-Ter-Mer one (0.032 against 0.070 s). Read additively on the same pool, where means *do* decompose, the picture inverts: the mean paired differences at 0.1% are +0.052 s (preprocessing) versus +8.35 s (solver) in Simple and +0.032 versus +13.76 s in SG-Ter-Mer. The two readings disagree because the paired solver differences are heavy-tailed—a handful of instances differ by seconds in either direction, while the typical instance differs by milliseconds—whereas the preprocessing difference is a small, almost constant positive offset. The component summaries suggest that this nearly constant offset affects many near-parity pairs, and a direct sign-and-rank decomposition on the estimand itself quantifies how much of the signed-rank result it carries. Going from the solver-time variable d_i^{solver} to $d_i^{\text{pre+solve}}$ at 0.1% flips the sign of 41 of 204 Simple pairs and 9 of 67 SG-Ter-Mer pairs; the count favoring Flat rises from 122 to 163 and from 36 to 45; the rank-biserial effect moves from –0.253 to –0.536 and from –0.313 to –0.507; and the Hodges–Lehmann ratio moves from 1.0135 to 1.0359 and from 1.1229 to 1.1577. So adding preprocessing does not merely shift a summary: it converts a large minority of near-parity solver-time pairs into Flat-favoring pairs, which is where the strengthening of the rank statistic comes from. A per-instance share is not a useful summary here either: writing $s_i = \Delta T_{\text{preproc},i} / \Delta T_{\text{pre+solve},i}$, the two components have opposite signs on 40% of Simple and 46% of SG-Ter-Mer pairs, so s_i falls outside $[0, 1]$ there and its median (0.35 and 0.00) should not be read as a fraction explained. In Full the direction reverses: in the 27 eligible pairs at 0.1% the selective augmented implementation has the lower observed time (median ratio 0.878, $q_{12}^{\text{pre+solve}} = 0.025$), but the result is conditional on those pairs. Four nominal instances (045–048) were never recorded, and an *adversarial signed-rank sensitivity* for those pairs—appending them with the adverse sign at above-maximal ranks, because multiplicity adjustment does not address missing-data selection—does not retain the contrast ($p = 0.47$). That construction bounds the signed-rank evidence under adverse missing pairs; it is not a universal worst-case bound for the median ratio, the Hodges–Lehmann estimate, or the reliability-adjusted aggregate costs. Because adverse above-maximal ranks do not uniquely determine the missing log-ratio magnitudes, we report no universal adversarial median or Hodges–Lehmann estimate; under the single imputation actually used— $d_{\text{miss}} = -(\max_i |d_i| + \varepsilon)$, one adversarial scenario among many—the location estimates still point toward selectivity, which shows only that what the sensitivity removes is the rank-based evidence rather than the sign of the point estimate. The three double timeouts enter as operational ties. At 1% the same Full direction is inconclusive. On the capped solver-time decomposition—the solver’s response to the exported matrix, reported as an exploratory reading—no 1% contrast remains below 0.05 after adjustment; at 0.1% the augmented Simple contrast favors full coverage with adjusted values below 0.05 throughout ($q_6 = 0.010$, $q_{12} = 0.021$, $q_{12}^{\text{blk}} = 0.032$), the augmented SG-Ter-Mer contrast is nominal ($p = 0.026$, $q_6 = 0.052$), and the augmented Full contrast is below 0.05 within tolerance ($q_6 = 0.045$) but not jointly ($q_{12} = 0.090$). The solver-time effects are small—median 0.9% and 1.6% in Simple and SG-Ter-Mer, with a family-bootstrap interval that includes one in SG-Ter-Mer. The Work-unit comparison reproduces both augmented 0.1% solver-time directions ($q_6^{\text{blk}} = 0.036$ and 0.006; Table S10), making pure wall-clock scheduling noise a less likely explanation; it does not eliminate seed- or trajectory-level variability, which would require replicated runs. None of the six matrix-only solver-time contrasts is conclusive. The Flat controls were not run under CPLEX, so cross-solver persistence of this ordering is untested.

Exporting the scaled matrices identifies the mechanism precisely, and it is not the coupling treatment. In SG-Ter-Mer the model has 2160 global rows: 360 demand-balance (coupling) rows and 1800 that are global but *not* coupling—chiefly the market shortage-segment rows, whose coefficients span the segment marginal costs (here up to 4000) against a unit shortage-penalty term, with right-hand sides of order 10^5 – 10^7 . The balance rows are already well balanced relative to the blocks ($\hat{\rho} \approx$

1.1), so the coupling stage correctly leaves them unscaled, and the flat kernel leaves them unscaled too (its factor falls in the near-identity band): the coupling rows are *not* where the two rules differ, and the exported balance rows are byte-identical under both. The 1800 shortage rows, however, fall outside both stages of the structure-aware rule—they are neither local-block rows nor coupling rows—so it never scales them, leaving coefficients of order 10^3 and right-hand sides of order 10^6 in the matrix. The flat kernel, scaling every row by $1/\sqrt{M_r m_r}$ over its coefficients *and* right-hand side, geometrically centers their smallest and largest nonzero magnitudes around one, substantially reducing each such row’s scale mismatch; on one SG-Ter-Mer instance that lowers the exported coefficient-and-right-hand-side range ratio—defined as $\max\{|a_{rj}|, |b_r|\} / \min^+\{|a_{rj}|, |b_r|\}$ over nonzero magnitudes, i.e. the ratio of [equation \(44\)](#) extended to include the right-hand side—from 3.7×10^{11} to 1.3×10^{10} , and it is the only difference between the two scaled matrices. In SG-Ter-Mer the structure-aware decomposition into local and coupling roles left a coverage gap over exactly the highest-magnitude rows, and the role-blind augmented kernel closes it without metadata; the implemented Flat pipeline has the lower observed preprocessing-plus-solver time at both tolerances, while the solver-time contrast favors Flat at 0.1% only nominally. The coefficient-only solver-time contrast remains neutral. (On the synthetic benchmark the taxonomy leaves no row unclassified, so there is no coverage gap of this kind; the differences there trace to the coupling-stage design instead, [section 6.6](#).) In Full the augmented contrast points toward selectivity in the 27 eligible pairs at 0.1%, but it is not retained by the adversarial signed-rank sensitivity over the unrecorded pairs, and the solver-time direction clears only the within-tolerance adjustment. Exporting its matrices shows that the distinction is not a different row family: the flat kernel scales exactly the same shortage-segment and generation-versus-demand rows as in SG-Ter-Mer, and no rows specific to the nonlinear-hydro Full model (verified by differencing the exported constraints on two instances). The matrix change is thus qualitatively identical across the two classes, while the observed runtime point estimate changes sign. This suggests, but does not establish, a solver-dynamics interaction. In Full those shortage rows are no longer the range-dominating extremes—the nonlinear-hydro height and thermal structure absent from SG-Ter-Mer sets the coefficient range—so centering them is a smaller part of the whole. Our static matrix analysis cannot resolve the runtime mechanism; the direct tests show that neither blanket coverage nor role selectivity can be presumed across classes or kernels. The structure-aware rules lower the solver-facing conditioning diagnostics ([table 9](#)), but on these operational classes that gain comes from the *local* stage: in the activation-audit runs the coupling-only variant leaves the solver-level conditioning ratio at one (recorded in the reproducibility artifacts; it is inert in both conditioning and runtime here), while the coupling stage’s ability to repair a *collective* displacement of the coupling rows is what the controlled synthetic coupling-uniform pattern demonstrates ([section 6.6](#)). The coupling metadata drives neither the conditioning improvement nor the runtime gain on the operational testbed. The row-role metadata, used as a *selection filter* for the per-row kernel, has no uniform runtime effect. In these exploratory controls, the orderings that remain below 0.05 after the joint multiplicity and operational-family sensitivities are: on the preprocessing-plus-solver endpoint, the implemented Flat pipeline in Simple and SG-Ter-Mer (both kernels and tolerances; the Full selectivity direction is conditional on its eligible pairs and is not retained by an adversarial signed-rank sensitivity); on the solver-time decomposition, only the augmented 0.1% Simple contrast—and its strict/mixed reliability-adjusted PAR10 ordering points in the opposite direction ([table 21](#)), so neither result should be read as a generally superior operational policy ([section 7](#)). The own-incumbent diagnostic in Table S10 sharpens the dissociation but does not isolate representation: the selective and Flat runs fix different integer solutions whenever their solver paths produce different incumbents.

Because the reliability-adjusted audit changes conclusions, its central numbers belong in the body ([table 21](#)). In Simple, Flat-Aug wins intent-to-treat PAR10 (30 versus 212 s, driven by one SA-side timeout), but it also fails the strict original-scale criterion on more completed runs (25 versus 22; 22 versus 18 under the mixed reading), so both reliability-adjusted readings reverse the aggregate ordering toward SA-Aug. In SG-Ter-Mer neither policy has a mixed failure and Flat-Aug retains a small descriptive advantage under all three readings. In Full the selective rule’s paired preprocessing-plus-solver advantage does not persist in the aggregate readings: SA-Aug is slightly ahead under intent-to-treat, but the mixed reading favors Flat-Aug (5109 versus 6342 s, one rescued failure), a further reason not to read the Full direction as established. For the four Matrix-only Simple and SG-Ter-Mer contrasts whose preprocessing-plus-solver ordering has adjusted values below 0.05, the strict and mixed common-pool PAR10 readings do *not* materially change the ordering. The two policies record identical failure counts in each of those cells (19/18 in Simple at 1%, 30/26 at 0.1%, and 1/0 in SG-Ter-Mer at both tolerances). The directly recomputed strict and mixed values also differ by at most a few seconds—in Simple, 3355.1/3178.7 (SA) against 3355.1/3178.6 (Flat) at 1% and 5491.9/4787.2 against 5492.1/4787.4 at 0.1%; in SG-Ter-Mer, 1083.7/555.0 against 1080.8/551.6 at 1% and 1085.6/556.5 against 1082.0/552.8 at 0.1%—and no reversal of the kind seen in Simple-Augmented occurs; the full audit is Table S9. No reversal here is a paired test; all use the fixed-pool aggregate convention of [table 5](#).

TABLE 18 Direct coverage-versus-selectivity comparison under Gurobi for both per-row kernels: structure-aware (SA) versus role-blind Flat reruns from the same dedicated class–tolerance batch, on the exploratory endpoint hierarchy of Section 5.6. Endpoints: *prototype* preprocessing-plus-solver time $T_{\text{pre+solve}}^{\text{prototype}} = T_{\text{preproc}} + T_{\text{solver}}$, which charges the prototype’s metadata reconstruction, and capped solver time (decomposition). The generator-integrated counterpart is the cached-metadata sensitivity of table 19. The paired variable is $d_i = \log(T_{\text{Flat},i}/T_{\text{SA},i})$; median ratios are SA/Flat, negative r_{rb} favors Flat, and q_6 is the BH value within the six-test tolerance family of each endpoint.

Gap	Class	Kernel	n	Prototype pre+solve time			Solver time (decomposition)		
				med. $\frac{T_{\text{pre+solve}}^{\text{SA}}}{T_{\text{pre+solve}}^{\text{Flat}}}$	r_{rb}	q_6	med. $\frac{T_{\text{solver}}^{\text{SA}}}{T_{\text{solver}}^{\text{Flat}}}$	r_{rb}	q_6
1%	Simple	Augmented	204	1.0292	−0.857	8×10^{-26}	1.0004	−0.086	0.321
	Simple	Matrix-only	204	1.0288	−0.967	3×10^{-32}	1.0009	−0.141	0.173
	SG-Ter-Mer	Augmented	67	1.0531	−0.510	5.7×10^{-4}	1.0074	−0.303	0.173
	SG-Ter-Mer	Matrix-only	67	1.0427	−0.376	0.0113	0.9976	−0.140	0.321
	Full	Augmented	31	0.9260	+0.319	0.1502	0.9193	+0.355	0.173
	Full	Matrix-only	32	0.9963	+0.197	0.3403	0.9866	+0.246	0.321
0.1%	Simple	Augmented	204	1.0320	−0.536	2×10^{-10}	1.0093	−0.253	0.0104
	Simple	Matrix-only	203	1.0210	−0.457	4.9×10^{-8}	0.9980	+0.092	0.381
	SG-Ter-Mer	Augmented	67	1.0473	−0.507	6.3×10^{-4}	1.0159	−0.313	0.0523
	SG-Ter-Mer	Matrix-only	67	1.0383	−0.337	0.0224	1.0003	−0.109	0.526
	Full	Augmented	27	0.8775	+0.513	0.0224	0.8771	+0.529	0.0452
	Full	Matrix-only	27	0.9818	+0.048	0.8408	0.9803	+0.053	0.822

The endpoint hierarchy is exploratory (section 5.6); adjusted values are nominal and define no confirmatory outcome. A one-sided timeout is censored at $T_{\text{max}} = 3600$ s on the solver term, so the intervention cost is still charged: $\tilde{T}_{\text{pre+solve}} = T_{\text{preproc}} + T_{\text{max}}$. Double non-completions are treated as operational ties regardless of their preprocessing-time difference, so the signed-rank analysis prioritizes successful completion over the small intervention-cost difference.

The Full 0.1% rows use the 27 eligible pairs of the 30 recorded instances; the adjusted augmented contrast is not retained by the adversarial signed-rank sensitivity over the four unrecorded instances (section 6.8).

Joint multiplicity, operational-family, bootstrap, Hodges–Lehmann, Work-unit and own-incumbent sensitivities are in Table S10. Attrition, batch timing, preprocessing summaries, build comparisons, transfer-factor analyses and adversarial missing-pair sensitivities are in Tables S11–S12. The reliability audit is in Table S9 and table 21. Reproducible with `scripts/flat_control_tests.py`.

TABLE 19 Cached-metadata sensitivity: the paired comparison of table 18 recomputed on $T_{\text{pre+solve}}^{\text{cached}} = T_{\text{diagnosis}} + T_{\text{application}} + T_{\text{solver}}$, which removes the prototype’s metadata-reconstruction stage. Preprocessing components come from the *release* build (−O2 −DNDEBUG), which is the implementation-cost sensitivity of record; the non-optimized diagnostic build is retained as an audit in Table S11. All Simple and SG-Ter-Mer exported models are byte-identical across the diagnostic and release builds. For Full, five instances exhibit last-bit model-construction differences across builds, including two that enter the analysed pools; the four Full rows (marked †) are therefore approximate cross-build timing sensitivities, whereas the central fast-class comparison changes preprocessing cost only. Median ratios are SA/Flat on the common log-ratio variable, and negative r_{rb} favors Flat. The value q_6 is BH-adjusted within the six-test tolerance family, whereas q_{12} uses the joint twelve-test family. The operational-family adjusted values are reported in Table S11. The analysis conditions on the per-instance median preprocessing times estimated from the low-contention replication, and its adjusted values do not propagate timing-estimation uncertainty. The cached Full-Augmented result at 0.1% uses the same 27 eligible pairs as table 18 and is likewise not retained by the adversarial signed-rank sensitivity over the four unrecorded instances. Exploratory, like table 18.

Gap	Class	Kernel	n	med. SA/Flat	r_{rb}	q_6	q_{12}
1%	Simple	Aug	204	1.0004	−0.090	0.318	0.364
	Simple	Mat	204	1.0011	−0.167	0.115	0.092
	SG-Ter-Mer	Aug	67	1.0075	−0.303	0.115	0.092
	SG-Ter-Mer	Mat	67	0.9976	−0.140	0.318	0.381
	Full†	Aug	31	0.9193	+0.355	0.173	0.173
	Full†	Mat	32	0.9866	+0.246	0.318	0.364
0.1%	Simple	Aug	204	1.0094	−0.253	0.010	0.021
	Simple	Mat	203	0.9981	+0.089	0.410	0.364
	SG-Ter-Mer	Aug	67	1.0159	−0.313	0.051	0.092
	SG-Ter-Mer	Mat	67	1.0008	−0.109	0.526	0.478
	Full†	Aug	27	0.8771	+0.529	0.045	0.090
	Full†	Mat	27	0.9803	+0.053	0.822	0.822

TABLE 20 Completed-pair component summaries from the original fixed-order solve batches associated with [table 18](#): medians of $\Delta T = T^{\text{SA}} - T^{\text{Flat}}$, in seconds. They are not from the order-randomized replication, whose concurrent load yields larger absolute differences. These summaries exclude one-sided non-completions that remain in the capped paired analysis, and therefore do not decompose the exact inferential pool of every row; nor are the three medians an additive decomposition, since the median of a sum is not the sum of medians. A positive value favors Flat. The preprocessing-plus-solver margins in the two fast classes are tens of milliseconds per instance; in Full the solver term dominates and reverses the sign.

Gap	Class	Kernel	n	med. $\Delta T_{\text{preproc}}$	IQR($\Delta T_{\text{preproc}}$)	med. ΔT_{solver}	med. $\Delta T_{\text{pre+solve}}$
1%	Simple	Aug	204	+0.050	0.0005	+0.001	+0.051
	Simple	Mat	204	+0.053	0.0006	+0.002	+0.054
	SG-Ter-Mer	Aug	67	+0.031	0.0007	+0.010	+0.041
	SG-Ter-Mer	Mat	67	+0.033	0.0009	-0.002	+0.030
	Full	Aug	31	+0.080	0.0015	-11.38	-11.30
	Full	Mat	30	+0.084	0.0014	-0.16	-0.08
0.1%	Simple	Aug	203	+0.051	0.0029	+0.016	+0.067
	Simple	Mat	203	+0.054	0.0027	-0.006	+0.049
	SG-Ter-Mer	Aug	67	+0.032	0.0002	+0.038	+0.070
	SG-Ter-Mer	Mat	67	+0.033	0.0004	+0.000	+0.033
	Full	Aug	27	+0.079	0.0008	-13.44	-13.36
	Full	Mat	25	+0.083	0.0010	-1.09	-1.01

TABLE 21 Common-pool reliability readings for the augmented SA–Flat contrasts at 0.1% MIPGap (Gurobi). PAR10 in seconds under the intent-to-treat, strict, and mixed readings of [section 6.1](#), on the capped solver-time cost $c^{(k, \text{solver})}$ (recomputing with the *prototype* preprocessing-plus-solver cost from the original solve batches shifts every cell by less than 0.2 s and changes no ordering); completed-run failure counts ($n_{\text{fail},s}, n_{\text{fail},m}$); timeouts are penalized in all three readings. The Full PAR10 pool contains all 30 recorded instances, including the three double timeouts penalized equally for both policies; those pairs carry no signed-rank direction and are therefore absent from the $n = 27$ inferential pool of [table 18](#). The full twelve-contrast audit is [Table S9](#).

Class	Variant	n	ITT	strict	mixed	($n_{\text{fail},s}, n_{\text{fail},m}$)
Simple	SA-Aug	204	212	4090	3387	(22, 18)
	Flat-Aug	204	30	4437	3908	(25, 22)
SG-Ter-Mer	SA-Aug	68	556	1085	556	(1, 0)
	Flat-Aug	68	542	1071	542	(1, 0)
Full	SA-Aug	30	3951	6342	6342	(2, 2)
	Flat-Aug	30	4004	6291	5109	(2, 1)

7 | DISCUSSION

The experiments support a bounded claim. Structure-aware positive row scaling lowers the reported own-incumbent path-specific fixed-LP diagnostic. Under fixed-pool intent-to-treat PAR10, at least one variant is below Base in each Gurobi class and tolerance. SG-Ter-Mer at 0.1% is the only cell in which both strict-adjusted costs remain below Base, but the SA-Aug margin is negligible, the comparison is descriptive, and the pattern is not replicated at 1%; under the mixed reading SG-Ter-Mer is descriptively below at both tolerances (section 6.1 and table 5). Under CPLEX 22.1.2.0 the same external scalings yield no robust runtime gain. The evidence thus separates algebraic equivalence, path-specific numerical diagnostics, reliability-adjusted aggregate cost, and mixed-integer search performance. The exploratory direct SA-Flat controls, developed in full in section 6.8, add a further separation: the coverage ordering depends on the endpoint, the margins in the fast classes are tens of milliseconds and carry a large prototype-implementation component, and the reliability-adjusted reading can invert the solver-time ordering (table 21; Tables S9–S10).

A first objection is that commercial solvers already scale internally—which is exactly why the scaled variants always run against the solver’s default internal scaling and presolve (the internal-scaling control varies those settings only for the unscaled model). The comparison is not external scaling versus a naive solver, but whether a generator can alter the solver-facing formulation usefully before the solver’s own pipeline begins. The mixed Gurobi–CPLEX outcome is informative about external preprocessing; it does not establish solver dependence of the Flat-versus-selective ordering, because Flat was run only with Gurobi.

A second objection is that any benefit may come from something other than the generator metadata, and it splits into three questions the controls answer separately. First, the choice of *kernel*: the Ruiz baselines swap the geometric-mean kernel for max-norm equilibration inside the same structural pipeline (section 5.3), so they isolate the kernel while holding the selection policy fixed—they do compress coefficient ranges, yet their matched runtime is irregular and does not reliably turn that compression into speed, so the geometric-mean kernel is not interchangeable with a generic one, but this comparison says nothing about the metadata. Second, the choice of *coverage policy*: whether the row-role selection helps is a metadata question, and it is answered only by the two role-blind controls (SA-Aug versus Flat-Aug and SA-Mat versus Flat-Mat, section 6.8). In the reported default-seed campaign, the direct tests answer it endpoint by endpoint. In the prototype preprocessing-plus-solver endpoint, Flat has lower observed time in the two fast classes at both tolerances; under the baseline cross-protocol construction of the cached-metadata sensitivity, which better approximates generator-integrated deployment, the effects move close to parity and only one fast-class contrast retains $q_6 < 0.05$; the number of within-tolerance crossings changes under the hypothetical transfer-factor stress grids of Table S12. The Full selectivity direction is conditional on its eligible pairs under both endpoints, and on solver time alone only the augmented Simple contrast at 0.1% remains below the nominal 0.05 threshold after every reported multiplicity and operational-family adjustment applied to that endpoint, with all matrix-only solver-time contrasts inconclusive. Third, whether the solver could have done this itself: the internal-scaling control (section 6.7) gives a class-dependent answer—a tie on the typical Simple instance, a sizable descriptive paired shift favoring the external rule in SG-Ter-Mer, and a tolerance-dependent outcome in Full. The metadata’s value is therefore attributed strictly to the second comparison, not to the Ruiz contrast, which concerns the kernel alone.

A mechanistic caveat must accompany the conditioning evidence. If better conditioning drove faster search, the best-conditioned variant would be fastest, but the data do not line up that way: the augmented rule improves solver-level conditioning more than the matrix-only rule, yet the matrix-only rule wins runtime more often in Simple and SG-Ter-Mer, and the association between the conditioning ratio and speedup is weak (a small Spearman correlation explaining a few percent of variance). The honest reading is that conditioning is at best a partial mediator: the path-specific diagnostic shift is real, but it is not a pure measure of representation and the runtime effect is plausibly routed through mechanisms it does not capture—which presolve reductions fire, or how many simplex bound-flips and node relaxations the scaled model induces. This reading has precedent: large-scale LP experiments find the runtime effect of scaling irregular even where conditioning improves¹⁵, and the conditioning of the LP relaxations is known to evolve along the branch-and-cut tree in ways a static snapshot of the exported matrix cannot summarize²⁰. We therefore treat the own-incumbent fixed-LP quantity as a pooled description of solver paths, not as evidence of a purer representation or a proven cause of speedup. The role-blind controls (table 18) make the dissociation sharpest: the selective rule has a lower diagnostic in SG-Ter-Mer and Full, yet that diagnostic compares different integer solutions whenever the two runs find different incumbents. It is therefore a representation-plus-path diagnostic, not a controlled common-incumbent comparison. The augmented solver-time comparison favors Flat in the two 0.1% Simple/SG cells (the

latter nominal) and points toward selectivity in Full within tolerance, while the matrix-only solver-time contrasts are all inconclusive. The structure-aware rules do lower the solver-level conditioning ratios below the Ruiz and unscaled baselines; but that gain flows through the local stage, whose kernel is itself role-blind, and the synthetic benchmark bounds the claim exactly: there the full-coverage flat kernel matches or beats every structure-aware variant on the global coefficient-and-right-hand-side magnitude-range proxy. This is the expected oracle-like behavior for complete-row multiplier injections when no safeguard or near-identity band binds; it is not a statement about spectral optimality. The coupling stage’s distinctive repair is confined to the oversized branch of its collective design case (section 6.6). The coupling metadata make the collective-factor intervention *possible*; the benchmark validates that intervention only for oversized uniform coupling displacement, and it does not establish a general conditioning advantage of metadata over full-coverage per-row scaling. The metadata’s operational contribution is a candidate *selection* policy—where to apply the kernel—whose runtime value is supported only in the kernel-, class-, tolerance-, and solver-scoped contrasts of table 18.

A third objection is that the right-hand side may be unnecessary, and the clean evidence comes from the local-only right-hand-side ablation pair in the coupling-stage activation audit, whose members differ *only* in whether b enters the per-row extremes: the coefficient-only version remains close in Simple and SG-Ter-Mer (2.06 vs 2.01 s and 8.83 vs 8.58 s, within 2–3%) and recovers most of the improvement in Full, although the augmented rule retains an approximately 11% solver-time advantage there (78.9 vs 70.9 s). Most of the benefit therefore does not require the right-hand side, with the largest residual advantage of including it in Full. The head-to-head SA-Aug–SA-Mat comparison is consistent with this but confounds the right-hand side with the coupling criterion (section 4.3), so we do not read it as an ablation. Other generated models may differ, but the present gains largely do not require b in the diagnosis.

A fourth objection comes from CPLEX: several transformations improve static or solver-level diagnostics, yet paired runtime evidence stays neutral to unfavorable. This does not weaken the numerical claim; it limits the performance claim. A commercial MIP run is not a linear-algebra experiment—presolve, internal scaling, relaxation bases, cuts, incumbents, branching, heuristics, and tolerances all stand between the exported matrix and the runtime—so external scaling may improve one stage while the binding bottleneck lies elsewhere. We probed the most natural such candidate—that CPLEX’s presolve neutralizes the external scaling—by re-running the Simple class under CPLEX with presolve disabled. In that class the per-instance effect stayed essentially neutral in both regimes (median paired speedup ≈ 1.00 , with the structure-aware variant faster on about half of the 204 instances). Thus presolve does not explain the Simple-class neutrality. No corresponding no-presolve run is available for SG-Ter-Mer or Full, so presolve remains an untested contributor to their CPLEX results.

The methodological lesson: numerical preprocessing for MIPs should be judged on four separable layers—algebraic equivalence, numerical diagnostics, matched runtime, and solver contrast—and never on one coefficient-range statistic or one runtime table. The study reports all four layers and keeps their claims separate.

The controlled synthetic benchmark closes part of the gap: when the source of imbalance is known by construction, the variants respond as intended at the level of the exported coefficient-and-right-hand-side magnitude-range proxy, and with the reliability floor in place original-scale feasibility and objective agreement hold across the grid; the same severe heterogeneous coupling cells expose, with the floor disabled, the numerical-reliability failure mode of the common-factor variants (section 6.6). It does not close the performance gap—the synthetic instances solve in milliseconds—so runtime claims still come from operational instances. The operational coupling-stage activation audit (section 5.4) establishes that the coupling treatment is inactive on these real data and that full exports reduce to local-row normalization. Other domains (logistics, production planning, network design) would further test whether the mechanism persists as the generator structure changes; it should transfer only insofar as it shares the premise of sparse global coupling over local blocks.

7.1 | Deployability and downstream effects

The method presupposes that the metadata it needs—row role, block ownership, and coupling incidence—are available at model-generation time. This is immediate in a generator that the team owns, as here, but it is not free in the mainstream algebraic modeling languages (Pyomo, JuMP, GAMS, AMPL) that many practitioners use, which typically export a flat matrix and do not surface component structure. Adoption therefore divides into two cases. A team that owns its generator can emit the three metadata fields at low cost, since the generator already iterates over components when it builds the rows. A team that does not own its generator would need either to instrument the export path or to *recover* block structure after export, for example by hypergraph or matrix partitioning of the kind used by automatic Dantzig–Wolfe structure detectors²¹; such a fallback widens applicability beyond owned generators at the price of an approximate, detected block map. The practical value proposition is

correspondingly narrow and concrete: for a Gurobi user running many hard instances of a generated model whose structure is known, the method can reduce aggregate runtime and may occasionally rescue a near-limit instance at low absolute but non-negligible preprocessing cost. For repeated hard instances, that cost may be small relative to solver time and may be amortized when structural metadata are cached; on fast instances, however, the measured preprocessing overhead is large enough to determine the preprocessing-plus-solver ordering (section 6.8)—though in absolute terms that ordering is decided by tens of milliseconds per instance (table 20), so it matters for throughput over many short solves rather than for any single run. The amortization is not hypothetical: 72–89% of the selective rule’s preprocessing time in our prototype is spent rebuilding metadata the generator already has, and charging only the remaining instrumented diagnosis-and-application work shrinks the gap against the role-blind rule by nearly an order of magnitude. This quantity is an estimated cached-metadata implementation cost in the measured hardware and toolchain environment, not a hardware-independent intrinsic cost. An implementation that consumes emitted labels instead of re-deriving them would therefore remove most—though, on this testbed, not all—of the measured penalty. The timeout evidence in this study is also not stable across campaigns (the SG-Ter-Mer PAR10 advantage rests on a single timeout-rescued instance that the coupling-stage activation-audit and internal-scaling campaigns do not reproduce). The completed-run shifted geometric mean is less directly driven by the timeout count, but it remains a descriptive solved-subset statistic and can be affected by differential completion—with no benefit demonstrated for CPLEX and no per-instance speedup promised on easy instances.

A second deployment caveat concerns quantities other than the optimal point. If the original row $a_r^\top x \leq b_r$ is multiplied by $d_r > 0$, the dual multiplier the solver reports for the scaled row satisfies $\pi_r^{\text{scaled}} = \pi_r^{\text{original}}/d_r$, so the original-scale value is recovered as

$$\pi_r^{\text{original}} = d_r \pi_r^{\text{scaled}}. \quad (54)$$

Operations workflows that read shadow prices—marginal costs of demand balances, water values, reserve prices—must apply equation (54) with the recorded row factors before interpreting them (the primal variables, by contrast, need no transformation under row-only scaling). Because the row factors are stored for the audit trail, the recovery is mechanical; but it must be applied, or downstream dual interpretation will be off by the per-row scale.

8 | STRENGTHS, LIMITATIONS, AND REPRODUCIBILITY

The study is strongest where the evidence is matched: runtime comparisons use the same instance, solver, and tolerance for base and variants; numerical diagnostics are reported separately from speed; and feasibility and integrality are checked in the original scale. These choices keep the argument auditable—a reader can see which evidence supports algebraic equivalence, which supports numerical improvement, and which supports solver-dependent acceleration.

The main limitation is domain coverage. The method applies to generated modular MIPs with identifiable local blocks and coupling rows, but the evidence comes from one hydrothermal dispatch platform. That platform supplies a realistic and numerically heterogeneous testbed. It does not prove that the same behavior will appear in other generated optimization domains.

A second limitation is that the operational testbed does not exercise the coupling mechanism. The coupling-stage activation audit finds zero activations and byte-identical full/local-only exports in every eligible pair, so it establishes that the observed operational transformation is entirely local-row normalization; it cannot estimate how the coupling stage would affect runtime on real instances where it activates. That marginal mechanism is tested only on the synthetic benchmark, and transferring it to another operational generator remains open.

A third limitation is the choice of the row as the unit of intervention. The method scales complete rows, which keeps the transformation trivially equivalence-preserving and leaves variable bounds and integrality untouched. It does not, however, test the competing hypothesis that *column*-scale heterogeneity—water volumes, flows, costs, and power mixed across variables—is the larger numerical lever in these models. Generic column scaling is included only inside the Ruiz+Cols baseline, and the synthetic generator injects row imbalance by construction, so it cannot, by design, reveal whether a structured *column*-scaling counterpart would help more. The row-only framing is thus a deliberate scoping decision, not an established claim that row imbalance dominates; a structure-aware column-scaling rule is a natural and untested companion to the present work.

Solver dependence is another threat. With internal presolve and scaling left enabled, the measured effect is that of an external layer inside realistic commercial pipelines—relevant to practice, but conditional on solver version, settings, and environment. The negative CPLEX evidence is therefore a finding about the tested pipeline, not a universal statement about CPLEX.

A related threat is run-to-run performance variability. Each instance–variant–scenario cell is measured by a single run under default solver seeds, so the well-documented variability of MIP performance is not averaged out. The repeated Base campaigns provide an empirical cross-campaign drift check, not a statistical bound on seed-level variability: they are separate controls, run in different batches and on different eligible subsets, and they supply only a handful of observed points. Across the observed campaigns—the main scenario, the internal-scaling baseline, and the coupling-stage activation campaign (tables 7 and 16; Table S8)—the Simple base *sgm* spans 3.73–4.05 s and the SG-Ter-Mer base 14.09–14.42 s, a difference of under 10% that is an order of magnitude smaller than the $1.8\text{--}2\times$ structure-aware effects; this contextualizes, but does not estimate, the seed variability of the direct SA–Flat contrasts. The deterministic-effort metrics (*Work* units and ticks) are less sensitive to hardware scheduling and wall-clock noise, but they can still vary when the solver follows a different seed- or parallel-search trajectory. The Full class drifts more (base *sgm* 82.5–109.4 s across the same campaigns): with 34 instances and heavy censoring, its solved-subset geometric mean is fragile, which is why every Full conclusion in this paper rests on within-campaign matched pairs rather than on cross-campaign level comparisons. The repeated Base campaigns contextualize the larger Base–SA effects but do not estimate seed variability for the smaller direct SA–Flat contrasts. Each dedicated control CSV contains both SA and Flat reruns from the same class–tolerance batch, and those batches are independent of the main campaign. No seed-replicated {SA-Aug, Flat-Aug} $\times$ {SG-Ter-Mer, Full} subcampaign is available. The nominal SG-Ter-Mer result is therefore scoped to the reported default-seed campaign, and the Full direction and magnitude must not be treated as stable. Moreover, the operational scenarios share generator structure and form identifiable Simple and SG-Ter-Mer families; treating all rows as independent can overstate precision. On solver time, the block-sign sensitivity retains Simple but not SG-Ter-Mer after adjustment; the prototype preprocessing-plus-solver and Work endpoints do retain SG-Ter-Mer (Table S10), whereas the cached-metadata sensitivity moves the SG-Ter-Mer comparisons to or above the nominal 0.05 threshold. The Work replication of both augmented 0.1% directions makes pure wall-clock scheduling noise a less likely explanation, but it does not eliminate seed- or trajectory-level variability; seed-replicated runs are left as future work. The cached-metadata comparison is toolchain-sensitive at the scale of the reported effects: compiler optimization changes the Simple coefficient-only 1% adjusted value from $q_6 = 0.0002$ in the diagnostic build to $q_6 = 0.115$ in the release build. We therefore use the release build as the implementation-cost sensitivity of record and interpret all near-parity cached contrasts cautiously. The full export audit also shows that build invariance is not exact everywhere: five Full instances construct last-bit-different nonlinear-hydro coefficients under optimization-dependent floating-point evaluation, so for those the release preprocessing time is measured on a model that differs from the archived one at the level of floating-point rounding. The two fast classes, which decide the cached comparison, are exactly invariant. The prototype preprocessing-plus-solver endpoint is built from calls that the dedicated batches timed once each in a fixed order; the separate order-randomized replication (section 6.8) confirms the *direction* of the preprocessing gap on all 306 instances but also shows its absolute size to be load-dependent, so those magnitudes—though not their sign—remain protocol-specific. The adjusted Full result is conditional on the 27 eligible observed pairs: the adversarial signed-rank sensitivity over instances 045–048 does not retain it, because multiplicity adjustment does not address missing-data selection.

The numerical diagnostics have limited reach: exported coefficient ranges are static matrix summaries, and solver-level conditioning depends on the state the solver exposes and on auxiliary fixed-integer LP computations; neither measures full MIP difficulty. The paper therefore reports original-scale feasibility, integrality in original variable units, objective checks, performance profiles, deterministic effort, paired tests, and time-averaged relative-gap progress as complementary evidence.

A related limitation is that the selective rule carries no global acceptance test: it can degrade an already well-scaled representation even when every individual factor passes the per-factor safeguards, as the synthetic negative control shows (SA-Aug widens the exported range about tenfold on *none*, table 15). An acceptance rule—scale provisionally, recompute the diagnostic, keep the transformation only if it does not worsen—would prevent this by construction, but it is not innocuous to bolt on: keyed to the exported range, it would have rejected the scaling on roughly a third of the operational structured runs, many of which nevertheless ran faster, precisely because the exported range is a weak predictor of runtime (table 11). Designing an acceptance diagnostic that tracks the runtime-relevant effect is left open.

Finally, the experiments use operational data whose raw inputs may not be fully redistributable, so the reproducibility package includes public code, aggregate results, paired-instance identifiers, processed logs, and rebuild scripts. The controlled synthetic benchmark (sections 5.2 and 6.6) is an auditable complement when raw data cannot be released: it reproduces the numerical mechanism without claiming operational speedup.

9 | CONCLUSION

Generated MIPs contain useful structure before export—local rows, component blocks, and coupling rows—that the solver usually never sees beyond the flat matrix. This paper uses that pre-export information to choose positive row scalings that preserve the optimization problem but change its numerical representation.

The answer is not uniformly affirmative. Structure-aware scaling lowers the reported own-incumbent path-specific fixed-LP diagnostic on the tested hydrothermal MIPs. Under fixed-pool intent-to-treat PAR10, at least one structure-aware variant is below Base in each Gurobi class and tolerance, whereas under CPLEX 22.1.2.0 there is no robust gain. SG-Ter-Mer at 0.1% is the only cell in which both strict-adjusted costs are below Base; the SA-Aug margin is negligible, this is a descriptive aggregate ordering, and it is not replicated at 1%. The diagnostic shift is a description of solver paths: because each formulation fixes its own incumbent, it is neither a controlled representation-only measure nor a proven instance-level predictor.

The contribution divides along the same line the evidence does. The local–block–coupling architecture—scale local rows first, summarize blocks second, scale coupling rows last with the scales of the blocks they connect—uses information the generator has but a flat-matrix routine may not, and, because each factor multiplies a complete constraint by a strictly positive scalar, feasible points, objective values, and integer restrictions are unchanged. On the *numerical* side, the structure-aware rules lower the path-specific diagnostic below the unscaled and Ruiz-kernel formulations on the operational testbed. The direct Flat batches add a different comparison: their own-incumbent diagnostics are equal in Simple, while the selective values are lower in SG-Ter-Mer and Full even when runtime does not follow that ordering. Separately, on the synthetic benchmark the full-coverage Flat rule matches or beats every structure-aware variant on the exported global coefficient-and-right-hand-side magnitude-range proxy, and the coupling stage’s distinctive repair is confined to its collective design case. No spectral condition number is measured there, so this supports no claim of spectral optimality or a general conditioning advantage of metadata over full coverage. On the *runtime* side, the operational activation audit establishes that the coupling stage is inactive on the hydrothermal instances and that the exported transformation coincides with the local-only rule; the operational effect therefore comes entirely from local-row normalization. The direct role-blind controls (table 18) show that coverage and selectivity cannot be generalized across the two kernels or across endpoints. These exploratory single-default-seed controls are developed in full in section 6.8; their lesson is that the ordering depends on what one measures—prototype preprocessing-plus-solver time, where the selective implementation pays mainly for reconstructing metadata, against the cached-metadata sensitivity, where only diagnosis and factor application are charged, against solver time alone. Under the baseline cross-protocol construction, removing metadata reconstruction moves the fast-class effects close to parity: six of the eight directional orderings remain, but only one small Simple contrast retains nominal adjusted values below 0.05 under the reported joint and operational-family sensitivities, where the effects are small and the reliability-adjusted reading can point the other way. The observed effect of external preprocessing differs between the tested Gurobi and CPLEX pipelines, while solver dependence of the Flat-versus-selective coverage ordering remains untested because the Flat controls were run only under Gurobi. Establishing this bounded result required separating four kinds of evidence—algebraic equivalence, original-scale feasibility, solver-facing diagnostics, and matched per-solver runtime—which is as much a methodological takeaway as the scaling rule itself: the rule with the lowest reported numerical diagnostic need not be the rule that solves fastest, and only matched per-solver runtime can distinguish the computational ordering.

Future work should test the mechanism beyond hydrothermal dispatch—larger synthetic suites and applications in logistics, production planning, and network design would show whether the pattern persists as the engineering structure changes—and should trace, inside the solver, why the *same* augmented rescaling that closes the SG-Ter-Mer coverage gap points in the opposite runtime direction in Full (a direction that is conditional on the eligible pairs and is not retained by the adversarial signed-rank sensitivity, so the solver-dynamics reading remains a hypothesis). The reported SG-Ter-Mer campaign shows that a selective taxonomy can miss important rows. The Full campaign raises—but does not establish—the complementary hypothesis that blanket normalization can overshoot on integrated models. A practical scaling layer should therefore measure both coverage policies, exactly as the role-blind control does here, and should time its own preprocessing, which on fast instances can decide the preprocessing-plus-solver ordering.

AUTHOR CONTRIBUTIONS

Mathias Rodríguez Castro: Conceptualization, Methodology, Software, Investigation, Formal analysis, Data curation, Visualization, Writing – original draft, Writing – review and editing.

ACKNOWLEDGMENTS

The author acknowledges Universidad de la República and the computational resources used for the experimental evaluation. The experiments presented in this paper were carried out using ClusterUY (<https://cluster.uy>)¹⁷. The author thanks Claudio Risso and Ignacio Ramírez for supervision, methodological discussions, and feedback during the development of this work. The MIP formulation used in this study belongs to a broader line of work on hydroelectric and hydrothermal dispatch models developed by Claudio Risso and collaborators.

FUNDING

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

CONFLICTS OF INTEREST

The author declares no conflicts of interest.

ETHICS STATEMENT

This study consists of computational experiments and does not involve human participants, animal subjects, patient data, or clinical-trial data.

DATA AVAILABILITY STATEMENT

The reproducibility package for this study is publicly available at <https://github.com/MathiasRodriguezCastro/structure-aware-row-scaling> and archived on Zenodo at <https://doi.org/10.5281/zenodo.20648950>. The repository contains the cleaned experimental source code, preprocessing scripts, all redistributable dispatch input instances and the synthetic benchmark instances, the aggregate result tables and processed representations required to reproduce the reported aggregates, paired-instance identifiers, and the scripts used to rebuild the reported tables and figures. Some original raw operational inputs cannot be redistributed; the affected runs are documented through paired identifiers, processed logs, and aggregate outputs, so their figures reproduce from the package even where the raw input is withheld. Large per-instance solver logs and bulky dispatch CSVs are distributed as external archives linked from the repository's `artifacts/README.md`. The repository also documents the solver versions, scenario definitions, command-line options, time limits, MIPGap settings, preprocessing flags, and log-parsing rules used to construct the matched comparisons. The aggregate results were produced by the original ClusterUY runs, while the released C++ code is a cleaned reproducibility version containing the monolithic MIP path and preprocessing methods used in the paper.

SUPPORTING INFORMATION

A Supporting Information document accompanies this article. Section S1 contains the proofs of Propositions 1 and 2. Section S2 contains the supplementary computational results: the 0.1%-MIPGap Gurobi and CPLEX distributional, diagnostic, and performance-profile views (Figures S1–S4) and twelve tables—hyperparameter sensitivity (Table S1), the per-variant reliability and PAR10 audit (Table S2), the coupling- and local-floor checks (Tables S3–S4), uniformly unavailable instances and full-pool sensitivity (Table S5), paired-test attrition (Table S6), the available internal-scaling sweep and verifier audit (Table S7), activation-audit runtime replicates (Table S8), the direct SA–Flat original-scale reliability audit (Table S9); multiplicity, operational-family, bootstrap, preprocessing-plus-solver, Work-unit and own-incumbent-diagnostic sensitivities (Table S10); per-contrast attrition and the cached-metadata endpoint on the release and diagnostic builds (Table S11); and the measurement environment, build comparison and cross-environment transfer-factor sensitivity (Table S12). Additional processed logs and rebuild scripts are available in the reproducibility repository.

References

1. Risso C, Nesmachnow S, Porteiro R, Vignolo JM, Sierra E, Ibarburu M. A Mixed Combinatorial Optimization Model for the Río Negro Hydroelectric Complex. In: Springer Proceedings in Energy. Springer Nature Singapore. 2025; Singapore:403–420. Presented at ICSC-Cities 2024, San Carlos, Costa Rica
2. Risso C, Nesmachnow S, Porteiro R, Vignolo JM, Sierra E, Ibarburu M. A Mixed Combinatorial Optimization Model to Manage the Hydrothermal Dispatch for the Uruguay River Hydroelectric Plant. In: ICSC-Cities. 2025; Puebla, México. Preproceedings paper; the Springer Proceedings in Energy volume is in press. Available at <https://www.colibri.udelar.edu.uy/jspui/handle/20.500.12008/53524>, accessed July 4, 2026.
3. Rodríguez Castro M, Montaña Cobas ME, Zampetti Garin E. Implementación de Modelo de Despacho Eléctrico para el Complejo Hidroeléctrico del Río Negro. undergraduate thesis. Universidad de la República, Facultad de Ingeniería, Instituto de Computación. Montevideo, Uruguay: 2025. Available at <https://hdl.handle.net/20.500.12008/53439>, accessed July 10, 2026.

4. Wood AJ, Wollenberg BF, Sheblé GB. *Power Generation, Operation, and Control*. Hoboken, NJ: Wiley. 3rd ed., 2013.
5. Conejo AJ, Carrión M, Morales JM. *Decision Making Under Uncertainty in Electricity Markets*. New York, NY: Springer, 2010
6. Vielma JP. Mixed Integer Linear Programming Formulation Techniques. *SIAM Review*. 2015;57(1):3–57. doi: 10.1137/130915303
7. Risso C, Nesmachnow S, Vignolo M, et al. An Enriched Mixed Combinatorial Optimization Model to Manage the Hydrothermal Dispatch for the Río Negro Hydroelectric Complex. *Programming and Computer Software*. 2026;52(3):180–201. doi: 10.1134/S0361768826700209
8. Klotz E, Newman AM. Practical Guidelines for Solving Difficult Linear Programs. *Surveys in Operations Research and Management Science*. 2013;18(1–2):1–17. doi: 10.1016/j.sorms.2012.11.001
9. Klotz E, Newman AM. Practical Guidelines for Solving Difficult Mixed Integer Linear Programs. *Surveys in Operations Research and Management Science*. 2013;18(1–2):18–32. doi: 10.1016/j.sorms.2012.11.002
10. Tomlin JA. On Scaling Linear Programming Problems. *Mathematical Programming Studies*. 1975;4:146–166. doi: 10.1007/BFb0120718
11. Curtis AR, Reid JK. On the Automatic Scaling of Matrices for Gaussian Elimination. *IMA Journal of Applied Mathematics*. 1972;10(1):118–124. doi: 10.1093/imamat/10.1.118
12. Sinkhorn R, Knopp P. Concerning Nonnegative Matrices and Doubly Stochastic Matrices. *Pacific Journal of Mathematics*. 1967;21(2):343–348. doi: 10.2140/pjm.1967.21.343
13. Ruiz D. A Scaling Algorithm to Equilibrate Both Rows and Columns Norms in Matrices. Tech. Rep. RAL-TR-2001-034, Rutherford Appleton Laboratory; Oxfordshire, United Kingdom: 2001.
14. Achterberg T, Bixby RE, Gu Z, Rothberg E, Weninger D. Presolve Reductions in Mixed Integer Programming. *INFORMS Journal on Computing*. 2020;32(2):473–506. doi: 10.1287/ijoc.2018.0857
15. Elble JM, Sahinidis NV. Scaling Linear Optimization Problems Prior to Application of the Simplex Method. *Computational Optimization and Applications*. 2012;52(2):345–371. doi: 10.1007/s10589-011-9420-4
16. Dolan ED, Moré JJ. Benchmarking Optimization Software with Performance Profiles. *Mathematical Programming*. 2002;91(2):201–213. doi: 10.1007/s101070100263
17. Nesmachnow S, Iturriaga S. Cluster-UY: Collaborative Scientific High Performance Computing in Uruguay. In: Torres M, Klapp J, eds. *Supercomputing*. 1151 of *Communications in Computer and Information Science*. Springer. 2019; Cham:188–202
18. Gurobi Optimization LLC. Gurobi Optimizer Reference Manual, Version 13.0.1. <https://www.gurobi.com/documentation/>; 2026. Accessed July 4, 2026.
19. IBM Corporation. IBM ILOG CPLEX Optimization Studio Documentation, Version 22.1.2.0. <https://www.ibm.com/docs/en/icos>; 2026. Accessed July 4, 2026.
20. Miltenberger M, Ralphs T, Steffy DE. Exploring the Numerics of Branch-and-Cut for Mixed Integer Linear Optimization. In: *Operations Research Proceedings 2017*. Springer 2018; Cham:151–157
21. Bergner M, Caprara A, Ceselli A, et al. Automatic Dantzig–Wolfe Reformulation of Mixed Integer Programs. *Mathematical Programming*. 2015;149(1–2):391–424. doi: 10.1007/s10107-014-0761-5